



**Conception, développement et évaluation d'un assistant conversationnel RAG
en français pour les démarches administratives du canton de Genève**

Travail de Bachelor réalisé en vue de l'obtention du Bachelor HES

par :

Daniel GARCIA

Conseiller au travail de Bachelor :

Frédéric Mencier

Genève, le 31 août 2026

Haute École de Gestion de Genève (HEG-GE)

Filière Informatique de gestion

Déclaration

Ce travail de Bachelor est réalisé dans le cadre de l'examen final de la Haute école de gestion de Genève, en vue de l'obtention du titre de Bachelor of Science HES-SO en Informatique de gestion.

J'atteste avoir réalisé seul le présent travail, sans avoir utilisé des sources autres que celles citées dans la bibliographie.

Néanmoins, je déclare avoir eu recours à l'outil Claude (Anthropic, Claude Fable 5, 2026) comme assistant à la rédaction : correction orthographique et grammaticale, formulation et reformulation de phrases, mise en forme du texte et amélioration du style et de la fluidité du texte, ainsi que la vérification de la cohérence structurelle du document. J'ai également eu recours à Claude Code (Anthropic) comme assistant au développement du prototype : aide à la génération et correction de code sous mes spécifications, aide à la mise au point des scripts et de la configuration de déploiement. L'architecture, les choix techniques et leur validation restent entièrement les miens, chaque décision étant consignée dans le journal des décisions du dépôt public du projet, et le code a été relu, exécuté et vérifié par mes soins. L'ensemble du contenu, les idées, les analyses ainsi que l'ensemble des choix intellectuels sont le fruit d'un travail intellectuel et d'une réflexion personnelle. Les sources citées ont été consultées directement et les données chiffrées proviennent des documents originaux. La responsabilité du contenu de ce document incombe entièrement à l'auteur.

J'ai envoyé ce document par email à l'adresse remise par mon directeur de mémoire afin qu'il l'analyse à l'aide du logiciel de détection de plagiat COMPILATIO.

J'accepte, le cas échéant, la clause de confidentialité. L'utilisation des conclusions et des recommandations formulées dans le travail de Bachelor, sans préjuger de leur valeur, n'engage ni la responsabilité de l'auteur, ni celle du ou de la conseiller-ère au travail de Bachelor, celle du juré ou celle de la HEG.

Remerciements

Je remercie d'abord Frédéric Mencier d'avoir accepté de suivre ce travail de Bachelor. Sa disponibilité et la confiance qu'il m'a accordée en acceptant ce sujet m'ont permis de mener le projet avec une grande liberté, et ses remarques constructives ont directement contribué à améliorer le résultat final.

Je tiens également à remercier la HEG-Genève et l'ensemble de son corps enseignant. Je mesure tout ce que ces années de formation m'ont apporté : des bases théoriques solides, des compétences pratiques, et une manière de conduire un projet du début à la fin, sans lesquelles ce projet n'aurait pas pu voir le jour. Ce travail est d'ailleurs, en grande partie, l'occasion de réunir et de mettre concrètement en application ce que ces années d'études m'ont appris.

Mes remerciements vont enfin à ma famille et à mes proches, dont les encouragements constants ont été une véritable source de motivation tout au long de ce parcours. Je pense tout particulièrement à ma compagne, Linda Gotti, qui partage mon quotidien et qui m'a soutenu sans réserve dans mon choix de reprendre des études à un âge où l'on n'en prend plus toujours le risque. Sa patience durant mes longues semaines de travail, sa confiance dans les moments de doute et sa présence de tous les jours ont énormément compté. Sans elle, rien de tout cela n'aurait été possible.

Résumé

Trouver une information administrative en ligne reste difficile pour beaucoup de citoyens : l'information officielle existe, mais elle est dispersée sur des milliers de pages et rédigée dans une langue administrative. Ce travail conçoit, développe et évalue Ge-Trouve, un assistant conversationnel qui répond en français courant aux questions sur les démarches administratives du canton de Genève, en s'appuyant sur les seules pages officielles et en citant ses sources. Il repose sur la génération augmentée par la recherche (RAG) et sur une contrainte de conception choisie dès l'origine : une exécution entièrement locale, sur une machine suisse, sans aucun service d'intelligence artificielle externe. La question de recherche est la suivante : dans quelle mesure un système RAG appliqué à des documents administratifs en français produit-il des réponses fiables et pertinentes pour des utilisateurs non-experts ?

La démarche instruit d'abord chaque choix de composant sur sources primaires : le framework Haystack, le modèle de langage Gemma 4 12B, le modèle d'embeddings Qwen3-Embedding-0.6B, la base vectorielle Chroma et le cadre d'évaluation RAGAS. Le prototype indexe 4 322 pages officielles de l'État et de la Ville de Genève, puis est déployé sur un serveur virtuel suisse ; il répond en ligne sur ge-trouve.ch. L'évaluation porte sur vingt questions genevoises écrites à la main, dont quatre volontairement hors périmètre, notées par quatre métriques RAGAS avec un juge local, et doublées d'un contrôle humain sur un quart des réponses.

Les résultats sont gradués. La recherche documentaire est le point fort (précision du contexte 0,92, rappel 0,93). La fidélité atteint 0,85, sans aucune invention pure : les erreurs sont des fusions de passages voisins. La pertinence, à 0,71, souffre de réponses trop longues, et l'aveu d'ignorance tient sur trois questions hors périmètre sur quatre. Dans son périmètre, le système produit donc des réponses fiables, sourcées et majoritairement pertinentes pour un non-expert, sous garde-fou humain. Le travail livre ainsi une évaluation documentée et reproductible d'un assistant RAG administratif genevois, avec ses limites et ses recommandations d'amélioration ; le code, les journaux et le protocole sont publics : github.com/DanielGarcia85/ge-trouve-ch.

Mots-clés : génération augmentée par la recherche (RAG) ; assistant conversationnel ; administration publique ; canton de Genève ; souveraineté numérique ; modèles de langage ouverts ; évaluation ; RAGAS.

Table des matières

Déclaration.....	i
Remerciements	ii
Résumé.....	iii
Table des matières	iv
Liste des tableaux	vi
Liste des figures.....	vi
1. Introduction	1
2. Comprendre le RAG	3
2.1 Du Transformer aux grands modèles de langage	3
2.2 Les limites des LLM sans mémoire externe	5
2.3 L'origine du RAG	7
2.4 Le pipeline RAG classique	9
2.5 L'évolution du RAG : Naive, Advanced, Modular.....	10
2.6 Pourquoi le RAG est pertinent pour Ge-Trouve.....	12
3. Choix du framework RAG	15
3.1 Le rôle du framework dans le pipeline.....	15
3.2 Les critères de choix.....	17
3.3 Les candidats	20
3.4 La comparaison et la décision	22
4. Choix du LLM	25
4.1 Le rôle du modèle et les contraintes d'exécution	26
4.2 Les critères de choix.....	30
4.3 Les candidats	33
4.4 La comparaison et la décision	38
5. Choix des embeddings	42
5.1 Le rôle du modèle d'embeddings et les contraintes du projet	42
5.2 Les critères de comparaison	44
5.3 Les candidats	46
5.4 La comparaison et la décision	48
6. Choix de la base vectorielle	51
6.1 Le rôle de la base vectorielle et les contraintes du projet	51
6.2 Les critères de comparaison	53
6.3 Les candidats	54
6.4 La comparaison et la décision	56
7. Évaluation d'un système RAG.....	59
7.1 Ce qu'il faut évaluer dans un système RAG	59
7.2 Le cadre retenu : RAGAS et ses métriques	60

7.3	L'évaluation par modèle juge et ses limites	62
7.4	Le protocole d'évaluation prévu pour Ge-Trouve	63
8.	RAG en contexte francophone	65
8.1	Le français administratif, le terrain de Ge-Trouve	65
8.2	Ce que le français change à chaque étage du pipeline	66
8.3	Un vide à combler, l'évaluation administrative genevoise	67
8.4	L'analyse du besoin	68
8.5	L'existant et le positionnement	69
9.	Spécifications et architecture du prototype	73
9.1	Cas d'usage et exigences	73
9.2	Le périmètre du corpus	75
9.3	L'architecture cible	76
9.4	L'environnement de développement et les premières mesures	79
9.5	L'interface	80
9.6	Le serveur frontal	81
10.	Construction du pipeline RAG	82
10.1	Le corpus pilote	82
10.2	L'indexation pilote	83
10.3	Le pipeline de réponse	84
10.4	Les mesures du système et le palier de serveur	85
10.5	La consigne de génération	85
10.6	Le corpus complet et l'ingestion reproductible	87
11.	Interface et déploiement	89
11.1	Une page pour le citoyen	90
11.2	Le streaming et la consigne V5	90
11.3	Le déploiement	91
12.	Résultats de l'évaluation	95
12.1	Le protocole exécuté	95
12.2	Ce que disent les scores	96
12.3	Les hors-périmètre et le garde-fou humain	97
12.4	Le diagnostic et les leviers	98
13.	Conclusion	100
	Bibliographie	102
	Annexe 1 : Le Transformer – vue d'ensemble et mécanisme d'attention	107
	Annexe 2 : Mesures datées de la partie pratique	108
	Annexe 3 : Captures de l'interface	114
	Annexe 4 : Résultats de l'évaluation	116
	Annexe 5 : Le calcul des métriques RAGAS	119

Liste des tableaux

Tableau 2.1 : Les cinq dimensions et les quinze critères d'évaluation des chatbots de service public	13
Tableau 3.1 : La grille des sept critères de choix du framework.....	19
Tableau 3.2 : La confrontation des quatre candidats aux sept critères	22
Tableau 4.1 : Les sept critères de choix du modèle de langage	32
Tableau 4.2 : Fiche d'identité des cinq candidats	37
Tableau 4.3 : La confrontation des cinq candidats aux sept critères	39
Tableau 5.1 : Les sept critères de comparaison du modèle d'embeddings	45
Tableau 5.2 : Fiche d'identité des trois candidats	47
Tableau 5.3 : La confrontation des trois candidats aux sept critères.....	48
Tableau 6.1 : Les sept critères de comparaison de la base vectorielle	54
Tableau 6.2 : Fiche d'identité des trois candidats	56
Tableau 6.3 : La confrontation des trois candidats aux sept critères.....	56
Tableau 7.1 : Les sept points de défaillance d'un système RAG	60
Tableau 7.2 : Les quatre métriques retenues pour l'évaluation de Ge-Trouve	61
Tableau 8.1 : Ce que le français change à chaque étage du pipeline	67
Tableau 8.2 : Les options d'un usager genevois face au besoin.....	69
Tableau 9.1 : Les exigences du prototype et leur moyen de vérification	75
Tableau 9.2 : Les sources du corpus : statuts et motifs	76
Tableau 10.1 : Les réglages du pipeline pilote et leur statut	85
Tableau 12.1 : Les moyennes des quatre métriques sur les questions avec réponse..	97

Liste des figures

Figure 2.1 : Le mécanisme d'attention.....	4
Figure 2.2 : L'architecture d'ensemble du Transformer	5
Figure 2.3 : Les quatre limites d'un LLM interrogé seul.....	6
Figure 2.4 : Mémoire paramétrique et mémoire non paramétrique	8
Figure 2.5 : Le pipeline RAG classique, de l'indexation à la réponse	9
Figure 2.6 : Du Naive RAG à l'Advanced RAG : les raffinements autour de la recherche .	11
Figure 2.7 : Les quatre limites d'un LLM seul et leurs réponses par le RAG	12
Figure 3.1 : Le rôle du framework dans le pipeline : les liaisons	16
Figure 4.1 : Le partage de la mémoire selon le palier de serveur, 12 ou 18 Go	29
Figure 5.1 : Le maillon embeddings dans le pipeline de Ge-Trouve	43
Figure 6.1 : Recherche exacte et recherche approximative	52
Figure 7.1 : Où chaque métrique regarde sur le pipeline	62
Figure 8.1 : La matrice SWOT de Ge-Trouve	71
Figure 8.2 : Le balayage PESTEL de Ge-Trouve	72
Figure 9.1 : L'architecture cible du prototype, de l'indexation à la réponse	78
Figure 10.1 : Les artefacts du pipeline, de la page en ligne à la base sur disque	89
Figure 11.1 : L'architecture de production : tout s'exécute dans le serveur suisse	93
Figure 12.1 : Le câblage de l'évaluation	96

1. Introduction

Un habitant de Genève tape une question dans son navigateur : « Où déposer ma demande de permis de séjour ? ». Cette question servira de fil conducteur à tout ce mémoire. Elle paraît simple. Elle ne l'est pas. La réponse existe, publiée sur les sites officiels du canton, gratuite et à jour. Mais elle est dispersée sur des milliers de pages, rédigée dans une langue administrative, et il faut déjà savoir où chercher, puis comment formuler sa demande. Aucune page ne s'appelle « déposer ma demande de permis de séjour ». Pour qui manie mal la recherche en ligne, les personnes âgées d'abord, les nouveaux arrivants ensuite, l'accès à l'information administrative reste une épreuve. Ce constat, ordinaire et que bien des familles connaissent, est le point de départ de ce travail.

La réponse proposée s'appelle Ge-Trouve un assistant conversationnel destiné à répondre en français courant aux questions sur les démarches genevoises, en s'appuyant sur les seules pages officielles et en citant ses sources. La technique qui le permet s'appelle la génération augmentée par la recherche (*retrieval-augmented generation*, RAG) : avant de rédiger, le système cherche dans une base de documents les passages qui répondent à la question, puis rédige à partir d'eux. Il répond à livre ouvert, et le chapitre 2 explique cette architecture en détail.

Concrètement, l'application à construire est une page web, accessible sans compte. L'utilisateur y écrit sa question comme il la poserait à un guichet, et l'assistant lui répond en quelques lignes, en français simple, avec les liens vers les pages officielles d'où vient l'information, pour qu'il puisse vérifier et agir. Elle doit s'en tenir aux démarches de l'État et de la Ville de Genève, refuser clairement de répondre quand la question sort de ce périmètre, et ne conserver aucune donnée sur ses usagers. Une règle de conception encadre le projet depuis l'origine : tout s'exécute localement, sur une machine suisse, et aucune question d'utilisateur ne transite par un service d'intelligence artificielle externe. La souveraineté n'est pas une contrainte subie, c'est un choix de conception, posé dès l'origine du projet. Les chapitres 3 à 6 choisissent les composants qui permettent de tenir ces engagements ; le chapitre 9 les traduit en exigences précises.

De ce point de départ découle la question de recherche de ce travail : « Dans quelle mesure un système RAG appliqué à des documents administratifs en français produit-il des réponses fiables et pertinentes pour des utilisateurs non-experts ? ». Quatre objectifs, posés dans la proposition de sujet, en organisent la poursuite. Le premier est la question elle-même : évaluer, mesures à l'appui, la fiabilité et la pertinence des

réponses. Le deuxième : concevoir, développer et déployer un prototype fonctionnel et accessible, bâti sur les sources officielles du canton. Le troisième : définir les critères de qualité des réponses, pertinence, fidélité aux sources, absence d'hallucinations, et les mesurer. Le quatrième : analyser les limites du système et formuler des recommandations pour l'améliorer. La conclusion reprendra ces quatre objectifs un à un.

La démarche suit trois parties. La Partie 1, du chapitre 2 au chapitre 8, pose les fondations : comprendre le RAG, puis choisir un à un les composants du système, le *framework*, le modèle de langage, le modèle d'*embeddings*, la base vectorielle et la méthode d'évaluation, avant d'examiner le terrain francophone, le besoin et l'existant. La Partie 2, chapitres 9 à 11, construit : spécifications, corpus, pipeline, interface, puis déploiement sur un serveur suisse, à l'adresse ge-trouve.ch. La Partie 3, chapitre 12, évalue le système sur vingt questions genevoises écrites à la main, et la conclusion répond à la question de recherche. Une ligne de méthode traverse le tout : chaque choix est instruit sur des sources primaires datées, chaque mesure est consignée dans un journal versionné, et l'ensemble du projet, code et journaux, vit dans un dépôt public.

2. Comprendre le RAG

Ce chapitre poursuit un objectif simple : comprendre ce qu'est un système RAG et pourquoi cette approche est la mieux adaptée au projet Ge-Trouve. Il est construit pas à pas, en partant des briques de base pour arriver à l'assemblage complet.

Deux notions doivent être posées d'emblée. Un grand modèle de langage (en anglais *large language model*, ou **LLM**) est un programme entraîné sur d'immenses quantités de texte, qui apprend à produire du langage en prédisant le mot suivant à partir de ceux qui le précèdent (Zhao et al. 2023). C'est la technologie derrière des outils grand public comme ChatGPT : on lui pose une question, il rédige une réponse. Le **RAG** (*retrieval-augmented generation*, littéralement « génération augmentée par la recherche ») est une manière particulière d'employer un tel modèle. Plutôt que de le laisser répondre uniquement à partir de ce qu'il a mémorisé pendant son entraînement, on le fait d'abord chercher, dans une base de documents, les passages pertinents pour la question posée, puis on lui demande de rédiger sa réponse à partir de ces passages (Lewis et al. 2020; Gao et al. 2023). L'image de l'examen résume bien l'idée : un modèle seul répond « à livre fermé », de mémoire ; un système RAG répond « à livre ouvert », en consultant les documents avant de se prononcer.

Cette distinction, seulement esquissée ici, sera précisée tout au long du chapitre. Mais pour comprendre comment un tel système fonctionne réellement, il faut d'abord s'intéresser à sa brique commune. Le moteur de recherche, le modèle qui rédige et les représentations de documents qu'un RAG assemble dérivent tous d'une même architecture, le Transformer ; c'est donc par elle que nous ouvrons (section 2.1). Nous verrons ensuite pourquoi un modèle de langage laissé à lui-même ne suffit pas (2.2), comment le RAG est né en réponse à ces limites (2.3), en quoi consiste concrètement son fonctionnement (2.4), comment il s'est perfectionné depuis (2.5), et enfin pourquoi il constitue le bon choix pour Ge-Trouve (0).

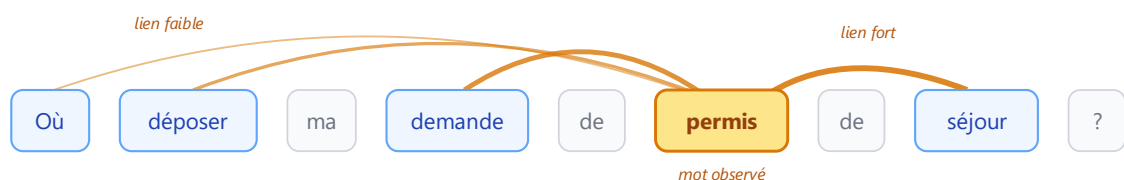
2.1 Du Transformer aux grands modèles de langage

Avant d'entrer dans le détail d'un système RAG, il faut s'arrêter un instant sur la brique qui rend tout le reste possible. Les composants que l'assistant Ge-Trouve va assembler, un moteur de recherche sémantique, un modèle qui rédige et des représentations vectorielles de documents, descendent tous d'une même architecture introduite en 2017,

le Transformer. La présenter brièvement évite de traiter ces composants comme des boîtes noires dans la suite du mémoire.

Jusqu'au milieu des années 2010, les meilleurs modèles de traitement du langage reposaient sur des réseaux de neurones récurrents¹, qui lisent une phrase mot après mot : le calcul est séquentiel, donc lent, et les liens entre mots éloignés se perdent en chemin. Vaswani et al. (2017) suppriment la récurrence au profit d'un mécanisme d'attention (*self-attention*) : chaque mot évalue directement sa pertinence vis-à-vis de tous les autres mots de la phrase. Pour cela, chaque mot reçoit trois rôles : une requête (*query*)², ce qu'il cherche à comprendre de son entourage ; une clé (*key*), ce qu'il propose aux autres ; une valeur (*value*), le contenu qu'il transmet quand on lui prête attention. Chaque mot compare sa requête aux clés des autres et récupère la valeur de ceux qui lui correspondent le mieux. Sur notre question type « Où déposer ma demande de permis de séjour ? » (Figure 2.1), le mot « permis » s'appuie surtout sur « séjour », « demande » et « déposer », et néglige « ma » ou « de ». Plusieurs lectures de ce type, les têtes d'attention, tournent en parallèle et saisissent chacune un type de relation. Le calcul devient massivement parallélisable et deux mots éloignés se relient en une seule opération : sur la traduction automatique, le Transformer établit de nouveaux records tout en s'entraînant plus vite que les modèles récurrents (Vaswani et al. 2017).

Figure 2.1 : Le mécanisme d'attention



Le mot « permis » se relie fortement à « séjour », qui précise sa nature, mais aussi à « demande », « déposer » et « Où », pourtant éloignés dans la phrase. L'épaisseur du trait suggère la force du lien. En bleu, les mots auxquels « permis » prête le plus attention ; en gris, ceux qu'il néglige presque. Chaque mot de la phrase effectue en parallèle cette même évaluation vis-à-vis de tous les autres.

(élaboration personnelle, d'après (Vaswani et al. 2017))

Le Transformer est devenu l'ossature de la quasi-totalité des grands modèles de langage actuels, rangés en trois familles selon la partie qu'ils réutilisent (Figure 2.2) : l'encodeur

¹ Un réseau de neurones récurrent (RNN, *recurrent neural network*) traite une séquence élément par élément en conservant, à chaque étape, un résumé de ce qu'il a déjà lu. Les LSTM (*long short-term memory*) et les GRU (*gated recurrent unit*) en sont des versions perfectionnées, dotées de mécanismes qui les aident à retenir l'information utile sur de plus longues portions de texte.

² Le terme « requête » (*query*) désigne ici un rôle interne attribué à chaque mot du texte traité ; il ne faut pas le confondre avec la question que pose l'utilisateur au système.

seul, tourné vers l'analyse d'un texte, comme BERT³ ; le décodeur seul, tourné vers la production de texte mot à mot, comme GPT, Mistral ou Llama⁴ ; ou les deux, comme BART⁵. Les briques d'un système RAG en sont toutes des descendantes : le moteur de recherche dense de l'architecture RAG d'origine, DPR⁶, repose sur BERT, son générateur BART est un encodeur-décodeur, et les modèles que le chapitre 4 comparera pour Ge-Trouve sont des décodeurs. L'Annexe 1 présente une vue détaillée de cette architecture et du mécanisme d'attention. Reste à voir pourquoi ces modèles, aussi puissants soient-ils, ne suffisent pas seuls à un assistant administratif : c'est l'objet de la section suivante.

Figure 2.2 : L'architecture d'ensemble du Transformer



L'encodeur analyse la question complète en une seule passe le décodeur rédige ensuite la réponse mot après mot, en consultant à chaque étape ce que l'encodeur a compris de la question.
En amont, les embeddings convertissent chaque mot en vecteur, la forme numérique de ces calculs.

(élaboration personnelle, d'après (Vaswani et al. 2017))

2.2 Les limites des LLM sans mémoire externe

Interrogé seul, un LLM présente des défauts structurels bien établis : hallucinations, connaissances obsolètes, absence de spécialisation et opacité (Gao et al. 2023). Tolérables pour un usage grand public, ils deviennent disqualifiants pour un service public dont les usagers ne peuvent pas vérifier la réponse. La Figure 2.3 les résume en quatre limites, qui structureront la suite du chapitre.

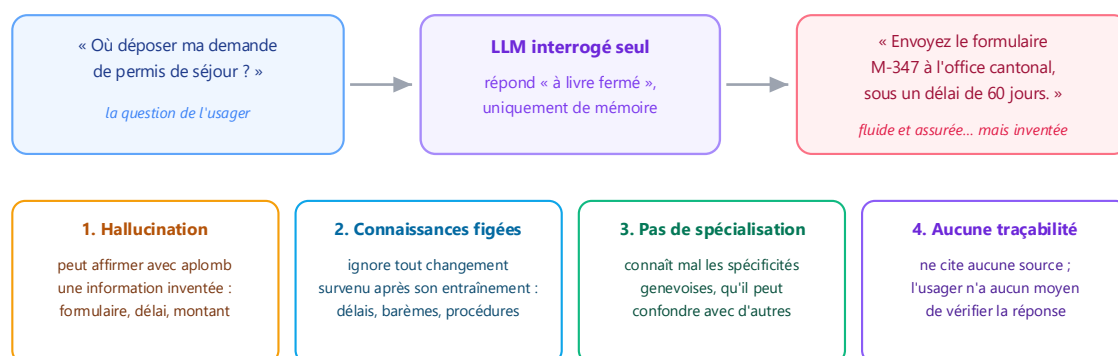
³ BERT, pour *Bidirectional Encoder Representations from Transformers*, est un Transformer développé par Google qui n'utilise que l'encodeur : il lit un texte dans les deux sens pour l'analyser, sans le produire (Devlin et al. 2018).

⁴ GPT (OpenAI), Mistral (société française Mistral AI) et Llama (Meta) sont des familles de grands modèles de langage de type décodeur, qui génèrent le texte mot après mot. Plusieurs versions de Mistral et de Llama sont librement téléchargeables, ce qui compte pour un déploiement local respectueux de la souveraineté des données.

⁵ BART, communément interprété comme *Bidirectional and Auto-Regressive Transformers*, est un Transformer proposé par Meta qui conserve à la fois l'encodeur et le décodeur, ce qui le rend adapté aux tâches où l'on transforme un texte en un autre (Lewis et al. 2019).

⁶ DPR, pour *Dense Passage Retrieval* (recherche dense de passages), est un moteur de recherche introduit par Meta en 2020 : au lieu de repérer des mots-clés identiques, il retrouve les passages dont le sens est proche de la question, en comparant des représentations vectorielles (Karpukhin et al. 2020)

Figure 2.3 : Les quatre limites d'un LLM interrogé seul



Quatre limites, une même racine : interrogé seul, le modèle ne consulte aucun document au moment de répondre — il ne peut que se souvenir.

(élaboration personnelle, d'après (Ji et al. 2022) ; (Gao et al. 2023))

La première limite, et la plus étudiée, est l'**hallucination** : un contenu fluide et formulé avec assurance, mais infidèle aux sources ou tout simplement faux (Ji et al. 2022). Elle découle du fonctionnement même du modèle, qui choisit à chaque étape le mot le plus plausible d'après ce qu'il a retenu dans ses paramètres⁷ ; or le mot le plus plausible n'est pas nécessairement le mot vrai, et quand le modèle ne sait pas, il ne s'arrête pas pour le dire, il complète. Un modèle interrogé seul peut ainsi répondre qu'il faut envoyer le formulaire M-347 à l'office cantonal et compter soixante jours : un formulaire et un délai qui n'existent nulle part, énoncés sur le même ton qu'une information exacte. Ji et al. distinguent la **fidélité** (*faithfulness*), conformité de la réponse aux sources sur lesquelles elle prétend s'appuyer, et la **factualité** (*factuality*), vérité dans le monde réel ; une réponse peut être fidèle à un document périmé, donc fidèle mais fausse. Le chapitre 7 s'appuiera sur cette grille.

La deuxième limite est le **caractère figé des connaissances** : un LLM ne sait rien après sa date de coupure⁸, alors que la matière administrative change en permanence, barèmes, délais, formulaires, horaires. Face à une règle modifiée, il répond l'ancienne, avec la même assurance, et réentraîner un modèle à chaque changement coûte bien trop cher (Gao et al. 2023).

La troisième est l'**absence de spécialisation** : entraîné sur un web majoritairement anglophone, où les spécificités genevoises ne pèsent presque rien, le modèle risque la

⁷ Les paramètres d'un modèle sont les milliards de valeurs numériques ajustées pendant son entraînement ; c'est en eux que réside tout ce que le modèle a « appris ». Les transformations et les vecteurs évoqués à la section 2.1 en font partie.

⁸ La date de coupure (*cutoff*) désigne la date au-delà de laquelle plus aucun texte n'a été intégré à l'entraînement du modèle ; tout événement postérieur lui est inconnu.

réponse « à la française », un formulaire CERFA⁹ recommandé à un usager genevois, et rien ne garantit qu'il distingue une règle genevoise d'une règle vaudoise.

La quatrième est l'**absence de traçabilité** : le modèle ne sait pas d'où vient sa réponse, et l'utilisateur n'a que deux options, croire sur parole ou refaire lui-même toute la recherche. Pour un service public, dont les réponses peuvent avoir des conséquences juridiques ou financières, c'est la limite la plus rédhibitoire.

Ces quatre limites ont une racine commune : interrogé seul, le modèle répond « à livre fermé », de mémoire, et cette mémoire est approximative, datée, générique et invérifiable. Un modèle plus gros n'y change rien ; il faut une autre architecture, qui lui donne accès, au moment de répondre, à une mémoire externe contenant les bons documents. C'est l'idée du RAG, dont la section suivante retrace l'origine.

2.3 L'origine du RAG

La section précédente s'est refermée sur un constat : les quatre limites tiennent à l'impossibilité de consulter quoi que ce soit au moment de répondre. Comment ouvrir le livre ? Une série de travaux répond entre 2020 et 2021, en trois jalons.

Le premier est REALM¹⁰, le premier modèle à intégrer un module de recherche dans son entraînement même : en apprenant à deviner des mots masqués, il apprend en même temps à retrouver, dans un vaste corpus, les documents qui l'aident. Sur les benchmarks¹¹ de question-réponse en domaine ouvert¹², il dépasse les méthodes antérieures, avec deux bénéfices durables : on voit quels documents ont été consultés, et le corpus peut évoluer indépendamment du modèle (Guu et al. 2020).

Le deuxième, quelques mois plus tard, donne son nom au domaine, *Retrieval-Augmented Generation*, et pose une distinction que tout le mémoire mobilisera (Figure 2.4) : la **mémoire paramétrique**, tout ce que le modèle a retenu de son entraînement, ce qu'il sait « par cœur », et la **mémoire non paramétrique**, un index de documents convertis en vecteurs, le classeur qu'il peut ouvrir au moment de répondre. L'architecture assemble un chercheur, DPR, et un générateur, BART, ajustés ensemble de bout en

⁹ Un CERFA est un document administratif normalisé requis pour effectuer des démarches officielles en France.

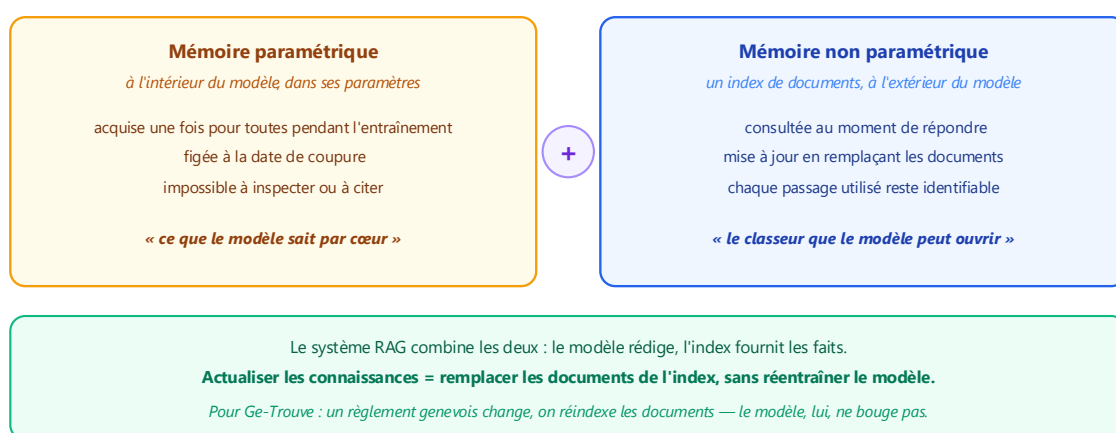
¹⁰ REAL M, pour *Retrieval-Augmented Language Model* (modèle de langage augmenté par la recherche documentaire), est le nom du modèle présenté par Guu et al. (2020).

¹¹ Un *benchmark* est un jeu de questions ou de tâches standardisé, partagé par la communauté scientifique, qui permet de comparer les systèmes entre eux sur une base identique.

¹² La question-réponse en domaine ouvert (*open-domain question answering*) consiste à répondre à des questions factuelles sans que le passage contenant la réponse soit fourni : le système doit d'abord le retrouver lui-même dans un vaste corpus.

bout¹³. Sa propriété décisive pour ce projet est le remplacement « à chaud » de l'index (*index hot-swapping*) : puisque la connaissance factuelle réside dans l'index, le remplacer suffit à mettre le système à jour, sans toucher au modèle ; les auteurs le démontrent en passant d'une Wikipédia de 2016 à celle de 2018, le même modèle adaptant ses réponses sans réentraînement (Lewis et al. 2020). Pour un assistant administratif, un règlement genevois qui change se traite donc en réindexant les documents concernés ; et puisque les passages utilisés sont connus du système, la réponse peut citer ses sources. Deux des quatre limites, les connaissances figées et la traçabilité, trouvent ainsi leur réponse de principe dès l'article fondateur.

Figure 2.4 : Mémoire paramétrique et mémoire non paramétrique



(élaboration personnelle, d'après (Lewis et al. 2020))

Le troisième jalon, RETRO¹⁴, apporte un enseignement d'échelle : adossé à une base de deux mille milliards de jetons¹⁵, il égale GPT-3 avec environ vingt-cinq fois moins de paramètres (Borgeaud et al. 2021). Un petit modèle rivalise avec un géant dès lors qu'il dispose d'une bonne mémoire externe, une leçon qui pèsera lourd pour Ge-Trouve, destiné à un serveur aux ressources limitées.

Le principe est posé : coupler un générateur à une mémoire documentaire externe produit des réponses plus factuelles, actualisables par remplacement de l'index et

¹³ L'ajustement fin (*fine-tuning*) consiste à poursuivre l'entraînement d'un modèle déjà entraîné, sur une tâche précise ; « de bout en bout » signifie que le chercheur et le générateur sont ajustés ensemble, chaque erreur corrigeant les deux composants à la fois.

¹⁴ RETRO, pour *Retrieval-Enhanced Transformer* (Transformer renforcé par la recherche documentaire), est le nom du modèle présenté par Borgeaud et al. (2021).

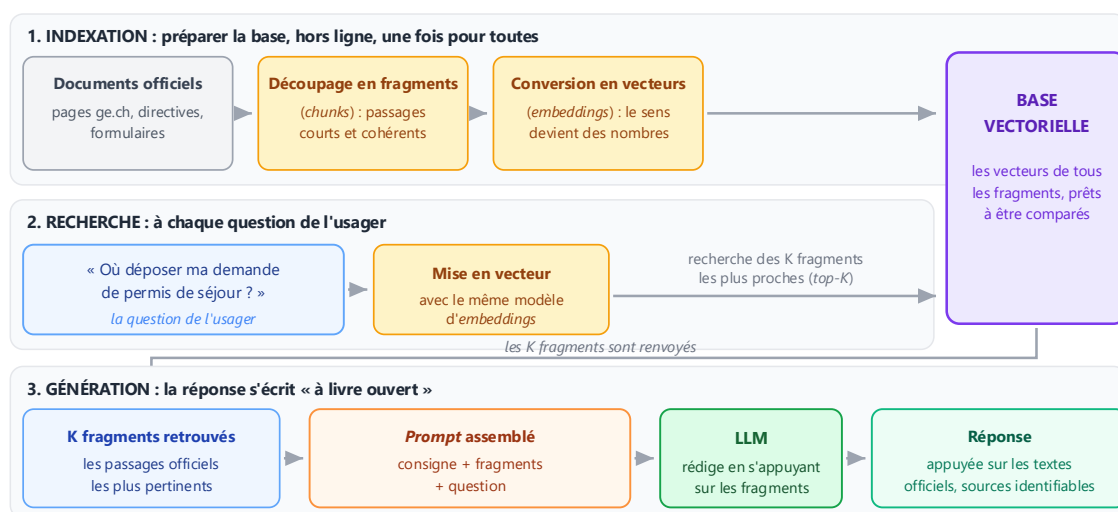
¹⁵ Un jeton (*token*) est l'unité de texte que manipule réellement un modèle de langage : un mot court, un fragment de mot long, ou un signe de ponctuation. Par simplification, ce mémoire parle de « mots » ; en toute rigueur, les modèles prédisent et comptent des jetons.

traçables jusqu'aux passages utilisés. Reste à voir comment un tel système fonctionne concrètement : c'est l'objet de la section suivante.

2.4 Le pipeline RAG classique

Un RAG moderne enchaîne trois étapes, l'indexation, la recherche et la génération (Gao et al. 2023) ; les deux dernières se produisent à chaque question, la première en amont, une fois pour toutes. La Figure 2.5 présente ce déroulement, exactement celui que la partie pratique construira pour Ge-Trouve.

Figure 2.5 : Le pipeline RAG classique, de l'indexation à la réponse



Contrairement au RAG fondateur de 2020, le modèle n'est plus ajusté : il reste tel quel et reçoit les fragments directement dans sa fenêtre de contexte. Il répond en consultant, sans rien réapprendre.

(élaboration personnelle, d'après (Gao et al. 2023))

La première étape, l'**indexation**, prépare la mémoire documentaire. Les documents de référence, pour Ge-Trouve les contenus officiels du canton, sont découpés en fragments¹⁶ courts, car c'est le passage précis traitant de la question qu'il faudra retrouver, pas le document entier. Chaque fragment est converti en vecteur, une suite de nombres qui capture son sens, par un modèle d'embeddings¹⁷, descendant direct des architectures de la section 2.1. Les vecteurs sont rangés dans une **base vectorielle**, spécialisée dans une seule opération : retrouver très vite, parmi des milliers de vecteurs, ceux qui ressemblent le plus à un vecteur donné.

¹⁶ Un fragment (*chunk*) est un court extrait de document, de quelques phrases à quelques paragraphes. Le découpage (*chunking*) vise des unités assez courtes pour être comparées finement, mais assez longues pour rester compréhensibles hors de leur contexte.

¹⁷ Un *embedding* (parfois traduit par « plongement ») est la représentation d'un texte sous la forme d'un vecteur, c'est-à-dire d'une suite de nombres, construite de telle sorte que deux textes de sens proche reçoivent des vecteurs proches. Un modèle d'*embeddings*, généralement dérivé de l'encodeur du Transformer, est un modèle spécialisé dans cette conversion ; le chapitre 5 en comparera plusieurs.

La deuxième étape, la **recherche**, se déclenche à chaque question. « Où déposer ma demande de permis de séjour ? » est convertie en vecteur par le même modèle d'embeddings que les fragments, ce qui rend la comparaison possible ; le système calcule la similarité avec tous les fragments de la base et retient les K plus proches, le *top-K*¹⁸, typiquement trois à dix fragments. Cette recherche est sémantique : « habiter à Genève » peut retrouver un texte parlant de « résidence sur le territoire cantonal », sans mot commun.

La troisième étape, la **génération**, assemble le tout : les fragments retenus, la question et une consigne forment le prompt¹⁹ transmis au LLM, la consigne lui enjoignant de répondre uniquement à partir des extraits fournis. Le modèle répond « à livre ouvert », et comme le système sait quels fragments il a fournis, il peut afficher les sources de la réponse, l'exigence de traçabilité de la section 2.2.

Une différence avec l'architecture fondatrice de 2020 explique la popularité actuelle du procédé. Chez Lewis et al. (2020), chercheur et générateur étaient ajustés ensemble, ce qui exigeait un entraînement spécifique ; dans le pipeline moderne, plus rien n'est réentraîné, le LLM est utilisé tel quel et reçoit les fragments dans sa fenêtre de contexte²⁰ (Gao et al. 2023). Monter un système RAG revient donc à assembler des composants existants, ce qui le met à la portée d'un projet comme Ge-Trouve et déplace la difficulté vers la qualité de chaque maillon.

Chaque maillon de la Figure 2.5 fera l'objet d'un chapitre : le framework au chapitre 3, le LLM au chapitre 4, le modèle d'embeddings au chapitre 5, la base vectorielle au chapitre 6. Mais ce pipeline simple a des limites connues, auxquelles la littérature répond par des raffinements : c'est l'objet de la section suivante.

2.5 L'évolution du RAG : Naive, Advanced, Modular

Le pipeline de la section précédente correspond à ce que Gao et al. (2023) nomment le Naive RAG. Ses faiblesses sont connues et se concentrent aux deux extrémités de la chaîne : la recherche remonte parfois des fragments hors sujet ou manque le passage

¹⁸ La notation *top-K* désigne les K éléments les mieux classés selon un critère ; ici, les K fragments dont les vecteurs sont les plus proches de celui de la question.

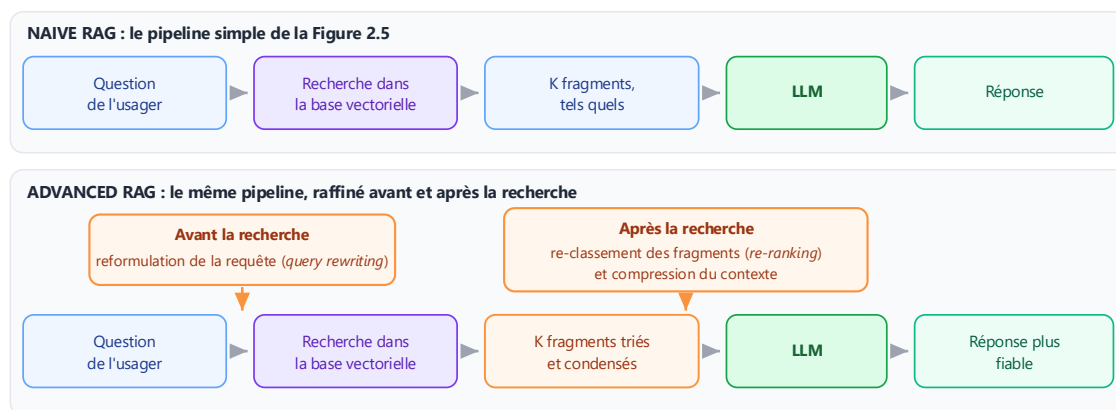
¹⁹ Le *prompt* est le texte d'entrée transmis au modèle : il rassemble la consigne (par exemple « réponds uniquement à partir des extraits fournis »), les fragments récupérés et la question de l'utilisateur.

²⁰ La fenêtre de contexte (*context window*) est la quantité maximale de texte qu'un modèle peut lire d'un seul coup pour produire sa réponse ; consigne, fragments et question doivent y tenir ensemble.

utile, ce que la littérature décrit en termes de précision et de rappel²¹ ; et si les fragments fournis sont mauvais, la réponse l'est aussi. Le RAG ne réduit donc les risques de la section 2.2 que si chaque étape fait son travail.

L'Advanced RAG ajoute des traitements de part et d'autre de la recherche (Figure 2.6) : avant elle, un meilleur découpage des fragments et la reformulation de la requête (*query rewriting*) ; après elle, le re-classement des fragments (*re-ranking*) et la compression du contexte transmis au modèle. Le Modular RAG va plus loin et fait de chaque fonction une brique indépendante, que l'on peut assembler, réordonner ou déclencher plusieurs fois dans une même réponse, au prix d'une complexité nettement plus grande (Gao et al. 2023).

Figure 2.6 : Du Naive RAG à l'Advanced RAG : les raffinements autour de la recherche



Le Modular RAG pousse la logique un cran plus loin : recherche, mémoire ou routage deviennent des briques interchangeables, que l'on assemble et réordonne selon les besoins du cas d'usage.

(élaboration personnelle, d'après (Gao et al. 2023))

Le choix de Ge-Trouve est assumé : partir d'un Naive RAG solide, dont chaque maillon est choisi avec soin, et n'y ajouter des raffinements Advanced que si l'évaluation en démontre le besoin. Un projet aux ressources limitées a plus à gagner d'un socle simple et maîtrisé que d'une architecture sophistiquée mal réglée, et la leçon de RETRO suggère qu'une bonne base documentaire compte davantage que la complexité du système. L'évaluation de la Partie 3 dira si le socle suffit.

²¹ En recherche d'information, la précision (precision) mesure la proportion des fragments récupérés qui sont réellement pertinents ; le rappel (recall) mesure la proportion des fragments pertinents existants qui ont effectivement été récupérés. Ces deux notions reviendront au chapitre 7, car les métriques d'évaluation des systèmes RAG s'en inspirent directement.

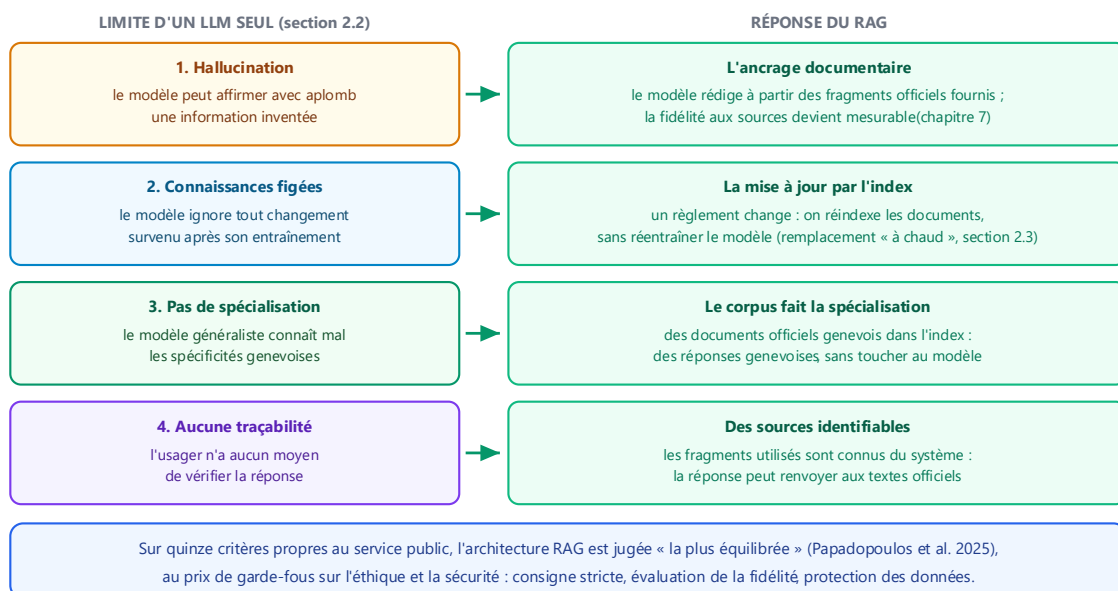
Reste à répondre à la question qui motive ce chapitre : en quoi cette approche est-elle la bonne réponse pour un assistant administratif genevois ? C'est l'objet de la dernière section.

2.6 Pourquoi le RAG est pertinent pour Ge-Trouve

Ce chapitre a établi ce que les grands modèles de langage savent faire, ce qui leur manque et comment le RAG y répond. Reste la question qui le motive : en quoi cette approche est-elle la bonne réponse pour un assistant administratif genevois ? La réponse tient en deux temps, technique puis institutionnel.

Le premier temps se lit à la Figure 2.7, qui répond terme à terme à la Figure 2.3. À l'hallucination, le RAG oppose l'ancrage documentaire : le modèle rédige à partir des fragments officiels fournis dans son prompt, sous une consigne qui lui enjoint de s'y tenir, et le risque devient une propriété mesurable, la fidélité, que le chapitre 7 quantifiera. Aux connaissances figées, il oppose la mise à jour par l'index : quand un barème, un délai ou un formulaire change, on réindexe, et le modèle ne bouge pas. À l'absence de spécialisation, il oppose le corpus : un modèle généraliste adossé aux documents officiels genevois produit des réponses genevoises. À l'absence de traçabilité, il oppose des sources identifiables : la réponse renvoie l'utilisateur aux textes officiels, et la confiance devient vérifiable.

Figure 2.7 : Les quatre limites d'un LLM seul et leurs réponses par le RAG



(élaboration personnelle, d'après (Gao et al. 2023) ; (Papadopoulos, Alexopoulos, Charalabidis 2025))

Ce constat rejoint la littérature du service public. Papadopoulos et al. (2025) comparent quatre familles d'architectures de chatbots pour la délivrance de services publics,

systèmes à règles, par recherche, génératifs purs et hybride RAG, sur une grille de quinze critères en cinq dimensions, que le Tableau 2.1 restitue.

Tableau 2.1 : Les cinq dimensions et les quinze critères d'évaluation des chatbots de service public

Dimension	Critères
I. Fonctionnalité et compréhension	Compréhension des demandes et reconnaissance de l'intention
	Exactitude des réponses et intégrité factuelle
	Cohérence et fiabilité
II. Expérience utilisateur et communication	Fluidité conversationnelle et prise en compte du contexte
	Gestion des erreurs et récupération
	Temps de réponse et réactivité
III. Éthique et sécurité	Atténuation des biais et équité
	Conformité éthique et responsabilité
	Protection des données et vie privée
IV. Adaptabilité et passage à l'échelle	Polyvalence de domaine et intégration
	Passage à l'échelle et efficacité des ressources
	Maintenabilité et efficacité des mises à jour
V. Inclusivité et délivrance du service	Soutien multilingue et accessibilité
	Personnalisation et contextualisation
	Délivrance du service et automatisation des processus

(traduit de (Papadopoulos, Alexopoulos, Charalabidis 2025))

Leur verdict désigne le RAG comme « l'architecture la plus équilibrée » : la meilleure sur la compréhension des demandes et l'exactitude factuelle, polyvalente, et facile à maintenir puisque la mise à jour passe par la base de connaissances (Papadopoulos, Alexopoulos, Charalabidis 2025). Leur réserve vaut cahier des charges : sur l'éthique et la sécurité, le RAG exige des garde-fous, que Ge-Trouve intègre dès la conception, consigne stricte, évaluation de la fidélité et exécution locale. Leur évaluation reste heuristique, fondée sur le jugement d'experts ; l'évaluation empirique de la Partie 3, sur un cas réel francophone, apporte le type de données que leur grille appelle.

Le second temps est institutionnel. La Confédération structure l'usage de l'intelligence artificielle dans son administration, avec une stratégie, des lignes directrices et un projet de réglementation attendu en consultation d'ici fin 2026 (Chancellerie fédérale s.d.) sur le terrain, plus de soixante projets étaient recensés à mi-2024, dont un agent conversationnel du Secrétariat d'État à l'économie fondé sur le RAG (Chavanne 2024). Genève suit la même logique : des lignes directrices sur l'IA générative pour son personnel en novembre 2024 (État de Genève 2024), puis, le 7 juillet 2026, une première stratégie de souveraineté numérique couvrant explicitement l'intelligence artificielle (État de Genève 2026a). Pour ce travail, la conséquence est directe : l'exécution entièrement

locale, sans qu'aucune donnée d'utilisateur ne transite par un service externe, posée dès l'origine comme contrainte de conception, devient l'application anticipée d'une exigence cantonale.

Au milieu de ce paysage, une case reste vide : le cadre genevois s'adresse au personnel de l'État, les chatbots fédéraux servent d'autres publics et d'autres langues, et aucun assistant conversationnel destiné aux citoyennes et citoyens genevois n'a pu être identifié. C'est ce vide que Ge-Trouve entend occuper, avec l'architecture que la littérature du service public recommande, en français, adossé aux documents officiels du canton et souverain par construction ; le chapitre 8 précisera le besoin et l'existant. En évaluant empiriquement un système RAG sur un cas local réel, ce travail documente aussi un terrain que la littérature, massivement anglophone, n'a pas encore couvert.

Le choix architectural est fondé, techniquement et institutionnellement. Reste à le construire : chaque maillon du pipeline de la Figure 2.5 appelle un arbitrage, et le premier, l'outil qui coordonnera l'ensemble, est l'objet du chapitre suivant.

3. Choix du framework RAG

Le chapitre précédent s'est refermé sur un pipeline désormais fondé, celui de la Figure 2.5, dont chaque maillon appelle un arbitrage. Ce chapitre ouvre la série de ces choix par celui qui conditionne tous les autres : l'outil qui coordonnera l'ensemble. Un système RAG n'est en effet pas un logiciel que l'on installe, c'est un assemblage de composants, un modèle de langage, un modèle d'embeddings, une base vectorielle, que du code doit charger, relier et faire dialoguer. Toute la question de ce chapitre tient en une phrase : comment écrire ce code ? En s'appuyant sur un *framework*²² spécialisé, une bibliothèque qui fournit ces liaisons toutes faites, ou en les programmant soi-même ?

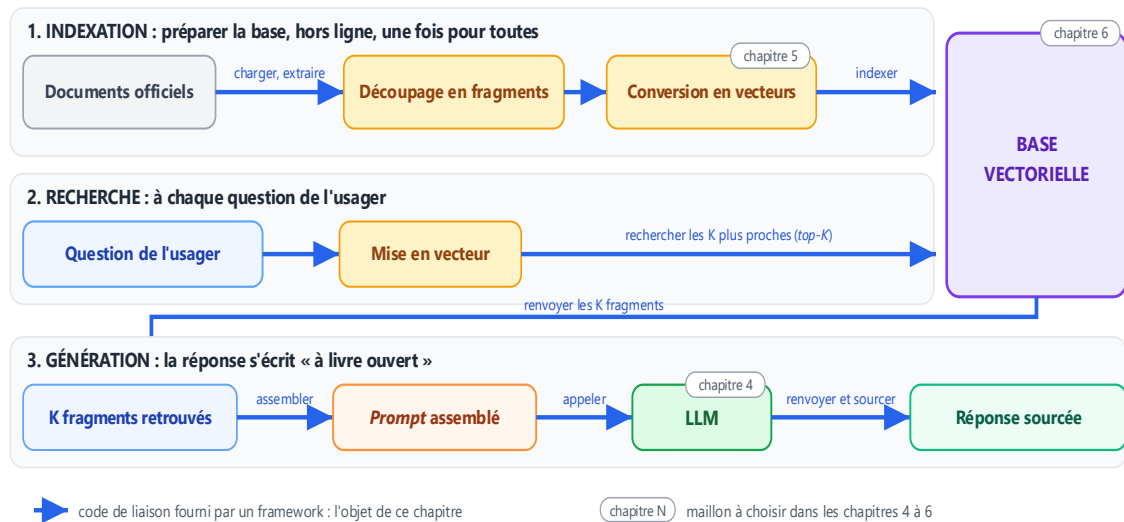
La démarche suit quatre temps. La section 3.1 précise ce qu'un framework RAG prend réellement en charge dans le pipeline, et pose une question moins triviale qu'il n'y paraît : en faut-il un ? La section 3.2 construit la grille de critères qui servira à trancher, chaque critère découlant d'une contrainte déjà établie dans ce mémoire. La section 3.3 présente alors les candidats, et la section 3.4 les confronte pour arrêter le choix.

3.1 Le rôle du framework dans le pipeline

Pour mesurer ce qu'un framework apporte, le plus simple est de suivre une nouvelle fois notre question type à travers le pipeline de la Figure 2.5. Entre le moment où l'utilisateur envoie « Où déposer ma demande de permis de séjour ? » et celui où la réponse s'affiche avec ses sources, le système enchaîne une série d'opérations : recevoir la question, la convertir en vecteur, interroger la base vectorielle, récupérer les fragments les mieux classés, assembler le prompt avec la consigne et ces fragments, appeler le modèle de langage, puis renvoyer la réponse accompagnée de ses références. En amont, l'indexation a exigé son propre enchaînement : collecter les pages de « ge.ch », en extraire le texte, le découper en fragments, le vectoriser, alimenter la base. Chaque flèche de la Figure 2.5 est donc du code que quelqu'un doit écrire. Un framework RAG est une bibliothèque qui fournit ce code de liaison prêt à l'emploi : des connecteurs vers les modèles et les bases de données, des découpeurs de documents, des gabarits de prompt, et la logique qui enchaîne le tout. Les maillons eux-mêmes restent à choisir, ce sera l'objet des chapitres 4 à 6. La Figure 3.1 met ce partage en évidence : le framework est représenté par les liaisons (flèches) entre ces maillons.

²² Un framework (littéralement « cadre de travail ») est une bibliothèque logicielle qui fournit l'ossature et des briques réutilisables pour un type d'application donné : le développeur assemble et configure ces briques au lieu de tout programmer lui-même. Le terme, passé dans l'usage courant de l'informatique en français, est conservé tel quel dans ce mémoire.

Figure 3.1 : Le rôle du framework dans le pipeline : les liaisons



Les boîtes sont les composants et les données du pipeline ; les flèches sont le code qui les relie : charger, découper, indexer, rechercher, assembler, appeler. C'est cette couche de liaison qu'un framework fournit.

(élaboration personnelle, d'après (Gao et al. 2023))

Cette plomberie pourrait passer pour un détail d'exécution ; la littérature d'ingénierie récente établit exactement l'inverse. Hasan et al. (2025), qui rapportent la construction de cinq systèmes RAG pour des organisations réelles, dont un assistant municipal répondant aux citoyens à partir de plus d'un millier de documents publics, constatent que l'essentiel de la difficulté s'est logé dans ces étapes de liaison : extraire un texte propre de documents numérisés, maintenir une collecte de pages web qui changent, régler la taille des fragments, stabiliser des environnements logiciels qui dérivent. Barnett et al. (2024) aboutissent au même constat par l'autre versant : leur rapport d'expérience catalogue, à partir de trois études de cas, sept points de défaillance d'un système RAG, et ces défaillances se répartissent sur tous les étages du pipeline, du fragment pertinent jamais retrouvé à la réponse incomplète. Ils en tirent une conclusion qui pèsera sur toute la suite de ce travail : un système RAG ne se valide qu'en fonctionnement, par ajustements successifs de tous ses réglages, taille des fragments, stratégie de recherche, contenu du prompt. L'outillage retenu ne détermine donc pas seulement le confort du développeur : il détermine la vitesse à laquelle on peut itérer, et par là, la qualité que le système atteindra dans le temps imparti.

L'argument en faveur d'un framework découle directement de ce constat. Il épargne d'abord la réécriture d'une plomberie déjà écrite, éprouvée et corrigée par de larges communautés de développeurs, là où le calendrier de ce travail ne pardonne aucun détour. Il rend ensuite, et surtout, les maillons interchangeables : ces bibliothèques exposent des interfaces uniformes, si bien que remplacer un modèle de langage ou une

base vectorielle par un autre se réduit le plus souvent à quelques lignes. Or les chapitres 4 à 6 comparent précisément plusieurs candidats pour chaque maillon : pouvoir les essayer tour à tour sans réécrire la chaîne n'est pas un confort, c'est la condition d'une comparaison loyale. Cette même souplesse sert enfin l'ajustement continu décrit par Barnett et al. (2024) : modifier un réglage, relancer, mesurer, recommencer.

L'argument inverse mérite pourtant d'être entendu. Un framework est une dépendance de plus, et une dépendance qui bouge : ces bibliothèques évoluent à un rythme soutenu, au point que la compatibilité d'une version à l'autre n'y est pas toujours garantie, et un projet bâti dessus hérite de ce mouvement. Leur généralité a aussi un coût : conçues pour couvrir des centaines de cas d'usage, elles embarquent bien davantage que ce qu'un pipeline volontairement simple, le Naive RAG assumé à la section 2.5, utilisera réellement. S'y ajoute une réserve propre à un travail de diplôme : la couche d'abstraction qui fait gagner du temps est aussi celle qui masque la mécanique, alors que ce mémoire s'est précisément attaché, depuis la section 2.1, à ne pas traiter ses composants en boîtes noires. Un fait, enfin, donne à l'option du « fait main » une réalité tangible : ni Hasan et al. (2025) ni Barnett et al. (2024), pourtant consacrés à l'ingénierie de huit systèmes RAG au total, ne mentionnent le moindre framework d'orchestration. Assembler la chaîne soi-même, en Python, n'est pas une vue de l'esprit ; c'est une pratique courante.

La question posée en ouverture du chapitre reste donc entière, et elle mérite mieux qu'un réflexe. Le framework ne produit rien par lui-même : il coordonne des composants qui font le travail. Un petit pipeline figé peut s'en passer ; un assemblage dont les pièces changent souvent, beaucoup plus difficilement. Tout dépend donc de ce que le projet exige de sa chaîne. Pour en juger loyalement, ce chapitre traite l'assemblage en Python pur comme un candidat à part entière, aux côtés des trois principaux frameworks du domaine, que la section 3.3 présentera. Encore faut-il des critères explicites pour les départager : c'est l'objet de la section suivante.

3.2 Les critères de choix

Le réflexe naturel, pour départager des outils, serait de s'appuyer sur une comparaison existante. Or une recherche documentaire menée en juin-juillet 2026 n'a permis d'identifier aucune comparaison académique publiée des principaux frameworks RAG : la littérature scientifique évalue des architectures et des systèmes, comme l'a montré le chapitre 2, mais pas l'outillage qui sert à les construire, et les articles d'ingénierie mobilisés à la section précédente ne nomment aucun framework (Hasan et al. 2025; Barnett et al. 2024). Les comparatifs disponibles en ligne émanent pour l'essentiel des

éditeurs eux-mêmes ou de billets de blog : sans méthode explicite, périmés en quelques mois, ils ne peuvent pas servir de source à un travail académique. Ce vide a une conséquence directe : la grille de choix doit être construite ici, pour ce projet, et chaque critère doit se justifier par une contrainte déjà établie dans ce mémoire plutôt que par une préférence.

Sept critères en découlent. Le premier est le support de l'inférence²³ locale sans appel externe imposé : le framework doit pouvoir faire fonctionner l'intégralité du pipeline avec des composants exécutés sur le serveur du projet, sans qu'aucune étape n'exige un service cloud. C'est la traduction technique de la contrainte de souveraineté posée dès l'origine et confortée par la stratégie cantonale (section 0) ; c'est aussi une pratique documentée par la littérature d'ingénierie, l'auto-hébergement réduisant les risques de conformité tout en améliorant les temps de réponse (Hasan et al. 2025). Le deuxième est l'empreinte : le framework et son cortège de dépendances doivent rester assez sobres pour un serveur virtuel aux ressources limitées, dans l'esprit de la leçon d'économie tirée de RETRO à la section 2.3. Le troisième est la maturité et la pérennité : licence libre, gouvernance identifiable, rythme de versions soutenable ; la section précédente a montré que ces bibliothèques évoluent vite, et un projet destiné à vivre au-delà de ce travail ne peut pas s'adosser à un outil fragile. Le quatrième est la documentation et la courbe d'apprentissage, dictées par le calendrier de ce travail : un outil brillamment conçu mais mal documenté coûterait des semaines qui n'existent pas.

Le cinquième critère, la facilité de configuration et d'itération, découle directement des deux enseignements de Barnett et al. (2024) : un système RAG ne se valide qu'en fonctionnement, et sa robustesse s'acquiert par ajustements successifs. Encore faut-il que ces ajustements restent bon marché : si les questions sur les permis de séjour remontent les mauvais passages, changer la taille des fragments ou le contenu du prompt doit se faire en quelques lignes, pas au prix d'une réécriture. Hasan et al. (2025) font le même constat par la pratique : c'est le retour d'usage qui guide le réglage. Le sixième critère porte sur les intégrations avec les composants des chapitres à venir : puisque ni le modèle de langage, ni le modèle d'embeddings, ni la base vectorielle ne sont encore arbitrés, le framework retenu doit laisser tous les candidats de ces chapitres raccordables, faute de quoi le choix d'aujourd'hui préjugerait des choix de demain. Le

²³ L'inférence désigne la phase d'utilisation d'un modèle entraîné : recevoir une entrée et produire une réponse. Elle s'oppose à l'entraînement, la phase où les paramètres du modèle sont ajustés. Une inférence locale s'exécute sur les machines du projet, sans appel à un service distant.

septième, enfin, est la compatibilité avec l'évaluation : RAGAS²⁴, l'outil qui mesurera la qualité des réponses au chapitre 7, a besoin d'accéder à la question posée, aux fragments récupérés et à la réponse générée ; le framework doit exposer ces trois éléments sans contorsion.

Un mot, avant de dresser la grille, sur ce qu'elle n'est pas : un barème. Attribuer des notes chiffrées à chaque critère puis les additionner donnerait au choix une apparence d'objectivité que rien ne fonderait, chaque note étant elle-même un jugement ; ce serait reproduire, en moins rigoureux, l'évaluation heuristique dont la section 0 a précisément relevé les limites chez Papadopoulos et al. (2025). La démarche retenue est différente : des faits vérifiables et datés, versions, licences, gouvernance, intégrations, relevés sur les sources primaires de chaque projet, puis un arbitrage argumenté critère par critère, que la section 3.3 conduira. Le Tableau 3.1 rassemble la grille : chaque ligne formule la question posée aux candidats et rappelle d'où vient l'exigence.

Tableau 3.1 : La grille des sept critères de choix du framework

Critère	Question posée au candidat	Origine de l'exigence
1. Inférence locale sans appel externe imposé	L'intégralité du pipeline peut-elle fonctionner avec des composants exécutés localement, sans service cloud ?	Souveraineté du projet (section 0 ; État de Genève 2026) ; pratique documentée (Hasan et al. 2025)
2. Empreinte et légèreté	Le framework et ses dépendances restent-ils sobres pour un serveur virtuel aux ressources limitées ?	Déploiement visé (section 0) ; leçon d'économie de RETRO (section 2.3)
3. Maturité et pérennité	Licence libre, gouvernance identifiable, rythme de versions soutenable ?	Vie du projet au-delà de ce travail ; volatilité des bibliothèques (section 3.1)
4. Documentation et courbe d'apprentissage	La prise en main est-elle réaliste dans le cadre de ce travail ?	Cadre du travail de Bachelor
5. Facilité de configuration et d'itération	Taille des fragments, <i>top-K</i> et <i>prompt</i> se règlent-ils en quelques lignes ?	Validation en fonctionnement (Barnett et al. 2024) ; réglage guidé par l'usage (Hasan et al. 2025)
6. Intégrations avec les composants à choisir	Les candidats des chapitres 4 à 6 sont-ils tous raccordables ?	Choix des maillons non encore arbitrés
7. Compatibilité avec l'évaluation	Question, fragments récupérés et réponse sont-ils accessibles pour RAGAS ?	Évaluation du chapitre 7 ; question de recherche

(élaboration personnelle)

²⁴ RAGAS (*Retrieval Augmented Generation Assessment*) est un cadre d'évaluation automatique des systèmes RAG : il mesure notamment la fidélité des réponses aux fragments fournis et la pertinence des passages récupérés (Es et al. 2023). Le chapitre 7 le présente en détail.

La grille est posée. Reste à rencontrer ceux qu'elle va interroger : les trois principaux frameworks du domaine, et l'option de s'en passer. C'est l'objet de la section suivante.

3.3 Les candidats

Trois bibliothèques dominent aujourd'hui le domaine par leur adoption, leur documentation et leur écosystème d'intégrations : **LangChain**, **LlamaIndex** et **Haystack**. Ce sont elles que la grille examinera, aux côtés de l'option d'assemblage en Python pur retenue à la section 3.1 ; les données factuelles qui suivent ont été relevées le 14 juillet 2026 sur la documentation et les dépôts officiels de chaque projet²⁵ ; cette date sert de référence unique au comparatif de ce chapitre. Deux pistes ont été écartées en amont. **OpenRAG**, cité dans la proposition initiale de ce travail, désigne désormais plusieurs projets sans lien entre eux. Le plus proche du besoin, porté par l'éditeur français Linagora, est publié sous licence AGPL²⁶, et se présente lui-même comme un terrain d'expérimentation ; son adoption demeure confidentielle, 230 étoiles GitHub²⁷ contre plusieurs dizaines de milliers pour le trio retenu (Linagora 2025). Conçu de surcroît comme une application complète à déployer plutôt que comme une bibliothèque dont on choisit les maillons, et dépourvu, contrairement aux trois retenus, de toute page d'intégration officielle pour Ollama²⁸, il répond mal aux critères de maturité et d'interchangeabilité. **DSPy**, souvent mentionné dans le même paysage, poursuit un autre objectif, l'optimisation programmatique des prompts, et non l'orchestration d'un pipeline documentaire²⁹.

LangChain est le poids lourd. Lancé en octobre 2022 par Harrison Chase et développé depuis par la société LangChain, Inc. sous licence MIT, il affiche environ 142 000 étoiles GitHub, plus d'un millier d'intégrations revendiquées et une version 1.3.13 (Chase 2022).

²⁵ Dépôts consultés :

LangChain : <https://github.com/langchain-ai/langchain>

LlamaIndex : https://github.com/run-llama/llama_index

Haystack : <https://github.com/deepset-ai/haystack>

OpenRag (Linagora) : <https://github.com/linagora/openrag>
[consultés le 14 juillet 2026].

²⁶ MIT et Apache-2.0, rencontrées plus loin, sont des licences libres dites permissives : elles autorisent l'usage, la modification et la redistribution du code, y compris commerciale ; Apache-2.0 y ajoute une concession explicite de brevets. L'AGPL est plus exigeante : elle impose de publier les modifications du code même lorsqu'il n'est exploité qu'à travers un service en ligne.

²⁷ Sur la plateforme GitHub, qui héberge le code source de ces projets, une étoile correspond à une mise en favori par un développeur. Ce compte est un indicateur imparfait mais standard de la notoriété d'un projet ; il est cité ici comme ordre de grandeur.

²⁸ Ollama est un logiciel libre qui exécute des modèles de langage et d'embeddings directement sur une machine locale et les expose comme un service que d'autres programmes peuvent appeler. Il s'est imposé comme l'outil standard pour faire fonctionner des modèles ouverts hors des services cloud ; le chapitre 4 y reviendra.

²⁹ DSPy (Stanford NLP) : <https://github.com/stanfordnlp/dspy> [consulté le 14 juillet 2026].

Sa philosophie : couvrir toute la chaîne des applications à base de modèles de langage au moyen d'interfaces standard par type de composant ; on installe le paquet d'un fournisseur, et changer de modèle revient à changer un nom, sans toucher au reste du code. Le projet a toutefois élargi son ambition : en 2026, sa documentation se présente comme une « plateforme d'ingénierie d'agents », flanquée d'outils dédiés aux agents et à leur supervision. Le RAG y reste pleinement couvert, avec une intégration Ollama officielle pour les modèles de conversation comme d'embeddings, mais il n'est plus le centre de gravité d'un écosystème devenu très vaste : c'est à la fois la promesse d'une richesse inégalée et le risque d'embarquer beaucoup plus que ce dont un pipeline volontairement simple a besoin.

LlamaIndex est le spécialiste des données. Lancé fin 2022 par Jerry Liu et porté par LlamaIndex, Inc. sous licence MIT, il compte environ 50 800 étoiles et une version 0.14.23 (Liu 2022). Son positionnement est le plus proche de la tâche de Ge-Trouve : se définissant comme le cadre de référence pour bâtir des applications sur ses propres documents, il organise tout autour de l'ingestion, de l'indexation et de l'interrogation d'un corpus, avec un annuaire public de connecteurs. L'éditeur finance le cœur libre du projet par des services cloud périphériques, dont un service d'analyse de documents ; ils restent optionnels, et l'intégration Ollama, officielle, documente précisément l'usage d'un service local. Un point appelle la vigilance : la numérotation en 0.x, après plus de trois ans d'existence, signale une interface encore susceptible de changements entre versions, ce que le critère de pérennité devra peser.

Haystack est le plus ancien des trois. Développé depuis 2019 par deepset, société berlinoise, sous licence Apache-2.0, il rassemble environ 25 900 étoiles et atteint la version 2.31 (Pietsch et al. 2019). Son orientation est la production : des composants explicites que l'on assemble en pipelines, et 159 intégrations maintenues par l'éditeur ou la communauté, parmi lesquelles Ollama et la base vectorielle Chroma, toutes deux entretenues par deepset. Sa feuille de route annonce un allègement du cœur en préparation de la version 3.0, signe d'une gestion disciplinée du périmètre, à rebours de l'inflation fonctionnelle ambiante. C'est la communauté la plus petite du trio, mais la gouvernance la plus lisible ; et le fait que l'éditeur soit européen n'est pas anodin pour un projet dont la souveraineté est une contrainte de conception.

Le quatrième candidat n'est pas une bibliothèque, c'est son absence. Assembler le pipeline en Python pur consiste à écrire soi-même les flèches de la Figure 3.1 : un client pour appeler les services locaux, quelques dizaines de lignes pour le découpage, l'appel direct des interfaces de la base vectorielle et du modèle retenus. Les bénéfices sont

réels : contrôle total, aucune dépendance superflue sur un serveur modeste, et une transparence pédagogique complète, rien de ce qui s'exécute n'étant caché derrière une abstraction. Les coûts le sont tout autant : chaque liaison est à écrire, à tester et à maintenir, précisément la part du travail que la littérature d'ingénierie décrit comme la plus lourde (Hasan et al. 2025), et chaque essai d'un nouveau composant aux chapitres 4 à 6 exige de refaire les raccordements que les frameworks fournissent. L'option est néanmoins sérieuse, et pratiquée : les huit systèmes documentés par Hasan et al. (2025) et Barnett et al. (2024) ont bien été construits sans qu'aucun framework d'orchestration soit jugé digne de mention.

Les quatre candidats sont posés. Reste à les passer au crible des sept critères du Tableau 3.1 : c'est l'objet de la section suivante.

3.4 La comparaison et la décision

Il reste à confronter les quatre candidats aux sept critères de la grille. Le Tableau 3.2 en donne la synthèse, sur la base des faits relevés à la section précédente ; conformément au choix méthodologique posé en 3.2, il se lit comme un support d'arbitrage, non comme un barème : la décision s'argumente critère par critère dans les paragraphes qui suivent.

Tableau 3.2 : La confrontation des quatre candidats aux sept critères

Critère	LangChain	LlamaIndex	Haystack	Python pur
1. Inférence locale sans appel externe imposé	Oui, intégration Ollama officielle	Oui, intégration Ollama officielle	Oui, intégration Ollama de l'éditeur	Oui, appels directs
2. Empreinte et légèreté	Écosystème très vaste	Cœur modulaire, paquets séparés	Cœur allégé en vue de la 3.0	Minimale
3. Maturité et pérennité	MIT ; v1.3.13 ; rythme très soutenu	MIT ; 0.14.23 : ruptures possibles entre versions	Apache-2.0 ; 2.31 ; transition 3.0 annoncée	Aucune dépendance, tout repose sur l'auteur
4. Documentation et courbe d'apprentissage	Très riche, périmètre étendu	Excellente porte d'entrée	Complète, concepts peu nombreux	Rien à apprendre, tout à concevoir
5. Facilité de configuration et d'itération	Bonne, via abstractions	Bonne, défauts condensés	Bonne, réglages exposés par composant	Faible : itérer, c'est développer
6. Intégrations avec les composants à choisir	Plus d'un millier	Annuaire public de connecteurs	Connecteurs 159, dont Ollama et Chroma par l'éditeur	À écrire, puis à refaire
7. Compatibilité avec l'évaluation	Oui	Oui	Oui, composant dédié	Oui, données passées à la main

(élaboration personnelle, données au 14 juillet 2026)

Trois critères, d'abord, ne départagent personne. L'inférence locale (critère 1) est acquise partout : les trois frameworks documentent une intégration Ollama officielle, et l'assemblage en Python pur appelle par construction les services qu'on lui désigne. La compatibilité avec l'évaluation (critère 7) l'est tout autant, RAGAS pouvant recevoir de n'importe quelle chaîne la question, les fragments et la réponse ; Haystack va jusqu'à l'intégrer comme un composant de pipeline. Quant à l'empreinte (critère 2), elle discrimine peu en pratique : une bibliothèque pèse quelques dizaines de mégaoctets là où le modèle de langage du chapitre 4 en pèsera des milliers ; l'avantage de principe revient à l'assemblage minimal, sans que rien ne s'y joue.

Le premier candidat tombe au critère 6. L'assemblage en Python pur suffirait à faire tourner un pipeline figé ; or les chapitres 4 à 6 comparent plusieurs modèles, plusieurs embeddings et plusieurs bases, et chaque essai exigerait de réécrire des raccords que les frameworks fournissent, quand le critère 5 rappelle qu'au-delà des réglages de base, toute itération redeviendrait du développement. L'argument de la section 3.1 se referme ici : le pipeline final de Ge-Trouve est petit et pourrait, une fois figé, se passer de framework ; mais la démarche des chapitres à venir essaiera plusieurs candidats à chaque maillon, et un assemblage dont les pièces changent sans cesse a besoin d'une couche qui les coordonne.

LangChain est écarté ensuite, sans avoir démérité nulle part : il passe chaque critère. Mais les critères 3 et 4 conjugués le desservent pour ce projet précis : son périmètre est devenu immense, sa courbe d'apprentissage charrie des concepts d'agents dont un pipeline volontairement simple n'a que faire, et son recentrage stratégique fait désormais du RAG une province d'un vaste empire plutôt que le cœur de sa feuille de route. Pour le calendrier de ce travail et le Naive RAG assumé à la section 2.5, c'est beaucoup de surface pour peu de besoin. Le choix se resserre donc entre LlamaIndex et Haystack, et il est serré. LlamaIndex offre le démarrage le plus rapide, la communauté la plus large des deux et un positionnement produit exactement aligné sur la tâche. Son talon d'Achille est le critère 3 : une numérotation en 0.x après plus de trois ans d'existence, c'est-à-dire, en versionnage sémantique³⁰, une interface qui s'autorise encore des ruptures d'une version mineure à l'autre. Le risque se neutralise pendant le développement par l'épingleage des versions (*version pinning*), l'une des leçons d'ingénierie de Hasan et al. (2025) ; il demeure pour la vie du projet au-delà du dépôt.

³⁰ Le versionnage sémantique (*semantic versioning*) numérote les versions X.Y.Z : les corrections incrémentent Z, les ajouts compatibles Y, et seul un changement de X autorise une rupture de compatibilité. Par convention, une numérotation en 0.x signale une interface qui ne garantit pas encore cette stabilité.

Ce travail retient **Haystack**. Trois raisons hiérarchisées fondent la décision. La première tient à la forme même du code : chez Haystack, le chemin normal est l'assemblage explicite, composant par composant, chaque liaison du pipeline s'écrivant noir sur blanc ; pour un mémoire qui s'est interdit les boîtes noires dès la section 2.1, l'outil dont la pratique par défaut est la transparence a une valeur pédagogique directe, jusque dans la soutenance. La deuxième est la gouvernance la plus lisible du trio : licence Apache-2.0, versionnage mûr, transition majeure annoncée et documentée à l'avance, autant de gages de prévisibilité pour un projet appelé à vivre au-delà de ce travail. La troisième est l'ajustement à la pile pressentie : les intégrations Ollama et Chroma sont maintenues par l'éditeur lui-même, et l'évaluation du chapitre 7 s'insérera comme un composant du pipeline. S'y ajoute un argument de cohérence, assumé comme tel : l'éditeur est européen, et son dépôt met en avant des usages dans le secteur public européen, de la Commission européenne à des administrations allemandes (Pietsch et al. 2019) ; pour un assistant de service public dont la souveraineté est une contrainte de conception, le fil est cohérent de bout en bout. Le choix a ses renoncements, qu'il faut nommer : la communauté la plus petite du trio, donc moins de réponses toutes faites en cas de blocage, et un démarrage un peu moins immédiat que celui de LlamaIndex, qui demeure un choix défendable ; l'arbitrage tient à la hiérarchie des critères posée par ce projet, pas à une supériorité absolue.

Le framework est choisi, et il tient la promesse du critère 6 : tous les candidats des chapitres à venir restent raccordables sans réécrire la chaîne. Restent les composants à choisir, un par chapitre. Le premier est le modèle de langage : c'est l'objet du chapitre suivant.

4. Choix du LLM

Premier composant à choisir : le modèle de langage, celui qui rédigera les réponses adressées aux usagers. Le choix ne part pas d'une page blanche. Dès l'origine, ce travail s'est fixé une règle de conception, que la stratégie cantonale est venue conforter depuis (section 0) : une fois l'assistant en service, aucune question d'utilisateur ne transite par un service d'intelligence artificielle externe. Seuls les modèles à « poids ouverts »³¹, ceux que l'on peut télécharger et exécuter sur le serveur du projet, entrent donc en jeu. Les modèles accessibles uniquement à travers un service cloud, comme GPT d'OpenAI ou Claude d'Anthropic³², sont hors périmètre par construction³³, et non par jugement de qualité : la question de savoir s'ils rédigeraient mieux ne se pose pas, puisque la règle les écarte avant elle.

Avant de rencontrer les candidats, il faut dire d'où viendront les faits qui les décrivent, car tous n'ont pas la même fiabilité. La source la plus solide est le rapport technique : l'article scientifique par lequel l'éditeur documente l'architecture, l'entraînement et les performances de son modèle, publié le plus souvent sur arXiv³⁴. Vient ensuite la carte de modèle (*model card*)³⁵ : la fiche descriptive que l'éditeur dépose sur Hugging Face³⁶, la plateforme où se publient et se téléchargent les modèles ouverts. La carte est plus à jour que le rapport, mais elle est aussi purement déclarative, et certains chiffres

³¹ Un modèle à poids ouverts (*open weights*) publie ses paramètres entraînés, les milliards de valeurs numériques rencontrées à la section 2.2 : quiconque peut les télécharger et exécuter le modèle sur sa propre machine. L'ouverture des poids ne dit rien, en revanche, des données ni du code d'entraînement, qui restent le plus souvent privés ; la suite du chapitre rencontrera des degrés d'ouverture plus complets.

³² Claude est aussi l'outil d'assistance à la rédaction déclaré en tête de ce mémoire. Les deux rôles ne se confondent pas : l'outil relève de la fabrication de ce travail, non du système qui en résulte ; aucune donnée d'utilisateur ne transite par lui.

³³ L'actualité récente en donne la mesure : le 12 juin 2026, une directive américaine de contrôle des exportations a suspendu l'accès aux deux modèles les plus avancés d'Anthropic pour tout ressortissant étranger, contraignant l'éditeur à les désactiver pour l'ensemble de ses clients ; l'accès n'a été rétabli qu'au 1er juillet (France 24 2026). Un service bâti sur une telle interface aurait cessé de répondre pendant près de trois semaines, par décision d'un gouvernement étranger.

³⁴ arXiv est un serveur public de prépublications scientifiques : les auteurs y déposent leurs articles sans attendre la relecture d'une revue. C'est le canal de publication dominant de la recherche sur les modèles de langage, où la rapidité du domaine prime sur les circuits éditoriaux classiques ; la plupart des articles cités dans ce mémoire y sont accessibles.

³⁵ La carte de modèle (*model card*) rassemble, sur une page normalisée, ce que l'éditeur déclare de son modèle : licence, usages prévus, langues couvertes, résultats aux benchmarks. Rédigée par l'éditeur seul, sans relecture indépendante, elle a la valeur d'une fiche produit : précieuse et actuelle, mais à recouper.

³⁶ Hugging Face est la plateforme centrale de publication des modèles d'intelligence artificielle ouverts : les éditeurs y déposent les poids de leurs modèles, leurs cartes descriptives et des jeux de données. Son rôle pour les modèles est comparable à celui que la plateforme GitHub, rencontrée au chapitre 3, joue pour le code source.

n'existent que là. Viennent enfin les pages de distribution, comme celles d'Ollama, déjà croisé au chapitre précédent : elles renseignent sur la manière d'exécuter un modèle, pas sur ses qualités.

La démarche suit le chemin du chapitre précédent. La section **Erreur ! Source du renvoi introuvable.** établit ce que le modèle fait exactement dans le pipeline, et montre que les conditions d'exécution du projet, un serveur virtuel aux ressources limitées, transforment des caractéristiques d'ordinaire secondaires en contraintes de premier ordre. La section **Erreur ! Source du renvoi introuvable.** en tire la grille de critères, la section **Erreur ! Source du renvoi introuvable.** présente les candidats retenus et les pistes écartées, et la section **Erreur ! Source du renvoi introuvable.** les confronte pour arrêter le choix.

4.1 Le rôle du modèle et les contraintes d'exécution

Reprenons notre question type là où la section 3.1 l'a laissée. « Où déposer ma demande de permis de séjour ? » a été convertie en vecteur, la base vectorielle a rendu ses fragments les mieux classés, et le framework retenu au chapitre précédent a assemblé le prompt : la consigne, les extraits officiels, la question. Ce prompt part vers le modèle de langage, qui accomplit la dernière opération du pipeline : rédiger, à partir de ces matériaux, la réponse que l'utilisateur lira. Tout ce qui précède prépare cette étape ; le modèle, lui, écrit. Sa place dans l'architecture doit être dite clairement : il est le rédacteur du système, pas sa source de savoir. La répartition des deux mémoires, posée à la section 2.3, l'a établi : les faits résident dans l'index des documents officiels, et le modèle apporte la langue, la compréhension de la question et la capacité de composer une réponse claire à partir des extraits. Cette répartition, la littérature RAG l'a théorisée, et les grands éditeurs la rejoignent maintenant de l'intérieur. Meta décrit ainsi la post-formation³⁷ de Llama 3 en matière de factualité : plutôt que d'élargir ce que le modèle sait, l'aligner sur ce qu'il sait déjà, et lui apprendre à refuser de répondre quand il ne sait pas (Grattafiori et al. 2024). Quand la connaissance manque, la suite logique de ce principe n'est pas d'espérer, mais de fournir au modèle les bons documents au moment de répondre. C'est exactement ce que fait la récupération, et on retrouve le fil des hallucinations suivi depuis la section 2.2 (Ji et al. 2022).

³⁷ La post-formation (*post-training*) regroupe les étapes qui suivent le pré-entraînement d'un modèle : ajustement fin sur des instructions, apprentissage des préférences humaines, règles de refus. C'est elle qui transforme un modèle brut en assistant utilisable ; la section 4.2 en fera un critère.

Quel rédacteur retenir, alors ? La tentation serait de chercher le meilleur modèle disponible, en s'en remettant aux classements généraux. La question de ce chapitre est plus étroite et plus concrète : quel est le meilleur modèle qui tient dans la machine ? Car la machine, ici, limite tout. Le serveur de production n'est pas encore souscrit, sa commande interviendra au moment du déploiement ; les offres de serveurs virtuels retenues comme candidates encadrent sa mémoire vive entre deux paliers, 12 et 18 Go, et ce chapitre raisonnera sur les deux : le palier plancher, hypothèse prudente, et le palier de confort, dont le surcoût mensuel devra se justifier. Quel que soit le palier, la mémoire ne sera pas réservée au modèle : elle logera aussi le système d'exploitation, le modèle d'embeddings, la base vectorielle et l'application web qui sert l'assistant. Le serveur n'aura pas non plus de carte graphique : l'inférence se fera sur processeur. Ces conditions changent l'ordre habituel des priorités. Des caractéristiques que les comparatifs mentionnent à peine, l'empreinte d'un modèle quantifié, la taille de son cache, sa verbosité, deviennent ici des critères de premier ordre.

Entre le modèle et le serveur s'intercale la couche que la note d'Ollama du chapitre 3 promettait de retrouver ici. Ollama charge les modèles en mémoire, les exécute et les expose comme un service local que le pipeline appelle ; c'est lui qui animera le développement de ce travail, et qui tournera sur le serveur. Les rôles ne se confondent pas avec ceux du chapitre 3 : le framework décide quand appeler le modèle et avec quel prompt, Ollama se contente d'exécuter. Les modèles qu'il consomme se présentent en fichiers GGUF³⁸, presque toujours quantifiés³⁹ : la quantification réduit la précision numérique des paramètres, de seize bits à huit, quatre, parfois moins, ce qui réduit d'autant l'empreinte mémoire, au prix d'une perte de qualité en général modeste. Reste à savoir qui quantifie. Parfois l'éditeur lui-même : OpenAI publie gpt-oss directement en précision réduite, dans un format de sa conception (OpenAI et al. 2025), et Google livre des versions de Gemma 4 entraînées en tenant compte de leur quantification, si bien que la compression fait partie du produit (Gemma Team et al. 2026). Parfois la communauté, qui convertit ce que l'éditeur n'a pas fourni, comme pour Apertus⁴⁰. La différence compte, et la section **Erreur ! Source du renvoi introuvable.** y reviendra :

³⁸ GGUF est un format de fichier, créé par le projet llama.cpp, qui rassemble en un seul fichier les paramètres d'un modèle et les informations nécessaires à son exécution locale ; c'est le format que consomme Ollama.

³⁹ La quantification (*quantization*) s'exprime en bits par paramètre : huit, quatre, parfois deux, contre seize en pleine précision. Les fichiers en portent la trace dans leur nom : Q4_0 désigne ainsi une quantification par blocs sur quatre bits, MXFP4 un format d'environ quatre bits conçu par OpenAI.

⁴⁰ Apertus est un modèle de langage à poids ouverts développé en Suisse par l'initiative Swiss AI, portée notamment par l'EPFL, l'ETH Zurich et le Centre suisse de calcul scientifique ; c'est l'un des candidats que la section **Erreur ! Source du renvoi introuvable.** présentera.

une quantification d'éditeur est testée et assumée par lui, une conversion communautaire repose sur le sérieux d'un tiers.

Une fois ces éléments posés, l'empreinte d'un modèle se calcule presque de tête. Elle tient pour l'essentiel en une multiplication : le nombre de paramètres par le poids de chacun après quantification. À quatre bits, un paramètre pèse un demi-octet : un modèle de 12 milliards de paramètres occupe donc environ 6 Go. S'y ajoutent le « cache d'attention »⁴¹, qui grandit avec la longueur du contexte traité, et une marge de fonctionnement. Gemma 4 12B permet de vérifier ce calcul avec des chiffres de source primaire : 24 Go de poids en pleine précision, ramenés à 7,65 Go en version quatre bits, l'écart avec les 6 Go théoriques venant de l'habillage du format, plus 0,28 Go de cache pour un contexte de 32 000 jetons (Gemma Team et al. 2026). Si le cache reste aussi modeste, ce n'est pas un hasard : les modèles récents utilisent l'attention à requêtes groupées (GQA)⁴², un choix que Meta motive justement par la réduction du cache à l'inférence (Grattafiori et al. 2024) ; Ministral 3, l'un des candidats à venir, fait ainsi partager huit têtes clé-valeur à trente-deux têtes de requête (Liu et al. 2026).

Le même calcul explique que deux modèles pourtant remarquables sortent du jeu avant même d'être présentés. GLM-5.2, modèle phare du moment, aligne 744 milliards de paramètres (GLM-5-Team et al. 2026) : même ramené à deux bits par paramètre par les conversions communautaires les plus agressives, il occupe encore quelque 239 Go⁴³, plus de treize fois le plus généreux des paliers envisagés. Aucune quantification ne comblera un tel écart. Le cas de gpt-oss-20b est plus instructif, parce que la marge est mince : OpenAI le publie pourtant déjà quantifié, mais annonce 16 Go de mémoire comme exigence minimale (OpenAI et al. 2025). C'est plus que le palier plancher tout entier, et davantage que ce que le palier de confort laisse au modèle une fois le reste du système servi. La frontière de ce chapitre n'a donc rien d'une abstraction : elle passe juste au-dessus des serveurs envisagés, et des modèles sérieux se trouvent immédiatement de l'autre côté. La section **Erreur ! Source du renvoi introuvable.** reviendra sur ces écartés.

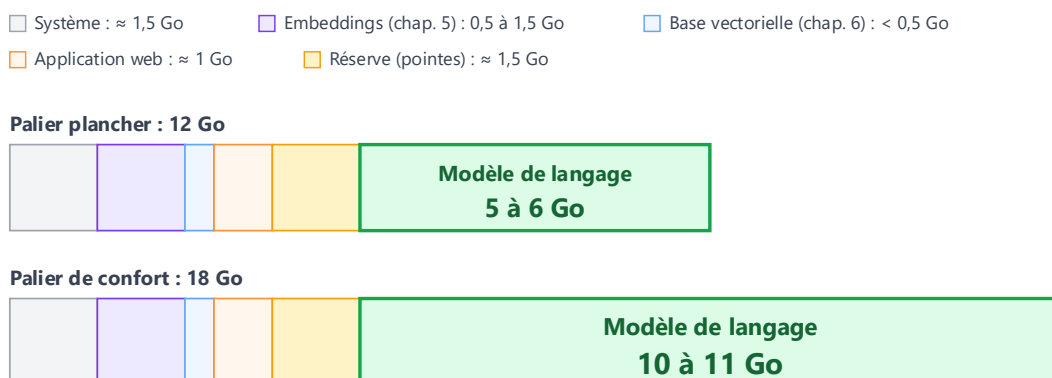
⁴¹ Pour ne pas recalculer, à chaque nouveau jeton, les clés et les valeurs (section 2.1) de tous les jetons qui précèdent, le modèle les conserve en mémoire : ce cache d'attention (*KV cache*) grandit donc avec la longueur du contexte.

⁴² L'attention à requêtes groupées (*grouped-query attention*, GQA) fait partager un même jeu de clés et de valeurs à plusieurs têtes de requête, ce qui divise la taille du cache d'attention sans dégrader sensiblement la qualité des réponses.

⁴³ Empreintes mémoire des conversions communautaires de GLM-5.2 : <https://unsloth.ai/docs/models/glm-5.2> et <https://huggingface.co/unsloth/GLM-5.2-GGUF> [consultés le 16 juillet 2026].

Reste à tracer cette frontière avec précision, en décomposant le budget. Les chiffres qui suivent sont des ordres de grandeur prudents, retenus comme hypothèses de dimensionnement ; la partie pratique les confrontera aux mesures relevées sur le serveur réel. Les postes fixes d'abord, identiques quel que soit le palier : le système d'exploitation et ses services, environ 1,5 Go ; le modèle d'embeddings, de quelques centaines de mégaoctets à 1,5 Go selon les candidats que le chapitre 5 chiffrera ; la base vectorielle du chapitre 6, quelques centaines de mégaoctets ; l'application web et sa mémoire de travail, environ 1 Go ; et une réserve pour absorber les pointes. La Figure 4.1 met le partage en image pour les deux paliers : une fois chacun servi, il reste au modèle de langage une enveloppe de 5 à 6 Go sur le palier plancher, et d'environ 10 à 11 Go sur le palier de confort. À quatre ou cinq bits par paramètre, cette fourchette donne un gabarit : des modèles d'environ 7 à 12 milliards de paramètres, quantifiés. Au-delà, même le palier plancher ne suit plus, un modèle de 14 milliards n'y logeant pas à côté des autres postes. Le périmètre du comparatif est ainsi posé. La question qu'il laisse ouverte, celle du palier, sera tranchée avec le choix du modèle à la section **Erreur ! Source du renvoi introuvable.** : les deux décisions n'en font qu'une.

Figure 4.1 : Le partage de la mémoire selon le palier de serveur, 12 ou 18 Go



À postes fixes identiques, le palier de confort porte l'enveloppe du modèle de langage de **5 à 6 Go** à environ **10 à 11 Go**.

(élaboration personnelle)

La mémoire n'est pas la seule ressource comptée : le temps l'est aussi. Sans carte graphique, la génération est séquentielle et chaque jeton produit coûte un temps à peu près constant ; une réponse deux fois plus longue fait simplement attendre l'utilisateur deux fois plus longtemps. La verbosité d'un modèle, invisible dans les classements, devient donc un poste de latence, et la sobriété une qualité : le rapport de Ministral 3 met justement en avant la concision de ses réponses, un avantage direct quand chaque jeton

coûte (Liu et al. 2026). Les modèles de 2026 ajoutent une variable nouvelle : le mode de réflexion (*thinking mode*), où le modèle produit un brouillon de raisonnement avant sa réponse. Le procédé améliore nettement les tâches complexes, mais chaque jeton de « pensée » est un jeton généré, donc de l'attente en plus ; gpt-oss va jusqu'à proposer trois niveaux d'effort de raisonnement (OpenAI et al. 2025), et Gemma 4 intègre le sien (Gemma Team et al. 2026). Surtout, un résultat récent contredit l'idée que réfléchir ne peut pas nuire : sur RULER⁴⁴, un banc d'essai de récupération en contexte long, le mode réflexion fait moins bien que le mode direct, 84,4 contre 89,1 pour Qwen3-8B, les auteurs suggérant que la pensée interfère avec les tâches de restitution, qui n'exigent pas de raisonnement (Yang et al. 2025). Or restituer fidèlement des passages fournis est précisément le travail d'un modèle en pipeline RAG. Le régime naturel de Ge-Trouve sera donc le mode direct : un candidat devra être bon sans réfléchir, et permettre de désactiver la réflexion proprement. Mémoire comptée, temps compté : les contraintes sont posées, et chacune appelle un critère. Les formuler est l'objet de la section suivante.

4.2 Les critères de choix

La section 3.2 partait d'un constat de rareté : aucun comparatif de frameworks ne répondait à notre situation. Les modèles de langage posent le problème inverse. Les classements publics abondent, les bancs d'essai se comptent par dizaines et chaque carte de modèle affiche ses records. Cette abondance ne répond pourtant pas mieux à notre question, car elle mesure autre chose : une performance maximale, relevée en pleine précision, sur des serveurs équipés de cartes graphiques, et le plus souvent en anglais. Rien de tout cela ne décrit un petit serveur sans carte graphique répondant en français à des usagers genevois. La grille de cette section traduit donc les contraintes de la section 4.1 en sept questions vérifiables, posées aux trois étages de sources présentés en tête de chapitre.

Une absence doit être expliquée d'emblée : la souveraineté des données, fil conducteur du projet, ne figure pas parmi les critères. Non qu'elle pèse peu : elle a déjà agi, en amont, au moment de définir le périmètre. Un critère départage des candidats admis à concourir ; la souveraineté, elle, a décidé en amont qui pouvait concourir. La grille reste par ailleurs qualitative, comme au chapitre 3 et pour les raisons exposées alors : chiffrer des poids relatifs, c'est déguiser des choix en mesures. Un argument de plus s'y ajoute ici. Les faits à comparer ne viennent pas du même étage de fiabilité, et additionner dans

⁴⁴ RULER dissémine des informations dans de très longs textes, puis mesure la proportion que le modèle retrouve et restitue correctement ; le score reflète la capacité à exploiter réellement une grande fenêtre de contexte, au-delà de sa taille annoncée.

une même note un chiffre de rapport technique et une déclaration de carte reviendrait à mesurer avec deux instruments différents et à faire comme si c'était le même.

Le premier critère est éliminatoire, comme l'était la souveraineté au chapitre 3 : l'exécutabilité dans le gabarit. Existe-t-il une version quantifiée du modèle dont l'empreinte, cache d'attention compris, tient dans les enveloppes de la Figure 4.1, et à quel prix : à l'aise dès le palier plancher, à l'étroit sur celui-ci mais serein sur le palier de confort, ou nulle part ? L'empreinte doit de plus être attestée par une source primaire ou par l'éditeur, pas par une promesse. La section 4.1 a montré ce qu'il en coûte de l'ignorer : un modèle qui ne tient sur aucun palier reste dehors, quels que soient ses mérites.

Deuxième critère : la qualité documentée en français. « Où déposer ma demande de permis de séjour ? » se pose en français, s'instruira sur des documents en français et recevra sa réponse en français. Or la génération augmentée reste nettement moins étudiée hors de l'anglais (Chirkova et al. 2024), et les éditeurs suivent la même pente : le français est presque toujours fondu dans une moyenne multilingue ou européenne dont il ressort rarement seul. Le critère pose donc une double question : le français est-il couvert, et surtout, est-il mesuré quelque part pour lui-même, dans le rapport, sur la carte ou par une évaluation externe ?

Troisième critère : la maturité de la version conversationnelle. Un modèle brut sait continuer un texte ; c'est la post-formation qui en fait un assistant capable de suivre une consigne, de préférer une réponse utile et de refuser ce qu'il ignore. Le critère demande si cette étape est complète et si l'éditeur présente sa version conversationnelle (dite *instruct*) comme prête pour un déploiement : il arrive qu'un modèle par ailleurs estimable annonce lui-même ses versions conversationnelles comme de simples démonstrations (Gouvert et al. 2025). Un assistant public répond à des usagers réels ; il ne peut pas être une démonstration.

Quatrième critère : la licence et les conditions d'accès. Les candidats se partagent entre licences permissives, Apache 2.0 en tête, qui autorisent tout usage sans contrepartie, et licences propriétaires assorties de conditions, parfois doublées d'une barrière au téléchargement. Pour un service public appelé à être redéployé et audité, la simplicité juridique est une qualité en soi.

Cinquième critère : le support Ollama. La pile d'exécution est arrêtée depuis le chapitre 3 ; un modèle sans version exécutable par Ollama obligerait à la rouvrir. La question ne s'arrête pas à la disponibilité, elle remonte la chaîne de provenance posée en 4.1 : qui a

produit le paquet, la bibliothèque officielle, l'éditeur lui-même ou un tiers bénévole ? Un emballage communautaire peut très bien fonctionner ; il repose quand même sur la diligence d'un inconnu, ce qu'un déploiement public doit au minimum savoir.

Sixième critère : la fenêtre effective et la sobriété. Les fenêtres de contexte annoncées se comptent désormais en centaines de milliers de jetons ; encore faut-il qu'une mesure les vérifie, car l'écart entre l'annonce et la capacité réellement exploitable peut être grand, la section suivante l'illustrera. S'y ajoute la leçon de latence de la section 4.1 : à coût fixe par jeton, un modèle économe vaut mieux qu'un modèle bavard, et son mode de réflexion doit pouvoir se désactiver proprement, puisque le mode direct sera le régime de service de Ge-Trouve.

Septième et dernier critère : la gouvernance, la pérennité et la transparence de l'éditeur. L'assistant est destiné à vivre au-delà de ce travail : son modèle devra être maintenu, corrigé, un jour remplacé par un successeur compatible. Le critère interroge la trajectoire de l'éditeur, la vitalité de sa communauté, et le degré d'ouverture : des poids seuls, téléchargeables mais muets sur leur fabrication, jusqu'à l'ouverture complète des données et du code d'entraînement (*fully open*⁴⁵), qui rend le modèle auditable de bout en bout. Le Tableau 4.1 rassemble la grille, au format du Tableau 3.1 : chaque critère, la question qu'il pose, et l'origine de cette question dans le mémoire.

Tableau 4.1 : Les sept critères de choix du modèle de langage

Critère	Question	Origine
1. Exécutabilité dans le gabarit	Une version quantifiée tient-elle, cache compris, dans les enveloppes des deux paliers, et à quel prix ? L'empreinte est-elle attestée par une source primaire ou d'éditeur ?	Budget mémoire du serveur (section 4.1, Figure 4.1)
2. Qualité documentée en français	Le français est-il couvert, et mesuré isolément : rapport, carte ou évaluation externe ?	Question type en français ; français fondu dans les moyennes multilingues (Chirkova et al. 2024)
3. Maturité de la version conversationnelle	La post-formation est-elle complète et la version instruct présentée comme prête pour un déploiement ?	Assistant destiné à des usagers réels (sections 0 et 4.1)
4. Licence et conditions d'accès	La licence est-elle permissive, le téléchargement libre de toute barrière d'accès ?	Redéploiement d'un service public sans insécurité juridique
5. Support Ollama	Une version pour Ollama existe-t-elle, produite par qui : bibliothèque officielle, éditeur, communauté ?	Pile d'exécution du chapitre 3 ; chaîne de provenance (section 4.1)

⁴⁵ Peu de modèles atteignent ce degré : l'usage dominant s'arrête à la publication des poids, la fabrication restant fermée. L'ouverture complète permet au contraire de reproduire le modèle et d'examiner ce qu'il a lu ; la section 4.3 dira où chaque candidat se situe.

6. Fenêtre effective et sobriété	La fenêtre annoncée est-elle vérifiée par une mesure ? Le modèle est-il économe en jetons, sa réflexion désactivable ?	Latence sur processeur et mode direct (section 4.1)
7. Gouvernance, pérennité et transparence	L'éditeur maintient-il ses modèles, la communauté vit-elle, jusqu'où va l'ouverture, des poids seuls au fully open ?	Patron du chapitre 3 ; service destiné à durer

(élaboration personnelle)

Les questions sont posées ; il reste des candidats à qui les poser. C'est l'objet de la section suivante, qui s'ouvre sur la convention rendant la comparaison honnête : une date de référence unique.

4.3 Les candidats

La liste des candidats ne sort pas d'un chapeau. Elle vient d'une veille documentaire menée en juin et juillet 2026, en croisant la presse spécialisée, les annonces des éditeurs, les classements publics et les bibliothèques de distribution : les modèles retenus sont ceux qui revenaient le plus souvent dans ces sources. Trois règles ont ensuite filtré cette récolte. Entrer dans le périmètre du chapitre, d'abord : des poids ouverts, exécutables localement. Tenir dans le gabarit de la section 4.1, ensuite : autour de sept à douze milliards de paramètres une fois quantifiés, la limite au-delà de laquelle même le palier plancher ne suit plus. Enfin, ne retenir de chaque famille que la génération la plus récente qui respecte les deux règles précédentes. Cinq candidats, portés par cinq éditeurs, ont passé ce filtre ; les modèles remarquables croisés en chemin mais écartés sont documentés en fin de section, chacun avec son motif.

La comparaison suit une convention simple : le 16 juillet 2026 sert de date de référence unique. Modèles, cartes et classements bougent chaque semaine ; fixer une date rend les portraits honnêtes, chacun décrivant l'état constaté à ce jour⁴⁶. Chaque portrait parcourt les sept critères et précise, en suivant la hiérarchie des sources posée en tête de chapitre, d'où vient chaque fait : du rapport technique, de la carte ou de la distribution. Tous les candidats concourent dans leur version conversationnelle, celle que leur nom signale par un suffixe, Instruct le plus souvent : c'est elle, et elle seule, qu'un assistant public déploierait. Les cinq sont présentés du plus ancien au plus récent.

Le plus ancien s'appelle Llama 3.1 8B Instruct, publié par Meta le 23 juillet 2024, et son premier argument est d'être installé partout : son paquet dans la bibliothèque officielle

⁴⁶ Quelques relevés de distribution (pages de la bibliothèque Ollama, classement compar:IA) ont été effectués les 17 et 18 juillet 2026 ; les références bibliographiques portent leur date exacte de consultation.

d'Ollama cumule plus de 117 millions de téléchargements (Ollama 2024a), de loin le plus gros volume des cinq candidats. L'exécutabilité ne pose aucun problème : 4,9 Go en quantification quatre bits, la plus étroite des enveloppes de la Figure 4.1 l'absorbe avec de la marge. Le rapport technique, déjà croisé à la section 4.1 pour sa doctrine de factualité, décrit un entraînement de 15 600 milliards de jetons et une post-formation en six tours, préférences et refus compris, qui en fait l'une des versions conversationnelles les plus abouties de sa génération (Grattafiori et al. 2024). Sa fenêtre de 128 000 jetons a un avantage rare : elle a été vérifiée par un tiers. Dans les mesures publiées par l'équipe d'Apertus, il obtient 86,2 sur RULER à 32 000 jetons (Project Apertus et al. 2025). Les réserves tiennent en trois points. Le français, d'abord : le rapport ne l'isole jamais, sa mesure multilingue étant une moyenne de sept langues traduites ; le score français de 62,34 en MMLU⁴⁷ n'existe que sur la carte (Meta 2024), première illustration du glissement de source annoncé. « L'arène »⁴⁸ francophone, elle, mesure le candidat lui-même : 99e du classement au score de préférence, une position assise sur plus de 7 500 duels, l'un des plus gros volumes de l'arène, dans cette fin de classement où les votants placent les petits modèles derrière les géants (compar:IA 2026). La licence, ensuite : la Llama 3.1 Community License impose ses conditions et le téléchargement sur Hugging Face passe par une barrière d'accès, même si la bibliothèque Ollama, elle, sert le modèle sans formalité. La trajectoire, enfin : les connaissances s'arrêtent fin 2023, ce que l'architecture RAG relativise puisque le modèle est ici rédacteur et non source. La famille a une suite, mais elle a changé d'échelle : les Llama 4 d'avril 2025, avec 109 et 400 milliards de paramètres, sont hors de portée du serveur, et Meta a depuis réorienté sa feuille de route, le prochain Llama n'étant plus annoncé avant 2027 (Wikipedia 2026). Dans le gabarit qui nous occupe, Llama 3.1 8B est donc un terminus ; la sous-section des écartés reviendra sur ses successeurs.

Un peu plus d'un an après lui arrive le candidat suisse. Apertus 8B Instruct, publié en septembre 2025 par l'initiative Swiss AI, qui réunit l'EPFL, l'ETH Zurich et le Centre suisse de calcul scientifique, occupe une position que personne ne lui conteste : seul modèle des cinq entièrement ouvert, données, code et recettes d'entraînement compris, il est aussi le seul que le classement francophone compar:IA étiquette *fully-open-source*,

⁴⁷ MMLU (*Massive Multitask Language Understanding*) soumet au modèle des questions à choix multiple couvrant une cinquantaine de matières, du droit à la médecine ; le score mesure l'étendue des connaissances restituées à livre fermé, sans document fourni.

⁴⁸ Le terme « arène » désigne le dispositif de compar:IA : deux modèles anonymes répondent à la même question, l'utilisateur vote pour la meilleure réponse sans savoir qui l'a produite, et le classement se construit à partir de ces duels. Les votants ne choisissent donc pas les modèles qu'ils évaluent ; un grand nombre de votes signifie qu'un modèle a été souvent tiré, pas qu'il est populaire.

tous les autres modèles ouverts y étant *open-weights* (compar:IA 2026). Le rapport traite la conformité comme un problème d'ingénierie : respect rétroactif des refus d'indexation, fonction de perte qui empêche la restitution mot à mot des textes appris, et, honnêteté rare, un coût chiffré, environ 4,3 % de performance moyenne perdus à cause du filtrage des données (Project Apertus et al. 2025). L'ancrage suisse n'est pas cosmétique : l'alignement suit une charte inspirée des valeurs constitutionnelles suisses, et le rapport introduit sa propre évaluation, SwitzerlandQA, qui interroge géographie, histoire, vie politique et assurances, c'est-à-dire à peu près le terrain de Ge-Trouve (Project Apertus et al. 2025), en gardant à l'esprit qu'elle mesure ce que le modèle sait par cœur, alors que notre pipeline lui apportera les documents. Les faiblesses sont nettes, et le rapport les assume : mathématiques, code et raisonnement en retrait ; un français servi par un corpus à 40 % non anglophone mais jamais mesuré isolément, l'arène francophone n'évaluant que sa version de 70 milliards, 74e du classement ; une fenêtre de 65 536 jetons qui se dégrade vite, 91,2 sur RULER à 4 000 jetons mais 65,9 à 32 000 (Project Apertus et al. 2025). La carte ajoute un score juridique, 22,44 sur LEXam⁴⁹, absent des 119 pages du rapport : une donnée déclarative, à prendre comme telle (Swiss AI Initiative 2025). La distribution, enfin, est son point faible : aucun canal Ollama d'éditeur, une conversion doublement communautaire, quantifiée par Unsloth⁵⁰ puis empaquetée par un bénévole, avec 3 642 téléchargements (Rosselli 2026), plus de quatre ordres de grandeur derrière Llama.

En décembre 2025, Mistral AI publie Ministral 3 8B Instruct, successeur de Mistral 7B dans les petits gabarits de la maison. Le modèle est né par distillation en cascade : élaguer un modèle plus grand, lui réapprendre à imiter le modèle d'origine, recommencer, jusqu'à concentrer dans huit milliards de paramètres une partie du savoir-faire de Mistral Small 3.1 (Liu et al. 2026). La continuité joue pour lui : même éditeur européen, licence Apache 2.0, et une distribution par le canal officiel de la bibliothèque Ollama, où le paquet 8b en quatre bits pèse 6,0 Go pour une fenêtre annoncée de 262 144 jetons (Ollama 2025). Le rapport documente une qualité que la section 4.1 a élevée au rang de critère : la sobriété. Le modèle produit des réponses nettement plus courtes que ses concurrents à qualité comparable, un avantage direct quand chaque jeton coûte

⁴⁹ LEXam rassemble plusieurs milliers de questions tirées de 340 examens de facultés de droit, couvrant droit suisse, européen et international, en anglais et en allemand ; le chiffre cité correspond à ses questions ouvertes, celles qui exigent un raisonnement juridique rédigé.

⁵⁰ Unsloth est une jeune entreprise connue de la communauté des modèles ouverts pour ses outils libres d'optimisation et, surtout, pour ses conversions quantifiées au format GGUF, publiées sur Hugging Face peu après chaque sortie de modèle. Tiers indépendant, elle n'engage ni l'éditeur du modèle converti ni la bibliothèque Ollama.

du temps (Liu et al. 2026). Les zones d'ombre sont réelles. L'empreinte, d'abord : le rapport n'en chiffre aucune ; la promesse « moins de 12 Go quantifié » vient de l'annonce commerciale (Mistral AI 2025), et les 6,0 Go du paquet officiel, en butée de l'enveloppe du palier plancher, s'expliquent en partie par un encodeur de vision de 410 millions de paramètres, embarqué et gelé, dont Ge-Trouve n'a pas l'usage. Le français, ensuite : listé parmi les langues servies, il se fond dans une moyenne européenne à 73,4 sans jamais apparaître seul, et les évaluations du rapport, menées en interne, comparent le modèle aux générations précédentes de ses concurrents plutôt qu'aux candidats de ce comparatif. La transparence, enfin : le rapport ne dit rien des données d'entraînement, et c'est la distribution qui donne l'information que la source primaire tait ; le gabarit de conversation livré avec les paquets situe la base de connaissances à octobre 2023 (Ollama 2025).

Au printemps 2026 arrive le plus multilingue des cinq. Qwen 3.5 9B Instruct, publié en mars par l'équipe Qwen d'Alibaba sous licence Apache 2.0, revendique sur sa carte 201 langues et une fenêtre de 262 144 jetons (Qwen Team 2026). Son portrait souffre pourtant d'un manque particulier : il est le seul des cinq sans article scientifique propre. La source primaire la plus proche est le rapport de la génération précédente, publié un an plus tôt : ce qu'il documente date la lignée sans noter le candidat, et le portrait assume cette lecture par procuration (Yang et al. 2025). La lignée a des arguments sérieux : c'est elle qui a fait de la réflexion un mode que l'on active ou coupe selon le besoin, plutôt qu'un modèle à part, et sa post-formation cible explicitement les tâches de génération augmentée, avec des récompenses dédiées à la fidélité au contexte fourni (Yang et al. 2025). C'est elle aussi qui offre la seule mesure du français isolé de tout le comparatif, et cette mesure est à double tranchant : 69,8 de moyenne en mode réflexion, mais 51,5 en mode direct (Yang et al. 2025). Or le mode direct sera le régime de service de Ge-Trouve, et la section 4.1 a montré que la réflexion se paie en latence sans servir la restitution. Le reste du dossier se lit sur la carte et la distribution. Les « 9 milliards » du nom sont en réalité 9,65 milliards de paramètres, et le paquet officiel de la bibliothèque Ollama pèse 6,6 Go en quatre bits : au-delà de l'enveloppe du palier plancher, dans celle du palier de confort seulement (Ollama 2026a). La distribution elle-même est ambiguë : la bibliothèque officielle le sert, mais la carte de l'éditeur recommande d'autres moteurs d'inférence et ne mentionne jamais Ollama (Qwen Team 2026), un canal réel que personne ne revendique vraiment.

Le dernier arrivé est le plus attendu. Gemma 4 12B Instruct, publié par Google DeepMind en juin 2026 au sein d'une famille lancée au printemps, sous licence Apache 2.0, arrive avec un atout qu'aucun autre candidat n'offre : son rapport est le seul des cinq à chiffrer

l'empreinte mémoire de ses modèles quantifiés, dans la table même qui a fourni les chiffres de la section 4.1 (Gemma Team et al. 2026). C'est sa force et son paradoxe : le mieux documenté est aussi le plus exigeant, ses 7,65 Go dépassant l'enveloppe du palier plancher plus nettement encore que Qwen ; seul le palier de confort lui rend de la marge. Le rapport a d'autres atouts utiles pour notre usage : une post-formation de factualité qui entraîne explicitement l'attribution au contexte, la prudence et le refus (Gemma Team et al. 2026), soit le critère 3 pris au mot ; une architecture unifiée, sans encodeurs séparés, un pari technique dont ce gabarit est l'essai ; une réflexion qui s'active et se coupe par un simple jeton de contrôle, documentée jusque dans sa page de distribution (Ollama 2026b) ; et la meilleure tenue de fenêtre du comparatif, 91,2 sur RULER à 128 000 jetons, mesurée en source primaire (Gemma Team et al. 2026). Le français, lui, reste un angle mort surprenant : le rapport ne le mesure qu'à l'oral, par la transcription, et c'est l'arène francophone qui donne l'indice le plus parlant, la famille y dominant les modèles ouverts, le 31B huitième du classement général et premier ouvert, le 26B dix-septième (compar:IA 2026), sans que le 12B y soit lui-même mesuré, même si son prédécesseur direct, Gemma 3 12B, y occupe le 42e rang (compar:IA 2026). Dernier point faible, la transparence : les données d'entraînement tiennent en une phrase et le rapport n'a pas de section de limites ; l'éditeur documente en revanche lui-même l'intégration Ollama de ses modèles, seul des cinq à le faire.

Tableau 4.2 : Fiche d'identité des cinq candidats

Candidat	Éditeur	Sortie	Licence	Paramètres	Contexte annoncé	Distribution Ollama
Llama 3.1 8B Instruct	Meta	07/2024	Llama 3.1 Community License	8 milliards	128 000	Bibliothèque officielle (4,9 Go)
Apertus 8B Instruct	Swiss AI Initiative	09/2025	Apache 2.0	8 milliards	65 536	Conversion communautaire (5,1 Go)
Ministral 3 8B Instruct	Mistral AI	12/2025	Apache 2.0	8 milliards	262 144	Bibliothèque officielle (6,0 Go)
Qwen 3.5 9B Instruct	Alibaba (équipe Qwen)	03/2026	Apache 2.0	9,65 milliards	262 144	Bibliothèque officielle (6,6 Go)
Gemma 4 12B Instruct	Google DeepMind	06/2026	Apache 2.0	12 milliards	262 144	Bibliothèque officielle (7,6 Go)

(élaboration personnelle, d'après les sources citées dans la section 4.3)

Restent ceux que la veille a croisés sans les retenir, et leurs motifs confirment les règles du chapitre. GLM-5.2, d'abord, la borne haute : 744 milliards de paramètres en

architecture à mélange d'experts⁵¹ (GLM-5-Team et al. 2026) ; la section 4.1 a fait le compte, environ 239 Go dans sa compression la plus agressive, plus de treize fois le plus généreux des paliers envisagés. Il rappelle ce que le périmètre coûte en performance brute, et que ce coût est un choix, pas un oubli. gpt-oss-20b, ensuite, l'écarté de justesse : les 16 Go exigés par son éditeur dépassent le palier plancher tout entier et ce que le palier de confort laisse au modèle ; l'éditeur le décrit par ailleurs comme principalement anglophone (OpenAI et al. 2025). Lucie-7B, enfin, le cas le plus instructif. Portée par OpenLLM-France, riche d'un corpus à 40 % francophone qui puise aux registres mêmes du français administratif, Gallica⁵², textes officiels, débats parlementaires, elle tient dans le gabarit et se distribue par l'espace Ollama de son éditeur (OpenLLM-France 2025). Son écartement ne doit rien au filtre : ses auteurs présentent eux-mêmes les versions conversationnelles comme des démonstrations, sans l'alignement complet qu'exige le critère 3 (Gouvert et al. 2025). Elle laisse deux choses au chapitre : la preuve qu'un écart peut séparer l'annonce de la mesure, sa fenêtre de 32 000 jetons de base tombant à 22 000 vérifiés pour la version conversationnelle, exactement ce que le critère 6 redoute ; et un corpus dont le chapitre 8 reparlera. Les successeurs de Llama, promis plus haut, se règlent en une ligne : les deux Llama 4 d'avril 2025, nommés Scout et Maverick, 109 et 400 milliards de paramètres en mélange d'experts, hors gabarit⁵³ ; Phi-4, la proposition compacte de Microsoft, 14 milliards, dépasse aussi le gabarit⁵⁴ ; chez Mistral, Nemo cède à la règle de la génération la plus récente et Small au gabarit. Quant aux modèles servis par interface distante, ils n'ont jamais concouru : le périmètre du chapitre les avait écartés avant tout critère. Les sept questions peuvent maintenant être posées aux cinq candidats : c'est l'objet de la section suivante.

4.4 La comparaison et la décision

Le Tableau 4.3 confronte les cinq candidats aux sept critères. Chaque case résume ce que les portraits de la section 4.3 ont établi. La lecture se fait en deux temps, puisque le chapitre a retenu deux paliers de serveur : d'abord au palier plancher, où le premier critère élimine une partie des candidats, puis au palier de confort, où tous peuvent tourner et où la comparaison porte sur la qualité.

⁵¹ Une architecture à mélange d'experts (*mixture of experts*, MoE) répartit les paramètres en sous-réseaux spécialisés dont seule une fraction s'active à chaque jeton : la mémoire doit loger le total, la vitesse ne paie que la fraction active.

⁵² Gallica est la bibliothèque numérique de la Bibliothèque nationale de France, qui donne accès à plusieurs millions de livres, journaux et documents patrimoniaux numérisés.

⁵³ Annonce des Llama 4 Scout et Maverick : <https://ai.meta.com/blog/llama-4-multimodal-intelligence/> [consulté le 16 juillet 2026].

⁵⁴ Carte du modèle Phi-4 : <https://huggingface.co/microsoft/phi-4> [consulté le 16 juillet 2026].

Tableau 4.3 : La confrontation des cinq candidats aux sept critères

Candidat	Llama 3.1 8B	Apertus 8B	Ministral 3 8B	Qwen 3.5 9B	Gemma 4 12B
1. Exécutabilité	À l'aise dès le plancher (4,9 Go)	À l'aise dès le plancher (5,1 Go)	En butée du plancher (6,0 Go)	Palier de confort seulement (6,6 Go)	Palier de confort seulement (7,6 Go)
2. Français documenté	Carte seule (62,34 MMLU) ; 99e de l'arène	Jamais isolé ; arène : le 70B seulement (74e)	Fondu dans une moyenne européenne	Lignée mesurée isolément, faible en mode direct	À l'oral seulement ; famille 8e et 17e, prédécesseur 42e
3. Maturité conversationnelle	Complète, six tours, refus compris	Complète, alignée sur une charte	Complète	Complète (lignée), pensée pour le RAG	Factualité entraînée à l'attribution
4. Licence et accès	Licence à conditions, barrière d'accès	Apache 2.0	Apache 2.0	Apache 2.0	Apache 2.0
5. Support Ollama	Bibliothèque officielle, le plus installé	Communautaire, doublement	Bibliothèque officielle	Bibliothèque officielle, non revendiquée par l'éditeur	Bibliothèque officielle, documentée par l'éditeur
6. Fenêtre et sobriété	86,2 à 32 000 (mesure tierce)	65,9 à 32 000, la plus faible mesurée	Annoncée sans mesure ; sobriété documentée	Annoncée sans mesure du candidat ; réflexion commutable	91,2 à 128 000 (source primaire) ; réflexion commutable
7. Gouvernance et transparence	Écosystème immense ; lignée en fin de cycle	Entièrement ouvert ; consortium public	Éditeur actif ; muet sur les données	Famille active ; sans article propre	Trajectoire soutenue ; données en une phrase

(élaboration personnelle, d'après les portraits de la section 4.3)

Au palier plancher, le premier critère règle une bonne partie de la question. Gemma et Qwen ne tiennent pas : 7,6 et 6,6 Go de poids pour une enveloppe de 5 à 6 Go, le calcul de la section 4.1 est sans appel. Ministral passe tout juste, sans marge pour les pointes de charge ni pour un cache un peu grand, ce qui est risqué pour un service public. Restent Llama et Apertus, tous deux à l'aise. Entre les deux, la différence se fait sur les critères 5 et 6. La distribution d'Apertus dépend d'un empaquetage bénévole, celle de Llama est la plus utilisée des cinq. Et la fenêtre d'Apertus se dégrade vite aux longueurs

dont un pipeline documentaire a besoin : 65,9 sur RULER à 32 000 jetons, contre 86,2 pour Llama. Son ouverture complète, unique dans le comparatif, ne suffit pas à compenser ces deux faiblesses pour un service destiné à des usagers. Au palier plancher, la grille désigne donc Llama 3.1 8B : le candidat le plus installé, le mieux vérifié, et sans problème d'exécution. Ce résultat n'est pas la conclusion du chapitre, mais il compte : quel que soit le palier finalement souscrit, le projet dispose d'un choix sûr sur le plus petit des deux.

Au palier de confort, la situation change : tous les candidats tiennent, et la question devient celle du meilleur modèle. Ministral garde sa sobriété, un vrai avantage sur processeur, mais son dossier reste le moins documenté des cinq : pas d'empreinte dans le rapport, un français noyé dans une moyenne, des évaluations internes faites contre des modèles d'anciennes générations. Qwen couvre le plus de langues et sa lignée a été entraînée pour la génération augmentée, mais son portrait repose sur le rapport de la génération précédente, et la seule mesure française de sa lignée chute justement en mode direct, celui que Ge-Trouve utilisera. Gemma apporte des choses qu'aucun autre n'a : la seule empreinte chiffrée en source primaire, une post-formation qui l'entraîne à répondre à partir du contexte fourni, à rester prudent et à refuser quand il ne trouve pas, ce qui correspond au rôle du modèle dans un pipeline RAG, la seule fenêtre mesurée par son propre éditeur, 91,2 sur RULER à 128 000 jetons (Gemma Team et al. 2026), une réflexion qui se désactive par un simple jeton, et les meilleurs résultats francophones de l'arène : la famille aux 8e et 17e rangs, le prédécesseur direct du candidat au 42e, alors que Llama est 99e (compar:IA 2026). Aucun de ces chiffres ne mesure le candidat lui-même, le portrait l'a précisé. Mais ils vont tous dans le même sens.

La décision conjointe annoncée à la section 4.1 peut alors se poser. Le palier plancher offre un très bon modèle ; le palier de confort, pour un surcoût mensuel contenu, en offre un meilleur. Concrètement, pour l'utilisateur qui demande où déposer sa demande de permis de séjour, la différence ne sera pas dans le style de la réponse, mais dans sa fiabilité : un modèle entraîné à s'appuyer sur les documents fournis se trompera moins souvent de formulaire ou d'adresse, et dira plus volontiers qu'il ne sait pas quand les documents ne suffisent pas. Le coût, lui, est technique : à douze milliards de paramètres, chaque jeton prend environ une fois et demie le temps d'un modèle de huit milliards sur le même processeur. Les processeurs supplémentaires du palier de confort compensent une partie de cet écart, et la partie pratique mesurera la latence réelle avant toute mise en service. Au final, le gain touche la qualité des réponses, donc le service lui-même ; le coût se limite à un temps d'attente un peu plus long, qui pourra être mesuré et réduit.

Ce travail retient donc **Gemma 4 12B Instruct**, et avec lui le palier de confort. Trois raisons, par ordre d'importance. D'abord, le modèle correspond au besoin : sa post-formation de factualité l'entraîne exactement à ce que le chapitre 2 a décrit comme la qualité principale d'un assistant administratif, répondre à partir des documents fournis et refuser quand ils ne suffisent pas. Ensuite, l'exécution est certaine : c'est le seul candidat dont l'empreinte, cache compris, est chiffrée par une source primaire, ce qui permet de dimensionner le serveur avec un calcul et non une estimation. Enfin, le dossier francophone : aucun candidat n'a de mesure directe de son français écrit, mais aucun n'approche les résultats que l'arène donne à sa famille.

Ce choix a des inconvénients, qu'il faut nommer. La transparence, d'abord : les données d'entraînement de Gemma tiennent en une phrase et le rapport n'a pas de section de limites. Apertus reste la référence de l'ouverture complète, et son écartement montre ce que ce comparatif accepte de perdre en échange de la qualité de service. La latence, ensuite, acceptée et à mesurer. Le français écrit, enfin, jamais mesuré isolément en source primaire : ce manque, commun aux cinq candidats, est justement ce que l'évaluation du chapitre 7 viendra combler sur le cas genevois, et c'est une partie de l'apport de ce travail. Un point de sécurité mérite d'être noté : si le déploiement devait finalement se limiter au palier plancher, le choix se reporterait sur Llama 3.1 8B, et l'interchangeabilité garantie par le framework au chapitre 3 rendrait le remplacement simple. Dernier avantage, en lien avec le début de ce chapitre : des poids téléchargés, sous licence Apache 2.0, tournent sur le serveur du projet sans qu'une décision extérieure puisse les couper.

Le premier composant est choisi. Le deuxième est le modèle d'embeddings, celui qui transforme questions et documents en vecteurs comparables : c'est l'objet du chapitre suivant.

5. Choix des embeddings

Deuxième composant : le modèle d'embeddings, celui qui ne rédigera aucune réponse mais dont dépendra tout ce que l'assistant retrouve. Son travail reste invisible pour l'utilisateur : transformer chaque texte en une suite de nombres, de manière que deux textes proches par le sens donnent des suites proches. C'est cette mécanique, décrite à la section 2.4, qui doit permettre de retrouver la page de ge.ch consacrée à l'autorisation de séjour quand un usager écrit « Où déposer ma demande de permis de séjour ? », alors que les deux formulations n'ont presque aucun mot en commun. Le cadre du choix ne change pas : le modèle s'exécutera en local sur le serveur du projet, dans le poste de mémoire que la section 4.1 lui a réservé, quelques centaines de mégaoctets à 1,5 Go ; les services d'embeddings en nuage sont hors périmètre par construction, pour la même règle qu'au chapitre précédent.

La démarche suit le chemin des deux chapitres précédents. La section 5.1 précise ce que ce modèle fait exactement dans le pipeline et les contraintes que le projet lui impose, de la mémoire disponible à la longueur des textes à encoder. La section 5.2 en tire une grille de sept critères, la section 5.3 présente les trois candidats retenus et les pistes écartées, et la section 5.4 les confronte pour arrêter le choix.

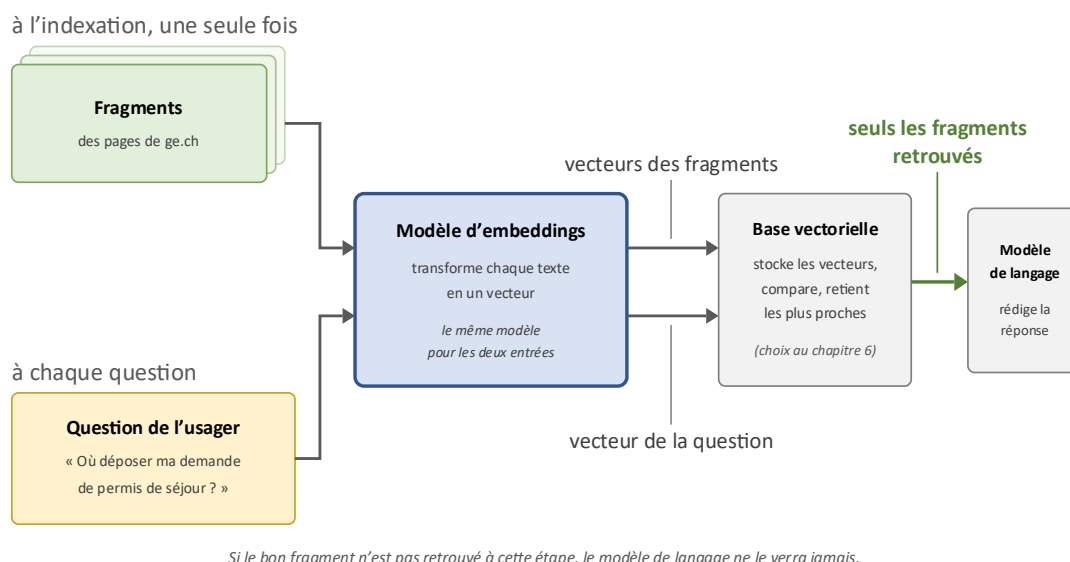
5.1 Le rôle du modèle d'embeddings et les contraintes du projet

La section 2.4 a décrit la mécanique une première fois : chaque page de ge.ch est découpée en fragments, chaque fragment devient un vecteur, et la question de l'utilisateur, convertie par le même modèle, est comparée à tous ces vecteurs pour retrouver les plus proches. Le modèle d'embeddings est la pièce qui fabrique ces vecteurs. Tout se juge à sa capacité de placer côte à côte des textes qui se répondent : un bon modèle rangera la page consacrée à l'autorisation de séjour tout près de la question « Où déposer ma demande de permis de séjour ? », alors que les deux formulations n'ont presque aucun mot en commun. La conséquence est mécanique, car le modèle de langage ne voit que les fragments retrouvés : si le bon fragment ne remonte pas, même une rédaction parfaite répondra à côté.

La difficulté propre du maillon tient dans un déséquilibre. D'un côté une question d'une ligne, de l'autre des fragments tirés de pages longues, rédigées dans la langue administrative : il faut apparier deux textes qui ne se ressemblent pas. La littérature parle de recherche asymétrique, et c'est pour elle que la recherche dense de passages a été conçue (Karpukhin et al. 2020). La distinction compte au moment de lire les

classements : un modèle excellent pour juger que deux phrases disent la même chose peut échouer à retrouver un document long à partir d'une question courte, et les bancs d'évaluation du domaine mesurent ces épreuves séparément (Muennighoff et al. 2022). La Figure 5.1 replace ce maillon dans le pipeline.

Figure 5.1 : Le maillon embeddings dans le pipeline de Ge-Trouve



(élaboration personnelle)

Les contraintes du projet prolongent celles du chapitre 4. L'exécution d'abord : le modèle tournera en local sur le serveur, par Ollama comme le modèle de langage (section 4.1), et chacun des trois candidats de la section 5.3 y dispose d'un paquet officiel. La mémoire ensuite : le poste réservé va de quelques centaines de mégaoctets à 1,5 Go. L'enveloppe paraît confortable ; elle écarte pourtant d'office une partie de la littérature récente, tournée vers des modèles d'embeddings de plusieurs milliards de paramètres (Vera et al. 2025).

Le stockage est un poste que le chapitre 4 n'avait pas. Chaque fragment indexé coûte la taille de son vecteur, et cette taille varie d'un facteur dix selon les modèles, de 1,2 à 16,4 ko par texte dans le premier grand comparatif du domaine (Muennighoff et al. 2022). Des modèles récents proposent pour cette raison des vecteurs à taille réglable⁵⁵ : entraînés à concentrer l'essentiel du sens dans les premières composantes, ils acceptent d'être tronqués, contre une précision moindre. La dimension retenue engagera donc à la fois la qualité de la recherche et le volume de la base vectorielle,

⁵⁵ *Matryoshka Representation Learning* (MRL), littéralement apprentissage de représentations « en poupées russes » : le vecteur complet contient des versions réduites de lui-même, utilisables telles quelles. Deux des trois candidats du comparatif l'annoncent (Zhang et al. 2025; Vera et al. 2025).

dont le chapitre 6 arbitrera le choix. Dernière contrainte, la longueur d'encodage : chaque modèle plafonne à un nombre de jetons par texte, et ce plafond varie fortement d'un candidat à l'autre (Tableau 5.2). Un plafond bas n'interdit rien, puisque les pages seront de toute façon découpées ; il impose un découpage plus fin, dont la partie pratique mesurera les effets.

De ces contraintes, la section suivante tire la grille qui servira à comparer les candidats.

5.2 Les critères de comparaison

La section 4.2 se méfiait de classements qui mesuraient autre chose que notre situation. Le problème change ici de nature : le banc de référence des embeddings, MTEB⁵⁶, mesure précisément la récupération, le nom que la littérature donne à la tâche décrite en 5.1 (Muennighoff et al. 2022). Les précautions portent donc moins sur ce qui est mesuré que sur la manière de le lire. La grille reste qualitative, pour les raisons exposées en 3.2, et la souveraineté n'y figure toujours pas : comme au chapitre 4, elle a agi en amont, en écartant les services en nuage avant tout concours.

Premier critère, la qualité mesurée en récupération. Le score se lira sur le classement public issu de MTEB, dans sa version multilingue actuelle (Enevoldsen et al. 2025), avec deux règles. La colonne d'abord : aucun modèle ne domine toutes les tâches, la moyenne générale ne dit donc rien de l'exercice qui compte ici ; c'est la colonne récupération qui fera foi, notée par la place des bons fragments parmi les dix premiers résultats⁵⁷ (Muennighoff et al. 2022). La date ensuite : le classement évolue en continu, tout score s'y cite relevé et daté (Enevoldsen et al. 2025).

Deuxième critère, la performance en français. Les rapports d'éditeurs fondent le français dans des moyennes à plusieurs centaines de langues. La différence avec le chapitre 4 est qu'un instrument dédié existe désormais : le volet français du même classement, construit par Ciancone et al. (2024). Le critère se lira là, même colonne, même date.

Troisième critère, l'empreinte mémoire. L'enveloppe de la section 4.1 fait foi, et le chiffre comparé sera le même pour chacun : la taille du paquet Ollama réellement téléchargé, relevée sur les fiches de distribution (section 5.3).

⁵⁶ MTEB (*Massive Text Embedding Benchmark*) est le banc d'essai de référence des modèles d'embeddings : des tâches variées évaluées sous un protocole commun, un code ouvert et un classement public hébergé sur Hugging Face. L'article fondateur date de 2022 ; le banc a depuis été étendu en version massivement multilingue et doté d'un volet français.

⁵⁷ *Normalized discounted cumulative gain* à dix résultats (nDCG@10) : la métrique de récupération du banc, héritée de BEIR (Muennighoff et al. 2022). Elle vaut d'autant plus que les bons documents sortent haut dans la liste des dix premiers.

Quatrième critère, la dimension des vecteurs. La section 5.1 l'a montré, chaque fragment stocké coûte sa dimension ; à qualité proche, un vecteur plus court donne une base plus légère et des comparaisons plus rapides. La grille note la dimension et la présence de vecteurs à taille réglable.

Cinquième critère, la longueur de contexte. Le plafond de jetons par texte encodé conditionne le découpage des pages : plus il est bas, plus les fragments devront être courts.

Sixième critère, la licence et les conditions d'accès. Le critère est repris du chapitre 4 : licence permissive ou conditions d'utilisation propres à l'éditeur, téléchargement libre ou soumis à une acceptation préalable. Pour un service public, la simplicité juridique reste une qualité en soi (section 4.2).

Septième critère, la maturité et la disponibilité. Il rassemble l'ancienneté du candidat, la relecture de son article par les pairs, la vitalité de son écosystème et la provenance de son paquet Ollama, selon la chaîne posée en 4.1 : bibliothèque officielle, éditeur ou tiers. Le Tableau 5.1 rassemble la grille, au même format que le Tableau 3.1 et le Tableau 4.1.

Une convention de lecture complète la grille. L'un des candidats sait chercher de trois manières, par un vecteur unique, par des mots-clés pondérés ou par une comparaison mot à mot, et ses meilleurs scores publiés combinent les trois (Chen et al. 2024). Le serveur du projet, lui, n'exposera que la première, dite dense. Pour que la comparaison reste honnête, tous les scores se liront sur cette fonction commune, jamais sur des combinaisons que le déploiement ne servira pas.

Tableau 5.1 : Les sept critères de comparaison du modèle d'embeddings

Critère	Question	Origine
1. Qualité en récupération	Quel score en récupération sur le classement MTEB, relevé et daté, colonne récupération et non moyenne ?	Tâche du maillon (section 5.1) ; banc de référence (Muennighoff et al. 2022; Enevoldsen et al. 2025)
2. Performance en français	Quel score sur le volet français du classement, même colonne, même date ?	Question type en français ; volet dédié (Ciancone et al. 2024)
3. Empreinte mémoire	Le paquet Ollama tient-il dans l'enveloppe réservée, avec quelle marge ?	Budget mémoire (section 4.1) ; fiches de distribution (section 5.3)
4. Dimension des vecteurs	Quelle dimension, et des vecteurs à taille réglable sont-ils prévus ?	Coût de stockage (section 5.1)

5. Longueur de contexte	Combien de jetons par texte encodé, avec quelle contrainte sur le découpage ?	Découpage des pages (section 5.1)
6. Licence et accès	La licence est-elle permissive, le téléchargement libre de toute barrière ?	Service public (section 4.2)
7. Maturité et disponibilité	Article relu par les pairs, écosystème vivant, paquet officiel ?	Chaîne de provenance (section 4.1) ; patron des chapitres 3 et 4

(élaboration personnelle)

Les questions sont posées ; la section suivante présente les trois candidats qui s'y soumettront, puis les pistes écartées.

5.3 Les candidats

La sélection suit la méthode de la section 4.3, appliquée en juillet 2026 aux modèles d'embeddings : croiser les classements publics, les bibliothèques de distribution et les annonces d'éditeurs, puis filtrer par trois règles. Des poids ouverts exécutables localement, d'abord. Une empreinte dans l'enveloppe de la section 4.1, ensuite. Par famille, enfin, la génération la plus récente. Trois candidats en sortent ; les pistes croisées mais écartées ferment la section, chacune avec son motif. Même convention qu'au chapitre 4 : le 20 juillet 2026 sert de date de référence unique, cartes, fiches et classements décrivant l'état constaté ce jour-là. Les trois candidats sont présentés du plus ancien au plus récent.

Le plus ancien s'appelle BGE-M3, publié en février 2024 par la BAAI, un institut de recherche pékinois, et c'est le mieux installé des trois. Son article est le seul du trio relu par les pairs (Chen et al. 2024), sa licence MIT est la plus simple qui soit (BAAI 2024), et son paquet cumule plus de cinq millions de téléchargements, le plus gros volume des trois candidats (Ollama 2024b). C'est aussi le modèle aux trois manières de chercher évoqué en 5.2, dont le déploiement ne servirait que la fonction dense. Dernier titre : il est le seul candidat déjà mesuré par le volet français du banc, où il figurait en 2024 comme premier modèle à poids ouverts du classement général (Ciancone et al. 2024). Son revers est l'âge : dans un domaine qui court, il concourt avec un socle de début 2024, et il est le plus lourd des trois (Tableau 5.2).

Le deuxième candidat inverse une situation du chapitre 4. Le candidat Qwen des modèles de langage concourait sans rapport technique dédié à sa version (section 4.3) ; Qwen3-Embedding-0.6B, publié en juin 2025 par l'équipe Qwen d'Alibaba, arrive au contraire avec le sien (Zhang et al. 2025). Plus petit d'une famille de trois tailles, bâti sur les modèles de langage Qwen3 déjà croisés au chapitre précédent, il documente en

source primaire les deux mécanismes annoncés en 5.1 : les vecteurs à taille réglable, et des consignes de tâche ajoutées à la question avant l'encodage, jamais au document (Zhang et al. 2025). Sa licence Apache 2.0 est permissive et son téléchargement libre de toute barrière d'accès (Qwen Team 2025). Son rapport le place, à sa sortie, devant des modèles dix fois plus grands sur le banc multilingue (Zhang et al. 2025) ; mais ce même rapport n'isole jamais le français, et le critère 2 se jouera donc entièrement sur le relevé.

Le plus récent est aussi le plus léger. EmbeddingGemma, publié en septembre 2025 par Google DeepMind, l'éditeur du modèle de langage retenu en 4.4, compte 308 millions de paramètres et pousse la sobriété jusque dans l'entraînement ; sa quantification est préparée dès cette étape, comme pour Gemma 4, et ses vecteurs à taille réglable descendent jusqu'à 128 dimensions (Vera et al. 2025). Son rapport le donne, à sa sortie, premier des modèles de moins de 500 millions de paramètres du classement multilingue (Vera et al. 2025) ; l'affirmation vient de l'éditeur, le relevé dira ce qu'il en reste. Deux réserves l'accompagnent : le contexte le plus court des trois, qui imposera les fragments les plus fins (section 5.1, Tableau 5.2), et des conditions d'utilisation propres à Google, avec une acceptation préalable au téléchargement (Google 2025), ce que le critère 6 retiendra.

Tableau 5.2 : Fiche d'identité des trois candidats

Candidat	Éditeur	Sortie	Licence	Paramètres	Contexte	Vecteurs	Distribution Ollama
BGE-M3	BAAI	02/2024	MIT	568 millions	8192	1 024	Bibliothèque officielle (1,2 Go)
Qwen3-Embedding-0.6B	Alibaba (équipe Qwen)	06/2025	Apache 2.0	596 millions	32768	1 024, réglables	Bibliothèque officielle (639 Mo)
EmbeddingGemma	Google DeepMind	09/2025	Conditions Gemma (acceptation préalable)	308 millions	2048	768, réglables jusqu'à 128	Bibliothèque officielle (622 Mo)

(élaboration personnelle, d'après les sources citées dans la section 5.3)

Restent les pistes croisées sans être retenues. Les services d'embeddings en nuage, de l'API d'OpenAI à Gemini Embedding, n'ont jamais concouru : le périmètre les écarte avant tout critère, comme au chapitre 4. Les embedders dérivés de grands modèles de langage de plusieurs milliards de paramètres, tels E5-Mistral ou GritLM, vers lesquels la littérature récente s'est largement tournée (Vera et al. 2025), sortent de l'enveloppe de la section 4.1. Les générations antérieures de modèles français, CamemBERT et FlauBERT en tête, produisaient leurs vecteurs par simple moyenne interne, sans l'entraînement pour la similarité que le volet français identifie comme le facteur décisif (Ciancone et al. 2024) ; leurs dérivés entraînés, Solon ou sentence-camembert, restent

honorables mais sans paquet dans la bibliothèque officielle d'Ollama, ce qui rouvrirait la pile d'exécution du chapitre 3. Dernier cas : multilingual-e5-large-instruct, premier du banc multilingue à la parution de son article de référence (Enevoldsen et al. 2025). Au 20 juillet 2026, la bibliothèque officielle d'Ollama n'en distribue aucune version, pour aucune taille de la famille ; seules circulent des conversions communautaires. Le chapitre 4 a montré avec Apertus qu'une conversion communautaire n'est pas rédhibitoire, mais elle demande un titre qui justifie de s'en contenter ; ici, rien de tel, et un contexte de 512 jetons, le plus court de tous les modèles cités dans ce chapitre, achève de régler le cas. Les sept questions peuvent maintenant être posées aux trois candidats : c'est l'objet de la section suivante.

5.4 La comparaison et la décision

Le Tableau 5.3 confronte les trois candidats aux sept critères ; chaque case résume ce que la section 5.3 a établi, les scores des deux premiers critères provenant du relevé du 20 juillet 2026. La lecture est plus simple qu'au chapitre 4 : un seul palier est concerné, l'enveloppe de la section 4.1 absorbant les trois paquets, et aucun critère n'élimine personne. La comparaison se joue donc sur les différences.

Tableau 5.3 : La confrontation des trois candidats aux sept critères

Critère	BGE-M3	Qwen3-Embedding-0.6B	EmbeddingGemma
1. Récupération multilingue (relevé du 20.07.2026)	54,59 ; 49e rang	64,65 ; 18e rang	62,49 ; 29e
2. Français (volet, même relevé)	Récupération non renseignée ; classification de paires 59,18 ; premier ouvert du classement 2024 (article)	Récupération non renseignée ; classification de paires 67,52	Récupération non renseignée ; classification de paires 61,99
3. Empreinte mémoire	1,2 Go, haut de l'enveloppe	639 Mo, à l'aise	622 Mo, le plus léger
4. Dimension des vecteurs	1 024, fixes	1 024, réglables	768, réglables jusqu'à 128
5. Longueur de contexte	8 192	32 768, le plus long	2 048, le plus court
6. Licence et accès	MIT, libre	Apache 2.0, libre	Conditions Gemma, acceptation préalable
7. Maturité et disponibilité	Article relu par les pairs ; paquet le plus téléchargé	Source primaire dédiée ; famille active	Le plus récent ; éditeur du chapitre 4 ; quantification entraînée

(élaboration personnelle, d'après les portraits de la section 5.3 et le relevé MTEB du 20 juillet 2026)

Les critères matériels et juridiques dessinent un ordre sans éliminer personne. L'empreinte départage peu, les trois paquets tenant dans l'enveloppe, BGE-M3 en occupant le haut. Le contexte oppose les extrêmes : 32 768 jetons pour Qwen3-Embedding, 2 048 pour EmbeddingGemma, qui imposerait les fragments les plus courts, 8 192 pour BGE-M3, largement assez pour des fragments de pages administratives. Les vecteurs réglables existent chez deux candidats sur trois, la dimension la plus courte chez EmbeddingGemma. La licence, enfin, sépare les deux candidats libres de toute barrière du troisième, soumis à une acceptation préalable, un point qui compte pour un service public appelé à être redéployé (section 4.2).

Restent les deux critères de qualité, ceux du relevé. Les rapports d'éditeurs donnent le point de départ, et il est rare qu'ils convergent comme ici : le tableau comparatif de Google et celui d'Alibaba, publiés à trois mois d'écart, créditent les mêmes valeurs aux mêmes modèles, Qwen3-Embedding-0.6B nettement en tête de la récupération multilingue, EmbeddingGemma derrière lui, BGE-M3 à distance (Zhang et al. 2025; Vera et al. 2025). Deux sources concurrentes qui tombent d'accord valent mieux qu'une seule ; aucune, pourtant, n'isole le français. Le relevé du 20 juillet 2026 tranche le premier critère et documente le second par une absence. Sur le banc multilingue, l'ordre des rapports se confirme à l'identique : Qwen3-Embedding-0.6B en tête des trois en récupération, EmbeddingGemma ensuite, BGE-M3 à distance, aux 18e, 29e et 49e rangs généraux (MTEB 2026a). Sur le volet français, les trois candidats figurent au classement, mais aucun n'y est mesuré en récupération : seule une épreuve de classification de paires est renseignée pour chacun, dans le même ordre que le multilingue (MTEB 2026b). Une absence est aussi une donnée : le critère du français ne peut pas se lire sur l'instrument public. La décision se prend donc sur ce qui est mesuré, et c'est l'évaluation de la Partie 3, menée sur le système complet une fois construit avec le modèle retenu, qui vérifiera la qualité en français sur les questions genevoises.

Concrètement, pour l'usager qui demande où déposer sa demande de permis de séjour, ces deux critères décident si la bonne page de ge.ch figure parmi les fragments retrouvés ; les cinq autres règlent le coût de cette recherche, pas sa justesse. La pesée en découle : à empreintes toutes admissibles et licences connues, la qualité de récupération commande ; le volet français existe mais ne mesure aucun des trois candidats en récupération au jour du relevé ; elle se lit donc sur le multilingue.

Ce travail retient donc **Qwen3-Embedding-0.6B**. Trois raisons, par ordre d'importance. D'abord la qualité mesurée : seul candidat que deux comparatifs concurrents placent en tête de la récupération multilingue, et que le relevé indépendant du 20 juillet 2026

confirme premier des trois (MTEB 2026a). Ensuite le dossier : une source primaire dédiée, une licence Apache 2.0 sans barrière, un paquet officiel léger, et les deux mécanismes utiles aux sous-décisions de 5.1, vecteurs réglables et consignes de tâche, documentés par l'éditeur. Enfin la marge : le contexte le plus long du trio laisse le découpage des pages libre, quand le plus léger des candidats l'aurait contraint.

Ce choix a ses inconvénients, qu'il faut nommer. Le français jamais mesuré en récupération, d'abord, ni par l'éditeur ni, au jour du relevé, par le volet français du classement : la seule colonne française disponible conserve l'ordre du multilingue, mais c'est l'évaluation de la Partie 3, sur le système complet bâti avec le modèle retenu, qui vérifiera le français en conditions réelles, là où aucun banc public ne va. La jeunesse relative de l'écosystème, ensuite : BGE-M3 reste le paquet le plus éprouvé des trois, et c'est lui que ce travail désigne comme solution de repli, français attesté par le volet 2024 et article relu par les pairs, si les mesures de la partie pratique contredisaient le choix ; l'interchangeabilité garantie par le framework au chapitre 3 rendrait le remplacement simple. Un embedder issu d'un modèle de langage, enfin, se paiera peut-être en vitesse d'indexation face à un encodeur classique : la partie pratique le mesurera avant toute mise en service. Le deuxième composant est choisi. Le troisième est la base vectorielle, celle qui rangera tous ces vecteurs et saura les retrouver vite : c'est l'objet du chapitre suivant.

6. Choix de la base vectorielle

Troisième composant : la base vectorielle, celle qui range les vecteurs fabriqués par le modèle d'embeddings et retrouve les plus proches à chaque question. Son rôle est déjà dessiné sur la Figure 5.1 : recevoir les vecteurs des fragments à l'indexation, comparer celui de la question à chaque requête, rendre les fragments retrouvés. Le chapitre qui s'ouvre a pourtant une particularité : c'est le premier où la performance ne départage pas les candidats. Le corpus de Ge-Trouve se comptera en milliers de fragments, pas en milliards, et à cette échelle toutes les bases répondent vite ; la section 6.1 le montrera, sources à l'appui. Le choix se jouera donc ailleurs : sur l'intégration au framework retenu au chapitre 3, sur la simplicité de service et de sauvegarde sur le serveur, et sur le filtrage des fragments par leurs métadonnées. Le cadre, lui, ne bouge pas : exécution locale, et les services en nuage hors périmètre par construction.

La démarche suit le chemin des chapitres précédents. La section 6.1 précise ce que fait la base et ce que l'échelle du projet change aux termes du choix. La section 6.2 en tire une grille de critères, la section 6.3 présente les trois candidats retenus et les pistes écartées, et la section 6.4 les confronte pour arrêter le choix.

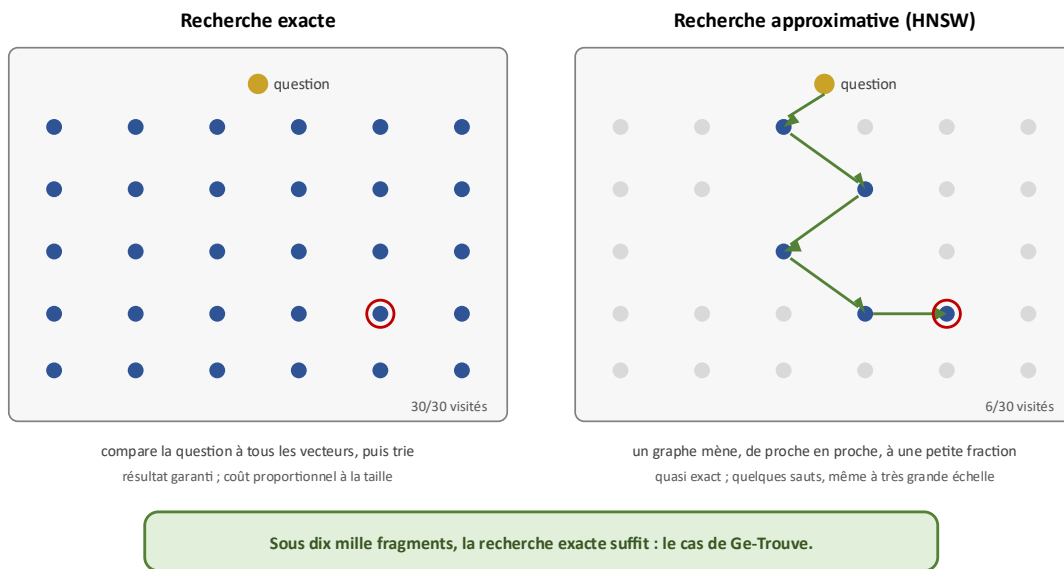
6.1 Le rôle de la base vectorielle et les contraintes du projet

La base vectorielle fait trois choses. À l'indexation, elle reçoit les vecteurs des fragments et les range avec leurs métadonnées : l'adresse de la page d'origine, sa section, sa date. À chaque question, elle compare le vecteur de la question à tous ceux qu'elle détient et rend les fragments les plus proches, ceux que la Figure 5.1 montrait seuls à atteindre le modèle de langage. Entre les deux, elle persiste : les vecteurs survivent aux redémarrages du serveur, sans réindexation. S'y ajoute une capacité que le pipeline exploitera : le filtrage, c'est-à-dire restreindre la recherche par les métadonnées, par exemple aux pages encore en vigueur de l'office concerné quand un usager demande où déposer sa demande de permis de séjour.

Reste la question du comment. La méthode simple est la recherche exacte : comparer la question à tous les vecteurs, puis trier. Les bases modernes emploient plutôt une recherche approximative : un index, typiquement le graphe hiérarchique HNSW⁵⁸, guide la recherche vers une petite fraction de la base, en acceptant de manquer parfois un voisin (Malkov, Yashunin 2016). La Figure 6.1 met les deux approches côte à côte.

⁵⁸ HNSW (*Hierarchical Navigable Small World*), littéralement « petits mondes navigables hiérarchiques » : un algorithme qui organise les vecteurs en graphe à étages, parcouru du général au précis, pour retrouver des voisins proches sans balayer toute la base.

Figure 6.1 : Recherche exacte et recherche approximative



(élaboration personnelle, d'après Malkov et Yashunin 2016 ; Douze et al. 2024)

Ces index ont été conçus pour un problème d'échelle. Les bases qui les ont rendus nécessaires se comptent en dizaines de millions, parfois en milliards de vecteurs ; l'article fondateur de HNSW situe l'ordre de grandeur en sens inverse, un million de vecteurs se balayant en 94 millisecondes sur un seul fil d'exécution (Malkov, Yashunin 2016). Le corpus de Ge-Trouve sera d'un tout autre ordre : les pages administratives de ge.ch, découpées en fragments, se compteront en milliers, au plus en dizaines de milliers, et une recherche exacte s'y fera en une fraction de milliseconde, sans aucun index.

La référence du domaine tire elle-même la conséquence. Le guide de choix de la bibliothèque Faiss oriente les petites bases vers l'index plat, c'est-à-dire la base rangée telle quelle et balayée en entier, et le corps du texte fixe le seuil : la recherche non exhaustive ne devient la pierre angulaire qu'au-delà d'environ dix mille vecteurs (Douze et al. 2024). Le même manuel règle au passage la question de la mémoire : quand la base brute tient plusieurs fois en mémoire vive, cette contrainte disparaît. C'est notre cas : les vecteurs de 1 024 dimensions retenus au chapitre 5 pèseront quelques dizaines de mégaoctets pour tout le corpus, une fraction du serveur dimensionné en 4.1.

Ce renversement fixe les termes du choix. La vitesse et la justesse de la recherche ne départageront pas des candidats qui, à cette échelle, répondent tous vite et savent tous chercher exactement. Les vraies questions sont d'exploitation : comment la base s'intègre au framework retenu au chapitre 3 ; si elle vit dans l'application ou en serveur

séparé ; comment elle persiste et se sauvegarde ; comment elle filtre par métadonnées ; ce qu'elle ajoute à la pile du serveur. De ces questions, la section suivante tire la grille.

6.2 Les critères de comparaison

La grille de ce chapitre hérite de la méthode des précédents, avec un renversement qui lui est propre : le critère roi des chapitres 4 et 5, la qualité mesurée, n'y figure pas. La section 6.1 en a donné la raison : à l'échelle du corpus genevois, tous les candidats savent chercher exactement et répondre vite ; un critère qui ne départage personne n'a pas sa place dans la grille. La souveraineté n'y figure pas non plus, pour la raison constante depuis le chapitre 4 : elle a agi en amont, en écartant les services en nuage avant tout concours. La grille reste qualitative et se concentre sur une seule question : laquelle des trois bases sera la plus simple à intégrer, à exploiter et à sauvegarder sur le serveur du projet.

Premier critère, l'intégration Haystack. Le chapitre 3 a fait du framework la pièce centrale du pipeline ; une base sans intégration officielle obligerait à du code sur mesure, fragile et hors périmètre. La grille note l'existence de l'intégration, ce qu'elle couvre (magasin de documents, *retrievers*⁵⁹), qui la maintient et depuis quand.

Deuxième critère, le mode de service. Une base peut vivre dans l'application, comme une bibliothèque chargée au démarrage, ou en serveur séparé, avec son processus, son port et sa mémoire propre. Sur un VPS administré par une seule personne, chaque pièce de plus est une pièce à surveiller : le critère pèse la simplicité contre les services rendus.

Troisième critère, la persistance et la sauvegarde. Les vecteurs doivent survivre aux redémarrages, et l'exploitant doit pouvoir répondre à une question simple : que copie-t-on pour sauvegarder la base, et comment la restaure-t-on ? Un fichier ou un dossier à copier vaut mieux, ici, qu'une procédure d'export.

Quatrième critère, le filtrage par métadonnées. La section 6.1 l'a montré sur la question type : restreindre la recherche par la source, la date ou la section des pages est une capacité de premier plan pour un corpus administratif. La grille note si le filtrage est natif et ce qu'il sait exprimer.

⁵⁹ *Retriever*, littéralement « récupérateur » : dans Haystack, le composant qui interroge le magasin de documents et rend les fragments les plus proches de la question ; chaque base vectorielle fournit le sien.

Cinquième critère, l'empreinte d'exploitation. Mémoire consommée, dépendances installées, pièces ajoutées à la pile du chapitre 3 : le poste doit rester marginal à côté du modèle de langage, comme l'était celui des embeddings (section 4.1).

Sixième critère, la licence, reprise des chapitres précédents. Les trois candidats annoncent des licences permissives : le critère ne devrait donc départager personne, mais il se vérifie quand même, sur les pièces de la section suivante, et cette vérification est en soi une information.

Septième critère, la maturité et l'écosystème : l'ancienneté du projet, la vitalité de son développement, la qualité de sa documentation, selon le patron des chapitres 3 à 5. Le Tableau 6.1 rassemble la grille, au même format que les précédents.

Tableau 6.1 : Les sept critères de comparaison de la base vectorielle

Critère	Question	Origine
1. Intégration Haystack	Une intégration officielle existe-t-elle, qui la maintient, depuis quand, que couvre-t-elle ?	Framework retenu (chapitre 3)
2. Mode de service	La base vit-elle dans l'application ou en serveur séparé, à quel coût d'exploitation ?	Serveur administré seul (section 4.1)
3. Persistance et sauvegarde	Que copie-t-on pour sauvegarder, comment restaure-t-on ?	Service public durable (section 4.2)
4. Filtrage par métadonnées	Peut-on restreindre la recherche par source, date ou section, nativement ?	Corpus administratif ; question type (section 6.1)
5. Empreinte d'exploitation	Que pèse la base en mémoire et en pièces ajoutées à la pile ?	Budget serveur (section 4.1)
6. Licence	La licence est-elle permissive, vérifiée sur pièces ?	Service public (section 4.2)
7. Maturité et écosystème	Projet ancien, vivant, documenté ?	Patron des chapitres 3 à 5

(élaboration personnelle)

Les questions sont posées ; la section suivante présente les trois candidats, en commençant par la convention désormais habituelle : une date de référence unique.

6.3 Les candidats

La liste n'a pas demandé de veille nouvelle : les trois candidats étaient pressentis dès la proposition de sujet, et la vérification de juillet 2026 a confirmé que chacun dispose d'une intégration Haystack officielle, condition d'entrée posée par le critère 1. Les pistes écartées ferment la section. Même convention qu'aux chapitres précédents : le 1er août 2026 sert de date de référence unique, dépôts, documentations et pages d'intégration

décrivant l'état constaté ce jour-là. Les candidats se présentent du plus ancien au plus récent, à l'ancienneté de leurs dépôts.

Le plus ancien est une bibliothèque, pas une base, et l'assume. Faiss, développé par Meta et publié en source ouverte en 2017 (Douze et al. 2024), sous licence MIT (Meta AI 2026), est la référence algorithmique du domaine : son article décrit les familles d'index et le guide de choix mobilisés en 6.1, en traçant d'emblée le périmètre, ni écritures concurrentes, ni transactions, ni répartition (Douze et al. 2024). Le candidat, c'est donc Faiss habillé par son intégration Haystack : un magasin de documents qui range l'index dans un fichier et les documents avec leurs métadonnées dans un second, sans aucun serveur, positionné par deepset pour les jeux de petite et moyenne taille où la simplicité prime sur le passage à l'échelle (deepset 2026a). La sobriété est maximale ; les services le sont aussi peu, tout ce qu'une base offre d'ordinaire restant à la charge de la couche du dessus.

Le deuxième est un serveur de métier. Qdrant, développé depuis 2020 en Rust sous licence Apache 2.0 (Qdrant 2026a), est une base vectorielle complète : un service dédié avec son interface, sa persistance gérée et un filtrage par métadonnées, les vecteurs y étant stockés avec une charge utile prévue pour cela (Qdrant 2026b). Son intégration Haystack est officielle : cosignée par deepset et par Qdrant (deepset 2026b) ; elle expose jusqu'aux réglages du graphe HNSW rencontré en 6.1 et offre un mode local pour les essais, le régime nominal restant le serveur séparé, avec ce que cela ajoute à la pile : un processus, un port et des mises à jour de plus à administrer.

Le plus récent est pensé pour vivre dans l'application. Chroma, développé depuis 2022, écrit en Rust pour son cœur et publié sous licence Apache 2.0 (Chroma 2026a), se présente comme une base embarquée : elle stocke vecteurs, documents et métadonnées, et se déploie en local comme en serveur, au choix (Chroma 2026b). Le filtrage par métadonnées est natif, appliqué au moment de la recherche (Chroma 2026b), et l'intégration Haystack couvre le magasin et son retrieveur (deepset 2026c). C'est le candidat pressenti depuis la proposition de sujet ; le pressentiment ne vaut pas décision, et la section suivante le soumettra à la grille comme les deux autres. Sa jeunesse est son revers : le plus court historique du trio.

Tableau 6.2 : Fiche d'identité des trois candidats

Candidat	Éditeur	Dépôt créé ⁶⁰	Licence	Langage	Mode de service	Persistance	Intégration Haystack
Faiss	Meta	2017	MIT	C++	Bibliothèque dans l'application	Fichier d'index et fichier de métadonnées	Officielle (magasin, retrieveur)
Qdrant	Qdrant	2020	Apache 2.0	Rust	Serveur dédié (mode local d'essai)	Stockage géré par le serveur	Officielle, cosignée par l'éditeur (réglages HNSW exposés)
Chroma	Chroma	2022	Apache 2.0	Rust	Embarquée (mode serveur optionnel)	Stockage local intégré	Officielle (magasin, retrieveur)

(élaboration personnelle, d'après les sources citées dans la section 6.3)

Restent les pistes écartées, chacune pour un motif. Les services vectoriels en nuage, Pinecone en tête, n'ont jamais concouru : hors périmètre par construction. Milvus et Weaviate sont des serveurs distribués, taillés pour des volumes et des équipes que le projet n'aura pas ; les adopter ajouterait de l'administration sans rien apporter à notre échelle. pgvector aurait un attrait, l'appui d'un moteur éprouvé, mais installerait un serveur PostgreSQL entier pour un besoin vectoriel simple, à rebours du critère 2. Elasticsearch et ses parents, enfin, sont des moteurs de recherche complets dont le pipeline n'exploiterait qu'une fraction : la pile s'alourdirait pour rien. Aucun de ces écartements ne tient à la qualité de recherche, qui n'aurait départagé personne. Les sept questions peuvent maintenant être posées aux trois candidats : c'est l'objet de la section suivante.

6.4 La comparaison et la décision

Le Tableau 6.3 confronte les trois candidats aux sept critères ; chaque case résume ce que la section 6.3 a établi, à l'état du 1er août 2026. La lecture est la plus simple des trois chapitres de choix : aucun critère n'élimine personne, les trois intégrations sont officielles et les trois licences permissives, vérifiées sur pièces. Tout se joue sur les différences d'exploitation.

Tableau 6.3 : La confrontation des trois candidats aux sept critères

Critère	Faiss	Qdrant	Chroma
1. Intégration Haystack	Officielle (magasin, retrieveur)	Officielle, cosignée par l'éditeur, réglages exposés	Officielle (magasin, retrieveur)
2. Mode de service	Bibliothèque, zéro serveur	Serveur dédié, mode local d'essai	Embarquée, serveur optionnel
3. Persistance et sauvegarde	Deux fichiers sur disque	Stockage géré par le serveur	Stockage local intégré

⁶⁰ Années de création des dépôts : métadonnées GitHub relevées le 1er août 2026.

4. Filtrage par métadonnées	À la charge de la couche du dessus	Natif, par la charge utile des vecteurs	Natif, au moment de la recherche
5. Empreinte d'exploitation	Une bibliothèque installée avec l'application, aucun service séparé	Un serveur à installer et maintenir en plus	Une bibliothèque installée avec l'application, stockage local intégré
6. Licence	MIT	Apache 2.0	Apache 2.0
7. Maturité et écosystème	2017, référence du domaine	2020, serveur établi	2022, le plus jeune

(élaboration personnelle, d'après la section 6.3)

Les modes de service dessinent trois philosophies. Faiss ne fournit rien autour du calcul : une bibliothèque dans l'application, rien à administrer, et rien de fourni au-delà du calcul. Qdrant fournit tout, au prix d'une pièce d'exploitation entière ajoutée au serveur. Chroma se tient entre les deux : les services d'une base, dans le processus de l'application. Le critère qui sépare vraiment est le filtrage : natif chez Chroma et Qdrant, à la charge de la couche du dessus chez Faiss. Pour un corpus administratif où restreindre la recherche par source et par date est une capacité de premier plan (sections 6.1 et 6.2), ce point coûte cher à Faiss. Les consommations réelles, elles, se relèveront sur le serveur en Partie 2, comme pour les modèles.

Concrètement, pour l'usager qui demande où déposer sa demande de permis de séjour, aucun des trois ne changera la vitesse ni la justesse de la réponse. Ce qui change est invisible et quotidien : le nombre de pièces que l'exploitant surveille, la facilité d'une sauvegarde, la possibilité de ne chercher que dans les pages en vigueur. La pesée en découle : à qualité égale par construction, la base la plus simple à vivre l'emporte, pourvu qu'elle filtre nativement.

Ce travail retient donc **Chroma**. Trois raisons, par ordre d'importance. D'abord le mode de service : la base embarquée supprime une pièce d'exploitation entière, pas de serveur à lancer, à surveiller ni à mettre à jour. Ensuite le filtrage natif : sources, dates et sections des pages de ge.ch se filtreront au moment de la recherche, sans code sur mesure. Enfin l'éventail de recherche annoncé par sa documentation, au-delà du vecteur dense : recherche lexicale et plein texte intégrées, une réserve d'évolution documentée (Chroma 2026b). Le pressentiment de la proposition de sujet se trouve ainsi confirmé, mais par la grille, pas par lui-même.

Ce choix a ses inconvénients, qu'il faut nommer. La jeunesse d'abord : le plus court historique du trio, une maturité qui se paie en recul. Le plafond ensuite : si le corpus, la charge ou les besoins de filtrage dépassaient un jour le cadre prévu, la réponse ne serait pas de tordre Chroma mais de passer au serveur de métier. Qdrant est la solution de

repli documentée, couvrant exactement le scénario où il faudrait plus, pas moins ; l'interchangeabilité des magasins garantie par le framework (chapitre 3) rendrait la bascule simple. Faiss, battu sur le filtrage, reste au projet par son article, qui a fourni à ce chapitre son argument central. Le troisième composant est choisi, et la pile est au complet : un framework pour coordonner, un modèle de langage pour rédiger, un modèle d'embeddings pour comprendre, une base pour retrouver. Reste la question qui occupera le chapitre 7 : comment savoir si l'ensemble répond juste.

7. Évaluation d'un système RAG

La pile est complète ; reste à savoir si elle répond juste. Les chapitres 3 à 6 ont choisi les composants, celui-ci construit l'instrument de mesure : comment vérifier, preuves à l'appui, que l'assistant répond correctement aux questions administratives des Genevois. La question n'est pas accessoire. Un système RAG peut échouer de plusieurs manières qui ne se voient pas à l'œil nu, et un développement mené sans instrument de mesure ne peut pas dire s'il progresse. C'est pourquoi la méthode d'évaluation se fixe maintenant, avant la première ligne de code de la partie pratique : elle guidera le développement, puis jugera le résultat.

Le cadre est acté depuis la proposition de ce travail : RAGAS⁶¹, un cadre d'évaluation conçu pour les systèmes RAG. Le chapitre ne compare donc pas des candidats ; il explique. La section 7.1 établit ce qu'il faut évaluer dans un système RAG et pourquoi une note globale ne suffit pas. La section 7.2 présente RAGAS et ses métriques, en langage courant. La section 7.3 expose la mécanique du modèle juge et ses limites. La section 7.4 en tire le protocole prévu pour Ge-Trouve, jeu de questions genevoises compris, que la Partie 3 exécutera.

7.1 Ce qu'il faut évaluer dans un système RAG

Reprenons la question type. « Où déposer ma demande de permis de séjour ? » traverse tout le pipeline : la question devient un vecteur, la base rend les fragments les plus proches, le modèle de langage rédige avec eux. Si la réponse est mauvaise, où est la faute ? Peut-être à l'étage de la récupération : le bon fragment n'est jamais remonté, et le rédacteur travaillait sans la bonne matière. Peut-être à l'étage de la génération : les fragments étaient bons, mais le modèle a brodé au-delà, l'hallucination décrite au chapitre 2. Vues de l'utilisateur, les deux pannes produisent la même chose, une réponse fautive ; elles appellent pourtant des remèdes opposés. Évaluer un RAG, c'est donc d'abord séparer les étages.

La littérature de terrain confirme et affine. En observant trois systèmes RAG en service, dans trois domaines distincts, Barnett et al. (2024) dressent une carte de sept points de défaillance, du contenu simplement absent du corpus, qui condamne la réponse avant toute technique, à la réponse incomplète, juste mais partielle ; le Tableau 7.1 la traduit.

⁶¹ RAGAS (*Retrieval Augmented Generation Assessment*), littéralement « évaluation de la génération augmentée par récupération » : un cadre d'évaluation proposé en 2023 par Es et ses co-auteurs, devenu depuis une bibliothèque ouverte du même nom.

Tableau 7.1 : Les sept points de défaillance d'un système RAG

Point de défaillance	Ce qui se passe	Étage concerné
1. Contenu absent <i>(missing content)</i>	La réponse n'existe pas dans les documents disponibles ; au mieux, le système devrait dire qu'il ne sait pas.	Corpus
2. Bons documents mal classés <i>(missed the top ranked documents)</i>	La réponse est dans un document, mais il n'est pas remonté assez haut pour être transmis.	Récupération
3. Perdus à l'assemblage <i>(not in context)</i>	Des documents contenant la réponse ont été retrouvés, mais pas retenus dans le contexte transmis au modèle.	Récupération
4. Non extraite <i>(not extracted)</i>	La réponse est dans le contexte, le modèle ne parvient pas à l'en tirer.	Génération
5. Mauvais format <i>(wrong format)</i>	Un format demandé, tableau ou liste, est ignoré par le modèle.	Génération
6. Spécificité inadaptée <i>(incorrect specificity)</i>	La réponse est trop générale ou trop précise pour le besoin de l'utilisateur.	Génération
7. Incomplète <i>(incomplete)</i>	Juste mais partielle, alors que l'information manquante était disponible.	Génération

(élaboration personnelle, d'après (Barnett et al. 2024))

Un point frappe en amont, dans le corpus lui-même, deux frappent la récupération, quatre la génération : la carte recouvre la séparation des étages posée plus haut. Cette carte servira deux fois : en Partie 2 comme liste de contrôle à la conception, en Partie 3 comme grille de lecture des erreurs constatées.

Restait à outiller cette évaluation par étage. Les créateurs de RAGAS posent le problème en plusieurs dimensions : la capacité du système de récupération à trouver des passages pertinents et ciblés, la capacité du modèle de langage à exploiter ces passages fidèlement, et la qualité de la rédaction elle-même ; leur apport tient dans la promesse de mesurer ces dimensions sans dépendre d'annotations humaines de référence (Es et al. 2023). Pour Ge-Trouve, l'argument est concret : chaque réglage de la partie pratique, taille des fragments, nombre de fragments transmis, consigne donnée au rédacteur, pourra être rejugé au même instrument, sans mobiliser des annotateurs à chaque tour. Ces dimensions demandent des métriques qui regardent chacune un endroit précis du pipeline : c'est l'objet de la section suivante.

7.2 Le cadre retenu : RAGAS et ses métriques

Le cadre d'évaluation est acté depuis la proposition de ce travail, et trois faits suffisent à l'asseoir. RAGAS est né pour le problème posé en 7.1 : ses créateurs proposent une suite de métriques couvrant les dimensions d'un RAG, sans dépendre d'annotations humaines de référence (Es et al. 2023). L'outil est devenu une bibliothèque vivante :

version 0.4.3, licence Apache 2.0, un million de téléchargements par mois (Ragas 2026a). Et il a grandi depuis l'article : la liste actuelle des métriques dépasse largement le RAG, couvrant jusqu'aux agents et à la comparaison de textes (Ragas 2026b). Le mémoire cite donc l'article pour le cadre et la documentation datée pour l'état courant. D'autres outils d'évaluation existent ; le choix de RAGAS, posé dès la proposition, tient à cette adéquation et à cette adoption, et le chapitre n'ouvre pas de comparatif.

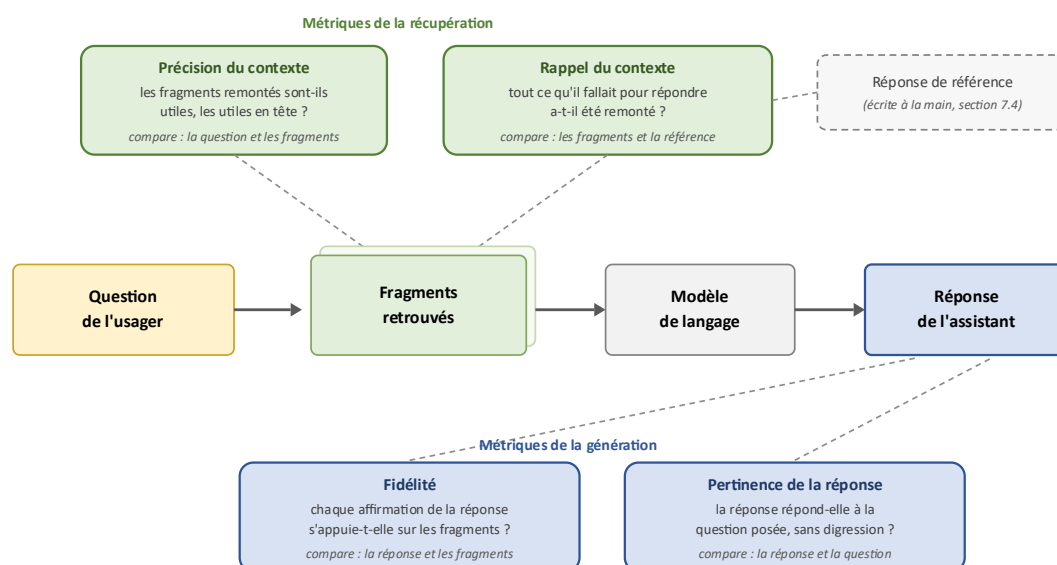
De cette liste, le protocole de Ge-Trouve retiendra les quatre métriques de la famille RAG qui répondent directement aux deux étages de la section 7.1 (Ragas 2026b). Deux regardent la récupération. La précision du contexte demande si les fragments remontés sont utiles, et si les utiles arrivent en tête : cinq fragments rendus dont un seul parle du permis de séjour, et la note chute. Le rappel du contexte demande l'inverse : tout ce qu'il fallait pour répondre a-t-il été remonté ? Si la page listant les documents à joindre manque à l'appel, la réponse sera incomplète quoi que fasse le rédacteur. Deux regardent la génération. La fidélité demande si chaque affirmation de la réponse s'appuie sur les fragments fournis : c'est le thermomètre de l'hallucination du chapitre 2. La pertinence de la réponse demande enfin si la réponse répond bien à la question posée, sans digression. Le Tableau 7.2 les rassemble et la Figure 7.1 les situe sur le pipeline ; leurs définitions opérationnelles précises, consignes du juge comprises, seront reprises de la documentation au moment de l'implémentation, en Partie 2.

Tableau 7.2 : Les quatre métriques retenues pour l'évaluation de Ge-Trouve

Métrique	La question qu'elle pose	Étage jugé
Précision du contexte (<i>context precision</i>)	Les fragments remontés sont-ils utiles, et les utiles en tête ?	Récupération
Rappel du contexte (<i>context recall</i>)	Tout ce qu'il fallait pour répondre a-t-il été remonté ?	Récupération
Fidélité (<i>faithfulness</i>)	Chaque affirmation de la réponse s'appuie-t-elle sur les fragments ?	Génération
Pertinence de la réponse (<i>response relevancy</i>)	La réponse répond-elle à la question posée, sans digression ?	Génération

(élaboration personnelle, d'après les dimensions de (Es et al. 2023) et la liste des métriques de la bibliothèque (Ragas 2026b))

Figure 7.1 : Où chaque métrique regarde sur le pipeline



Deux métriques jugent la récupération, deux jugent la génération ; chacune compare deux pièces précises du pipeline.

(élaboration personnelle, d'après (Es et al. 2023) ; (Ragas 2026b))

Une particularité traverse ces métriques et prépare la section suivante. La documentation l'annonce elle-même : les métriques fondées sur un modèle de langage peuvent nécessiter un ou plusieurs appels à un modèle pour établir leur note (Ragas 2026b). Autrement dit, c'est un modèle qui note un modèle. Cette mécanique, ses réglages et ses limites occupent la section suivante.

7.3 L'évaluation par modèle juge et ses limites

La section 7.2 l'a annoncé : les métriques retenues s'établissent par un ou plusieurs appels à un modèle de langage. Il faut le dire simplement : c'est un modèle qui note un modèle. Pour juger la fidélité d'une réponse sur le permis de séjour, un modèle juge reçoit la réponse et les fragments, et rend son verdict ; aucun humain ne relit chaque copie. C'est ce qui rend l'évaluation répétable et bon marché, et c'est aussi ce qui oblige à choisir le juge avec soin.

La bibliothèque laisse ce choix ouvert. Les deux modèles de l'évaluation, le juge et l'encodeur qui l'assiste, se configurent selon la disponibilité de chacun : des fournisseurs directs, OpenAI, Anthropic et Google, ou une passerelle générique vers plus de cent autres (Ragas 2026c). Deux voies s'offrent donc à Ge-Trouve. Le juge externe, d'abord : un grand modèle commercial, vraisemblablement meilleur juge, mais un compte, une clé, un coût, et des données de test qui sortent de l'infrastructure, à contre-courant de la ligne du projet même pour des questions fabriquées sur des pages publiques. Le juge local, ensuite : un modèle servi sur les machines du projet, branché en local. C'est cette

seconde voie que ce travail retient, avec une précaution : le juge ne sera pas le modèle qui rédige. Ce sera Llama 3.1 8B, le repli documenté du chapitre 4, d'une autre famille que le rédacteur, servi sur la machine de développement, dont la mémoire dépasse celle du serveur, sans rien installer ni consommer sur ce dernier. Aucune pièce nouvelle, aucun compte, aucune clé.

Ce choix a trois limites, à écrire avant de mesurer. La taille du juge, d'abord : un modèle de huit milliards de paramètres notera des copies qu'un grand modèle noterait mieux ; les notes vaudront comme instrument de comparaison entre réglages, plus que comme vérité absolue. Le juge, ensuite, reste un modèle : même d'une autre famille que le rédacteur, ce qui écarte par construction le biais d'un modèle notant ses propres copies, il peut se tromper ; le protocole y répondra par un contrôle humain sur un échantillon de notes. Le câblage, enfin : la documentation citée illustre la passerelle par des services en nuage, pas par un serveur local ; le branchement précis du juge local sera donc établi et documenté en Partie 2, et cette vérification fait partie du protocole. La section suivante assemble le tout : le jeu de questions genevoises, le déroulé des mesures et la place du contrôle humain.

7.4 Le protocole d'évaluation prévu pour Ge-Trouve

Le protocole assemble les pièces des trois sections précédentes, et commence par sa matière première : le jeu de questions. La bibliothèque saurait le générer automatiquement, elle propose une génération de jeux de test alignés sur l'usage réel (Ragas 2026a) ; ce travail fait un autre choix, et c'est son apport propre : des questions genevoises écrites à la main. Le jeu visera une vingtaine de questions couvrant les grandes familles de démarches du corpus, posées dans plusieurs registres, de la langue courante à la langue administrative, et comprendra des questions volontairement sans réponse dans le corpus, le premier point de défaillance de la section 7.1, pour observer si l'assistant sait dire qu'il ne sait pas. Une précision d'honnêteté sur les annotations : la promesse « sans annotations humaines » de la section 7.2 vaut pour la notation courante ; le rappel du contexte, lui, se mesure contre une réponse de référence. Chaque question s'accompagnera donc de sa réponse de référence et de la page qui la fonde, écrites une fois, réutilisées à chaque campagne.

Le déroulé suit. Le corpus indexé, chaque question sera posée au système complet, dans la configuration unique arrêtée aux chapitres 3 à 6 ; pour chacune, le protocole conservera la question, les fragments remontés et la réponse produite. La notation viendra ensuite, sur la machine de développement, par le juge de la section 7.3, métrique par métrique. Les résultats se liront en tableaux, par métrique et par famille de questions,

et s'interpréteront en comparant : entre familles de questions d'abord, entre réglages ensuite si la partie pratique en fait varier, taille des fragments ou nombre de fragments transmis. Pas de seuil absolu : la section 7.3 a posé que les notes valent comme instrument de comparaison, et c'est ainsi qu'elles serviront.

Deux garde-fous ferment le protocole. Le contrôle humain, d'abord : un échantillon d'au moins un quart des réponses, tiré au hasard, sera relu à la main et confronté aux notes du juge ; si les deux lectures divergent trop, le juge sera remplacé par un modèle local plus grand, la machine de développement le permettant. L'analyse des erreurs, ensuite : chaque mauvaise note sera classée sur la carte des sept points de défaillance (Tableau 7.1), pour dire non seulement combien le système se trompe, mais où et pourquoi ; ce classement nourrira les limites et la conclusion du mémoire. Le système a désormais son protocole : ce qu'il produira pourra être jugé. Reste un dernier chapitre avant la pratique : le terrain lui-même, le français et ses particularités, objet du chapitre 8.

8. RAG en contexte francophone

La pile a ses composants et son protocole d'évaluation ; reste le terrain où elle servira. Il est genevois, et la langue de travail est le français. Le français a traversé tous les chapitres de choix comme un critère parmi d'autres : performance multilingue du modèle de langage, volet français des bancs d'essai d'embeddings, questions genevoises du protocole d'évaluation. Ce dernier chapitre de la Partie 1 rassemble ce fil dispersé et le confronte au terrain réel : la langue des démarches administratives du canton de Genève, celle que Ge-Trouve devra comprendre chez l'utilisateur et restituer depuis les pages officielles.

La section 8.1 caractérise le français administratif et l'écart de registre qui sépare la langue des usagers de celle des pages officielles. La section 8.2 reprend le pipeline étage par étage et rassemble ce que le français change à chacun, le Tableau 8.1 en récapitulatif. La section 8.3 situe le projet dans le paysage francophone et suisse de l'intelligence artificielle publique, et nomme le vide scientifique que ce travail entend combler. Les deux dernières sections changent de registre : la 8.4 formule le besoin qui a fait naître Ge-Trouve, et la 8.5 examine l'existant pour établir le positionnement du projet.

8.1 Le français administratif, le terrain de Ge-Trouve

Reprenons une dernière fois la question type, et regardons-la de près. Un usager écrit « Où déposer ma demande de permis de séjour ? », ou plus court encore, « c'est où pour le permis ? ». La page officielle qui détient la réponse parle une autre langue : elle traite de l'octroi d'une autorisation de séjour, d'une requête à déposer auprès de l'office compétent, de pièces justificatives à joindre. Même besoin, deux vocabulaires, deux grammaires. Le premier ne connaît pas les termes exacts, le second ne connaît qu'eux. Cet écart de registre n'est pas un détail : c'est le problème central que le pipeline doit absorber, puisque la recherche consiste précisément à apparier la formulation courante d'une question avec la formulation administrative des fragments. C'est le travail confié aux embeddings du chapitre 5, rapprocher des phrases différentes qui disent la même chose, et la raison pour laquelle le protocole du chapitre 7 posera ses questions dans plusieurs registres.

Le terrain genevois ajoute ses particularités. Les sigles d'abord : les démarches vivent dans les offices cantonaux, OCPM⁶² pour la population, AFC⁶³ pour les impôts, et ces sigles sont souvent la clé d'entrée des pages comme des courriers. Le vocabulaire ensuite : la Suisse romande a ses termes administratifs propres, le permis B ou C pour les autorisations de séjour, l'attestation de domicile, qui ne recouvrent pas toujours l'usage de France. L'architecture enfin : une même démarche peut relever du droit fédéral et s'exécuter au guichet cantonal. Le corpus de ce travail reste genevois ; une question qui relève du seul droit fédéral sort du périmètre et appelle un aveu orienté plutôt qu'une réponse improvisée. Retrouver la bonne page tient alors moins à la langue qu'aux métadonnées : la source et la section des pages, précisément ce que le filtrage de la base retenue au chapitre 6 sait exploiter.

Ces particularités dessinent trois exigences, une par étage à venir. Comprendre les deux registres : c'est l'affaire de la recherche, et elle est déjà armée. Répondre dans la langue de l'usager sans perdre les mots qui servent à agir : la réponse doit rester simple, mais conserver le nom exact de l'office, du formulaire ou du permis, car c'est avec ces mots-là que l'usager fera sa démarche ; la consigne donnée au modèle de langage, en Partie 2, devra tenir les deux bouts. Être jugé en français, enfin : le chapitre 7 a préparé l'instrument, la section suivante dira ce que la langue y change. Étage par étage, donc.

8.2 Ce que le français change à chaque étage du pipeline

Premier étage, les embeddings, et le chapitre 5 a déjà rencontré le terrain de front. Les trois candidats revendiquaient le multilingue, mais l'instrument français du domaine, le volet MTEB-French du banc d'essai (Ciancone et al. 2024), ne mesurait aucun des trois en récupération au relevé du 20 juillet 2026 : la décision s'est prise sur le volet multilingue, et la vérification française a été confiée à l'évaluation finale. L'écart de registre de la section 8.1 précise maintenant ce que cette vérification devra contenir : des questions en langue courante face à des fragments en langue administrative, exactement ce que le jeu de questions du chapitre 7 prévoit avec ses registres multiples.

Deuxième étage, le modèle de langage. Le chapitre 4 a traité le français en critère de plein rang, avec l'arène francophone compar:IA comme instrument principal (compar:IA 2026) : les utilisateurs y comparent à l'aveugle les réponses de deux modèles, et le candidat retenu s'y était montré solide. Le terrain ajoute une exigence que l'arène ne mesure pas : conserver les termes officiels, le nom du permis, de l'office ou

⁶² OCPM : Office cantonal de la population et des migrations, l'office genevois compétent notamment pour les autorisations de séjour.

⁶³ AFC : Administration fiscale cantonale, l'administration genevoise des impôts.

du formulaire, tout en restant simple ; ce sera l'affaire de la consigne rédigée en Partie 2, et le protocole la jugera. La Suisse n'est pas absente du paysage des modèles : Apertus, instruit comme candidat en 4.3 (Swiss AI Initiative 2025), a montré qu'un modèle à ambition souveraine existe aussi de ce côté de la frontière.

Troisième étage, la base vectorielle, et c'est le plus court : les vecteurs n'ont pas de langue. Une fois les fragments encodés, la base compare des nombres, et le français n'y change rien. Le terrain y entre par un autre chemin, les métadonnées : la source et la section des pages, que le filtrage retenu au chapitre 6 exploitera. Dernier étage, l'évaluation, et une particularité à écrire noir sur blanc : RAGAS est né en anglais. L'article détaille ses consignes internes en anglais, celles que le juge reçoit pour découper une réponse en affirmations ou en extraire les phrases pertinentes (Es et al. 2023), et rien dans les pièces réunies ne documente leur comportement sur des copies en français. Le dispositif du chapitre 7 encadre précisément cette inconnue : un juge multilingue, et surtout le contrôle humain d'un quart des réponses, qui confrontera les notes du juge à une lecture genevoise. Le Tableau 8.1 rassemble cette traversée.

Tableau 8.1 : Ce que le français change à chaque étage du pipeline

Étage	La question francophone	La réponse déjà en place
Embeddings	Apparier registre courant et registre administratif	Modèle multilingue (chapitre 5) ; vérification française confiée au protocole, le volet français du banc ne mesurant pas la récupération
Modèle de langage	Rédiger un français simple sans perdre les termes officiels	Arène francophone au chapitre 4 ; consigne à écrire en Partie 2
Base vectorielle	Aucune : les vecteurs n'ont pas de langue	Le terrain entre par les métadonnées : filtrage par source et section (chapitre 6)
Évaluation	Juger des copies françaises avec des consignes internes en anglais	Juge local multilingue et contrôle humain d'un quart des réponses (chapitre 7)

(élaboration personnelle)

Le terrain est couvert ; reste à situer le projet dans son paysage, et à nommer ce qu'il apporte.

8.3 Un vide à combler, l'évaluation administrative genevoise

Le paysage francophone de l'intelligence artificielle publique n'est pas vide, et le projet s'y inscrit. La France en donne la mesure la plus visible : sa Commission de l'intelligence artificielle, installée par le gouvernement, a remis en mars 2024 un rapport qui fixe une ambition nationale, vingt-cinq recommandations et un investissement public annuel proposé de cinq milliards d'euros pendant cinq ans (Commission de l'intelligence

artificielle 2024). L'outillage concret suit la même ligne : l'arène compar:IA de la section précédente est un instrument public français, pensé pour évaluer les modèles en français par les francophones eux-mêmes. La Suisse avance sur un autre front, celui des modèles : Apertus, rencontré au chapitre 4, est né des écoles polytechniques fédérales avec une ambition assumée de souveraineté. Des politiques, des instruments, des modèles : le cadre existe.

Ce qui manque, c'est la dernière marche, et la revue conduite dans cette première partie n'a pas permis de l'identifier ailleurs : l'évaluation d'un assistant RAG sur un corpus administratif réel, en français, à l'échelle d'un canton, avec des questions d'utilisateurs et une lecture des erreurs. Les articles fondateurs évaluent sur des jeux académiques anglophones ; les travaux sur les assistants de service public proposent des grilles générales, sans corpus francophone ni système complet ; les instruments francophones mesurent des modèles nus, pas des pipelines branchés sur les pages d'une administration. C'est ce vide, modeste et précis, que ce travail entend combler : la question de recherche posée en introduction y trouvera sa réponse chiffrée en Partie 3, sur les pages de ge.ch, avec le protocole du chapitre 7.

8.4 L'analyse du besoin

L'idée de Ge-Trouve part d'une observation ordinaire, que bien des familles connaissent : trouver une information administrative en ligne reste une épreuve pour qui ne manie pas aisément le web. Les personnes âgées y renoncent souvent les premières, mais elles ne sont pas seules. L'information existe pourtant. Elle est publiée, officielle et gratuite. Mais elle est dispersée sur des milliers de pages, rédigée dans une langue administrative, et il faut déjà savoir où chercher, puis comment formuler sa demande. Notre question type le montre : « Où déposer ma demande de permis de séjour ? » n'a pas de page portant ce nom. La réponse vit dans une fiche de démarche que l'utilisateur doit d'abord localiser. La difficulté ne vient donc pas d'un défaut des sites officiels. Elle vient de l'écart entre leur logique, celle des offices et des prestations, et celle de l'utilisateur, qui part d'une situation vécue.

Le besoin se formule alors simplement. Une porte d'entrée unique, qui accepte la question telle qu'elle vient, en langage courant. Une réponse directe, qui cite ses sources et renvoie vers les pages officielles pour agir. Un service ouvert à toute heure, sans guichet ni attente, et gratuit. Il ne s'agit pas de remplacer ge.ch, encore moins les guichets : il s'agit de rendre l'existant accessible, en particulier à celles et ceux que la recherche en ligne rebute.

Si ce besoin peut trouver une réponse aujourd'hui, c'est que deux conditions sont réunies. La première est technique : les modèles ouverts ont atteint le niveau requis pour ce service, la Partie 1 l'a montré. La seconde tient à la confiance. Pour un service public de fait, les questions des usagers ne doivent ni quitter la Suisse, ni nourrir un service commercial : la souveraineté n'est pas un ornement du projet, elle fait partie du besoin. Reste une vérification avant de construire : personne n'offre-t-il déjà cette réponse ? C'est l'objet de la section suivante.

8.5 L'existant et le positionnement

Que peut faire aujourd'hui un Genevois qui a une question de démarche ? Quatre options s'offrent à lui. La recherche de ge.ch, officielle et à jour, qui liste des pages mais ne répond pas : il faut ouvrir, lire et trier soi-même. La recherche Google ensuite, le réflexe majoritaire, désormais coiffée d'un Aperçu IA : sur notre question type, la réponse tombe en quelques secondes, correcte, l'office nommé, le formulaire et l'adresse cités, des pages de ge.ch en lien. Un assistant générique encore, ChatGPT ou l'un de ses semblables, qui répond dans un français limpide mais sans garantie de s'appuyer sur les pages officielles à jour. Le guichet ou le téléphone enfin, où un humain répond, aux heures d'ouverture. Le Tableau 8.2 résume ce paysage au regard du besoin formulé en 8.4 : aucune option ne coche toutes les cases, et c'est la ligne que Ge-Trouve entend occuper.

Tableau 8.2 : Les options d'un usager genevois face au besoin

Option	Réponse directe en langage courant	Sources officielles genevoises citées	Question traitée en Suisse	À toute heure	Gratuit
Recherche de ge.ch	non, une liste de pages	oui, les pages elles-mêmes	oui	oui	oui
Recherche Google (avec Aperçu IA)	oui, quand l'aperçu s'affiche	en partie, mêlées à tout le web	non	oui	oui
Assistant générique (ChatGPT et semblables)	oui	non garanties	non	oui	en partie
Guichet, téléphone	oui	oui	oui	non	oui
Ge-Trouve	oui	oui	oui	oui	oui

(élaboration personnelle)

L'Aperçu IA de Google mérite un examen honnête : sur la vitesse, il est imbattable, et il faut l'écrire. La différence est ailleurs, et elle est structurelle. La question de l'utilisateur part aux États-Unis, chez un acteur dont le modèle repose sur l'exploitation des données ;

pour des demandes qui touchent au séjour, à l'aide sociale ou à l'état civil, ce n'est pas neutre. La réponse puise dans tout le web, pages officielles ou non, genevoises ou non : rien ne garantit le périmètre ni la fraîcheur, et l'aperçu ne s'affiche que lorsque le moteur le décide. Aucune administration n'en maîtrise le contenu, et Google avertit lui-même que ces réponses peuvent contenir des erreurs. C'est enfin une boîte noire, sans corpus connu, sans évaluation publiée, sans code. Ge-Trouve ne prétend pas courir cette course-là. Il offre ce qu'un moteur commercial ne peut structurellement pas offrir : un périmètre exclusivement officiel et genevois, une exécution entière en Suisse, un système que chacun peut inspecter et évaluer.

Ce besoin n'est pas une intuition isolée : d'autres administrations y répondent déjà, ailleurs. Le précédent le plus proche est fribourgeois. En mars 2026, l'État de Fribourg a mis en ligne le chatbot IA de fr.ch (<https://fribourg.ch/fr/chatbot/>) : la population pose sa question en langage naturel, l'outil extrait les contenus pertinents du portail cantonal et de la législation, cite ses sources officielles, et le périmètre doit s'étendre aux communes, pour 62 000 francs de développement (Chancellerie de l'Etat de Fribourg 2026). Développé par l'agence Liip sur un modèle de Mistral AI et hébergé en Suisse chez Infomaniak, l'outil revendique la même souveraineté d'hébergement que Ge-Trouve (Dénervaud 2026). Un canton romand vient donc d'investir dans ce service exact, et il ne couvre pas Genève. Avant lui, la même agence avait ouvert la voie avec des démonstrateurs non officiels, LausanneGPT ou ZüriCityGPT, bâtis sur les sites de villes suisses mais adossés aux services d'OpenAI (Busslinger 2024). À l'échelon fédéral, les chatbots recensés servent des périmètres étroits, fondations, statistiques, caisse de compensation, souvent en interne (Chavanne 2024). En France, l'État a suivi l'autre chemin : Albert, l'assistant souverain de l'administration, répond aux agents du réseau France Services, pas aux citoyens (Gouvernement français 2024), et sa généralisation dans sa forme initiale a été abandonnée début 2026 (Agence France-Presse 2026). À Genève même, l'État explore : un assistant conversationnel a été expérimenté sur les géodonnées du territoire, une première suisse, dont l'expérimentation publique est aujourd'hui close (État de Genève (SITG) 2026). Rien, en revanche, pour les démarches administratives.

Le positionnement se laisse alors résumer en une matrice SWOT⁶⁴, la Figure 8.1 : forces et faiblesses internes, opportunités et menaces de l'environnement.

⁶⁴ SWOT (Strengths, Weaknesses, Opportunities, Threats) : matrice d'analyse stratégique croisant les forces et faiblesses internes d'un projet avec les opportunités et menaces de son environnement.

Figure 8.1 : La matrice SWOT de Ge-Trouve

Le positionnement de Ge-Trouve en quatre quadrants : l'interne en haut, l'environnement en bas.

ORIGINE INTERNE

Forces

- Souveraineté totale : modèle ouvert exécuté sur un serveur suisse, sans API externe
- Réponses sourcées vers les pages officielles
- Corpus dédié aux démarches genevoises
- Coût d'exploitation modeste
- Code et journaux publics
- Évaluation publiée, protocole reproductible

Faiblesses

- Latence élevée sur processeur
- Une seule génération à la fois
- Pas de mémoire conversationnelle
- Couverture partielle des sites officiels
- Projet individuel, maintenance non garantie

ORIGINE EXTERNE

Opportunités

- Extension à d'autres cantons ou aux communes
- Adoption ou reprise institutionnelle
- Modèles ouverts plus rapides à venir
- Précédent fribourgeois qui légitime la demande

Menaces

- Aperçus IA de Google : instantanés, gratuits, déjà dans le réflexe de recherche
- Évolution des sites sources cassant la collecte
- Exigences réglementaires croissantes sur l'IA

(élaboration personnelle)

Un balayage PESTEL⁶⁵ complète la matrice (Figure 8.2). Deux appuis y sont sourcés. Côté politique et légal : la stratégie cantonale de souveraineté, la dynamique de collaboration autour de l'IA publique et le cadre posé pour la fonction publique (État de Genève 2024; 2026a; 2026b). Côté socioculturel : l'ampleur du public concerné, la Belgique évaluant à 40 % la part de ses citoyens en vulnérabilité numérique et garantissant déjà par charte le droit d'obtenir un humain plutôt qu'une IA (Lokmane-Khelil 2025). Le reste de la figure relève du constat d'analyse.

⁶⁵ PESTEL : grille d'analyse de l'environnement d'un projet selon six dimensions : politique, économique, socioculturelle, technologique, environnementale et légale.

Figure 8.2 : Le balayage PESTEL de Ge-Trouve



(élaboration personnelle)

La différenciation tient alors en une phrase : Ge-Trouve est le seul service qui réponde en français courant aux démarches genevoises, en citant les pages officielles, avec un modèle ouvert exécuté intégralement sur une machine suisse et une évaluation publiée. Chacun de ces éléments existe ailleurs, séparément ; leur combinaison, non. C'est cette place, étroite mais réelle, que le prototype occupe.

La Partie 1 s'achève ici. Chaque composant est choisi, chaque choix instruit avec sa solution de repli ; la méthode d'évaluation est arrêtée, le terrain est reconnu, genevois et francophone, et le positionnement montre qu'aucun service équivalent n'existe. La Partie 2 peut passer à la construction : installer, assembler et faire fonctionner et la Partie 3 évaluera, une vingtaine de questions.

9. Spécifications et architecture du prototype

La Partie 1 s'est refermée sur des choix arrêtés ; la construction commence ici. Avant de construire le pipeline, ce chapitre fixe ce que le prototype doit faire et dans quel cadre il le fera. La section 9.1 traduit la question de recherche en exigences vérifiables. La section **Erreur ! Source du renvoi introuvable.** arrête le périmètre du corpus, resté ouvert jusqu'ici. La section **Erreur ! Source du renvoi introuvable.** pose l'architecture cible, du poste de développement au serveur. La section **Erreur ! Source du renvoi introuvable.** décrit l'environnement réellement installé et les premières mesures relevées. Les sections **Erreur ! Source du renvoi introuvable.** et **Erreur ! Source du renvoi introuvable.** instruisent enfin les deux dernières décisions de la pile : l'interface et le serveur frontal.

9.1 Cas d'usage et exigences

Le prototype vise un usager précis : une personne qui ne connaît pas l'administration genevoise. Un nouvel arrivant, par exemple, ou quelqu'un qui n'a jamais eu affaire à l'office concerné. Cette personne ne sait pas qui est compétent, ignore le nom des formulaires, et pose sa question avec ses mots : « Où déposer ma demande de permis de séjour ? », ou plus court encore. Ce profil découle de la question de recherche posée en introduction : c'est pour des utilisateurs non experts que les réponses doivent être fiables et pertinentes.

Le parcours attendu tient en trois temps. L'utilisateur écrit sa question dans une page web, en langue courante. L'assistant cherche dans les pages officielles indexées, rédige une réponse simple et l'accompagne des pages sur lesquelles elle s'appuie. L'utilisateur peut alors agir : il connaît l'office compétent et le formulaire à remplir, et il peut vérifier chaque affirmation à la source.

De ce parcours découlent les exigences du prototype, rassemblées dans le Tableau 9.1. Aucune n'est nouvelle : chacune reprend une conclusion déjà établie dans la Partie 1, et le tableau indique où. Les exigences fonctionnelles⁶⁶, codées F1 à F5 dans le tableau, décrivent ce que l'assistant doit faire. Comprendre les deux registres, la langue courante des questions et la langue administrative des pages, puis répondre dans un français

⁶⁶ Une exigence fonctionnelle décrit ce que le système doit faire ; une exigence non fonctionnelle décrit les conditions dans lesquelles il doit le faire : performances, exploitation, contraintes d'environnement.

simple qui conserve les noms exacts des offices, des formulaires et des permis, ces mots avec lesquels l'utilisateur agira ensuite (section 8.1). Citer ses sources dans chaque réponse, pour que l'utilisateur puisse vérifier au lieu de croire sur parole (section 2.2). Couvrir les démarches des deux administrations genevoises, l'État et la Ville, la source de chaque fragment restant lisible (section 9.2). Et savoir dire « je ne sais pas » quand les documents ne couvrent pas la question, plutôt que d'inventer : le contenu absent est le premier point de défaillance de la carte du chapitre 7, et le jeu d'évaluation contient des questions prévues pour l'éprouver (section 7.4).

Les exigences non fonctionnelles, codées NF1 à NF5, fixent le cadre d'exécution. L'assistant fonctionne entièrement en local : aucune question d'utilisateur, aucun document ne transite par un service d'intelligence artificielle externe. Cette règle, posée dès l'origine du projet, rejoint la stratégie cantonale de souveraineté numérique, qui vise la maîtrise des choix technologiques et de la gestion des données et veut éviter les dépendances difficilement réversibles (État de Genève 2026a). L'assistant tourne sur processeur seul, dans la mémoire du palier retenu au chapitre 4, aux côtés des autres postes du budget (section 4.1). Son temps de réponse est mesuré et publié, sans seuil fixé d'avance : la latence sur processeur est un coût assumé depuis la décision du chapitre 4, et les mesures diront ce qu'il vaut (section 4.4). Il reste exploitable par une seule personne, ce qui a déjà orienté le choix de la base embarquée : installation scriptée, aucun serveur séparé, sauvegarde par simple copie (section 6.4). Sa connaissance, enfin, se met à jour en réindexant les pages modifiées, sans réentraîner le modèle, la propriété démontrée dès l'article fondateur du RAG (section 2.3).

Chaque ligne du Tableau 9.1 porte ainsi son moyen de vérification : une mesure, une inspection ou une épreuve du protocole du chapitre 7. C'est ce qui permettra à la Partie 3 de juger le prototype sur pièces. Reste à dire sur quoi il sera compétent : le périmètre du corpus, objet de la section suivante.

Tableau 9.1 : Les exigences du prototype et leur moyen de vérification

Exigence	Origine	Vérification
F1. Comprendre la langue courante et la langue administrative	Section 8.1	Questions du jeu en plusieurs registres ; précision et rappel du contexte
F2. Répondre en français simple, noms exacts conservés	Section 8.1	Consigne de génération (chapitre 10) ; contrôle humain
F3. Citer ses sources dans chaque réponse	Section 2.2	Inspection des réponses ; fidélité
F4. Couvrir l'État et la Ville de Genève	Section 9.2	Présence des deux sources parmi les fragments remontés
F5. Dire « je ne sais pas » hors du corpus	Sections 7.1 et 7.4	Questions sans réponse du jeu
NF1. Exécution entièrement locale	Section 0	Revue de configuration : aucun appel externe
NF2. Processeur seul, dans la mémoire du palier	Section 4.1 ; mesures du 06.08.2026	Relevés de RAM (annexe des mesures)
NF3. Temps de réponse mesuré et publié	Sections 4.1 et 4.4	Mesures de latence de bout en bout
NF4. Exploitation par une seule personne	Sections 6.2 et 6.4	Scripts au dépôt ; procédure documentée
NF5. Mise à jour par réindexation	Section 2.3	Réindexation d'une page modifiée

(élaboration personnelle)

9.2 Le périmètre du corpus

Le corpus est la mémoire non paramétrique de l'assistant (section 2.3) : il ne saura répondre que sur ce qu'on y met. Son périmètre répond à deux questions à la fois. Ce que l'assistant doit couvrir, d'abord : le titre de ce travail le borne aux démarches administratives du canton de Genève, celles du catalogue officiel, du permis de conduire au logement subventionné. Ce que l'évaluation pourra juger, ensuite : une vingtaine de questions ne peuvent pas couvrir un site entier, et un corpus trop large dégrade la recherche elle-même en multipliant les fragments hors sujet (section 2.5). Le périmètre retenu s'en tient donc aux administrations genevoises, deux sources officielles, et le Tableau 9.2 en fixe les statuts.

La colonne vertébrale est le catalogue des démarches de <https://ge.ch/>, l'index officiel qui recense et classe les démarches de l'État par thématiques ; s'y ajoutent les pages pratiques qui les entourent, contacts, horaires et adresses des offices, et les pages publiques qui présentent les e-démarches⁶⁷. La seconde source est la Ville de Genève :

⁶⁷ E-démarches : guichet en ligne de l'État de Genève, accessible avec un compte personnel ; ce travail n'en indexe que les pages publiques de présentation.

les démarches de <https://geneve.ch/>, des crèches à la voirie, parce que l'utilisateur ne distingue pas toujours l'État de sa commune et que les guichets municipaux font partie de son quotidien administratif. La distinction reste lisible sans machinerie : chaque fragment porte sa source.

Quatre exclusions sont posées d'emblée. Les espaces e-démarches connectés : leur contenu vit derrière une authentification, seules leurs pages publiques de présentation entrent au corpus. Les annuaires privés comme search.ch : la règle des sources publiques officielles les écarte, et l'annuaire de ge.ch couvre le besoin. Le versant fédéral, ensuite : ch.ch et les offices fédéraux restent volontairement hors corpus, le sujet de ce travail étant genevois ; une question purement fédérale est une question hors périmètre. Les textes de loi bruts du recueil genevois (silgeneve.ch), enfin : leur registre juridique est précisément celui que l'assistant doit épargner à l'utilisateur, et les pages de démarches y renvoient déjà ; ils restent en réserve si l'évaluation révélait des manques. Quant aux documents PDF, formulaires et directives, le corpus initial s'en tient aux pages HTML ; leur ajout est une sous-décision de l'ingestion, à trancher au chapitre 10. Ce découpage a une conséquence assumée : une question hors périmètre, le calcul d'une rente AVS par exemple, doit recevoir un « je ne sais pas » qui oriente vers le bon guichet. C'est l'exigence F5 du Tableau 9.1, et le premier point de défaillance de la carte du chapitre 7 (section 7.1).

Ce périmètre s'opérationnalisera par le manifeste des sources : un fichier versionné qui liste les pages retenues, chacune datée à sa capture, et dont le pipeline d'ingestion fera sa seule entrée. Le Tableau 9.2 arrête les statuts ; l'architecture qui exploitera ce corpus est l'objet de la section suivante.

Tableau 9.2 : Les sources du corpus : statuts et motifs

Source	Statut	Motif
ge.ch catalogue des démarches, pages pratiques et pages publiques des e-démarches	Retenu, cœur	Index officiel des démarches de l'État ; contacts, horaires, adresses ; espaces connectés exclus
geneve.ch démarches de la Ville	Retenu	Guichets municipaux du quotidien ; source lisible fragment par fragment
silgeneve.ch recueil législatif	Réserve	Textes de loi bruts, registre éloigné des usagers

(élaboration personnelle)

9.3 L'architecture cible

L'architecture tient en deux chaînes, que la Figure 9.1 met en page. Une chaîne hors ligne construit la mémoire documentaire ; une chaîne en ligne la consulte à chaque

question. C'est le pipeline classique de la section 2.4, incarné par les composants arrêtés aux chapitres 3 à 6 et complété par les pièces propres au déploiement.

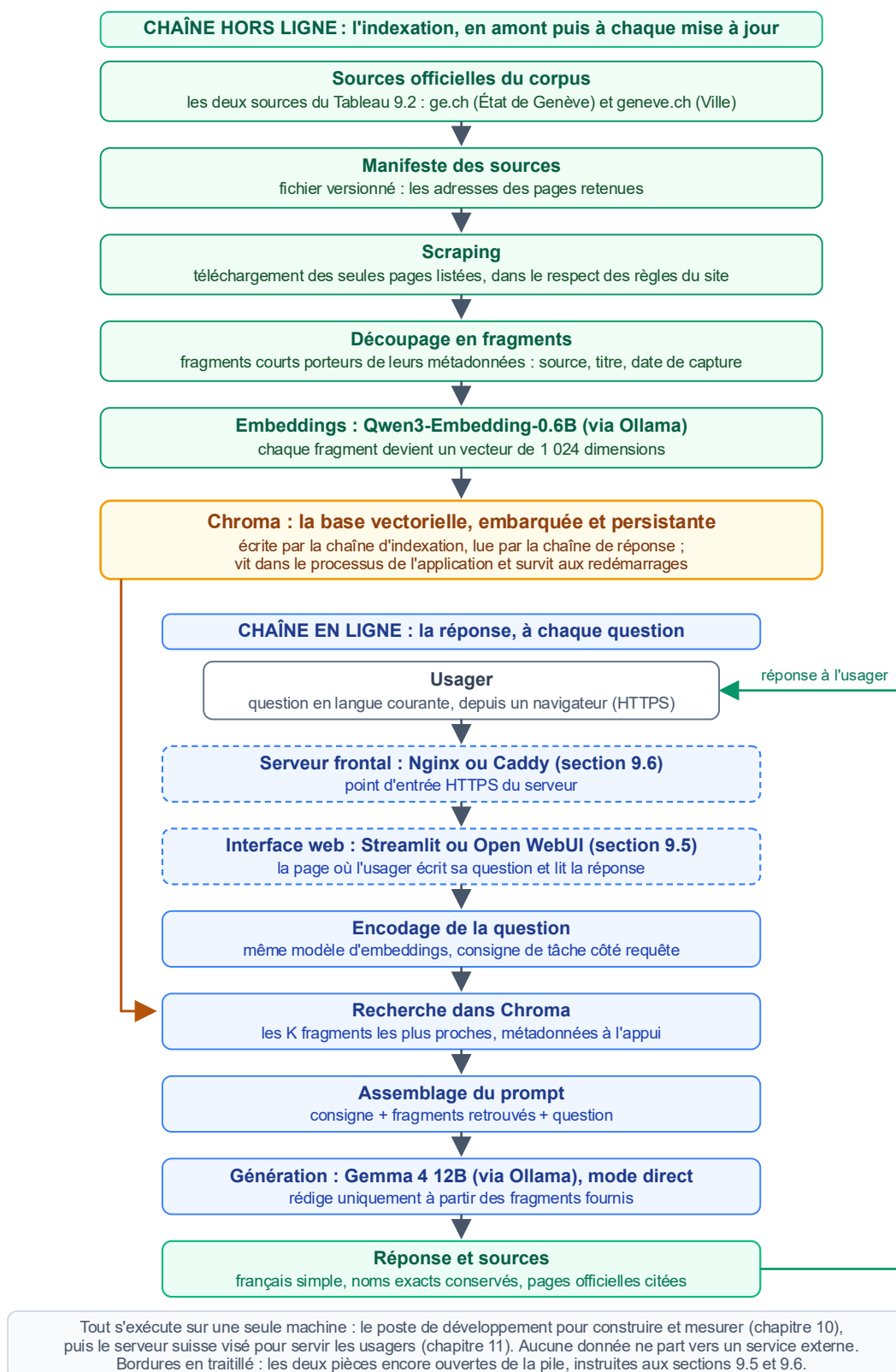
La chaîne hors ligne part du manifeste des sources posé à la section 9.2 : le scraping télécharge les pages listées, le découpage produit des fragments porteurs de leurs métadonnées, le modèle d'embeddings les convertit en vecteurs, et Chroma les range et les conserve. Elle s'exécute en amont, puis à chaque mise à jour du corpus : réindexer une page modifiée suffit à actualiser l'assistant (exigence NF5 du Tableau 9.1).

La chaîne en ligne se déclenche à chaque question. La requête de l'utilisateur traverse le serveur frontal puis l'interface web, les deux pièces encore ouvertes qu'instruisent les sections 9.5 et 9.6 ; le pipeline Haystack encode la question avec le même modèle d'embeddings, interroge Chroma, assemble le prompt et confie la rédaction à Gemma 4 12B en mode direct ; la réponse revient à l'utilisateur avec ses pages sources. Tout s'exécute sur une seule machine : d'abord le poste de développement, où le chapitre 10 construira et mesurera l'assemblage, puis le serveur suisse visé, où le chapitre 11 le déploiera à l'identique. Aucun composant externe n'entre en jeu, ni pour indexer ni pour répondre (exigence NF1).

Cette architecture suppose un environnement qui la porte : c'est l'objet de la section suivante.

Figure 9.1 : L'architecture cible du prototype, de l'indexation à la réponse

L'assistant assemble deux chaînes qui s'exécutent entièrement en local, sans aucun service externe : la chaîne hors ligne construit la mémoire documentaire ; la chaîne en ligne la consulte à chaque question.



(élaboration personnelle)

9.4 L'environnement de développement et les premières mesures

Le développement s'appuie sur deux postes Windows, et ce doublon est un choix. Le premier, PC : DANIELGARCIA, est la machine principale : un AMD Ryzen 7 7840HS avec 32 Go de mémoire DDR5. Le second, PC : GARCIAD, un Intel Core i7-12700 avec 16 Go de DDR4, sert de contrepoint : sa mémoire, plus proche des paliers du serveur visé, donne une idée plus honnête de ce que le VPS supportera. Sur les deux, l'inférence se fait sur processeur seul, la puce graphique intégrée n'étant pas prise en charge par Ollama sous Windows ; chaque poste a son environnement Python, le dépôt du projet étant partagé entre les deux.

L'environnement a été installé et figé le 6 août 2026 : Python 3.13, Ollama (0.32.5 et 0.32.6 selon le poste), et les trois paquets Haystack épinglés dans requirements.txt, haystack-ai 3.0.0, ollama-haystack 6.8.0 et chroma-haystack 4.4.0. Une précision de version s'impose. La section 3.4 avait arrêté la lignée 2.x, alors en version 2.31 avec une transition 3.0 annoncée ; à l'installation, cette 3.0.0 était devenue la version courante. Elle conserve l'architecture à pipelines explicites qui fondait la décision, vérification faite sur les composants de référence, et l'épinglage fige l'environnement : le choix du chapitre 3 reste inchangé, seule la version a suivi son cours. Les trois modèles ont été tirés par un script versionné, identiques sur les deux postes : gemma4:12b (7,6 Go sur disque), llama3.1:8b (4,9 Go), qui servira de repli et de juge d'évaluation (section 7.3), et qwen3-embedding:0.6b (639 Mo).

Les premières mesures datent du même jour ; le détail, poste par poste, est consigné dans l'Annexe 2 des mesures. Trois chiffres comptent pour la suite. Les empreintes en mémoire des modèles chargés, d'abord, identiques sur les deux machines : 8,9 Go pour Gemma, 5,6 Go pour Llama, 2,4 Go pour Qwen. Le budget de production, ensuite : Gemma et Qwen chargés ensemble occupent environ 11,3 Go, ce qui tient à l'aise dans le palier de confort de 18 Go et confirme, mesure à l'appui, l'arbitrage conjoint du chapitre 4 ; au palier plancher de 12 Go, la bascule sur Llama redeviendrait nécessaire (section 4.4). Un écart, enfin, à noter honnêtement : le poste embeddings mesuré dépasse l'hypothèse de la section 4.1 (2,4 Go chargés contre 0,5 à 1,5 Go supposés). Ces hypothèses étaient annoncées comme telles, et la confrontation complète du budget se fera au chapitre 10, sur le pipeline entier.

Deux enseignements de vitesse complètent le tableau. Sur processeur, la génération atteint 5,6 jetons par seconde pour Gemma et 9,3 pour Llama sur le poste principal ; le second poste va nettement plus lentement, tout en produisant son premier jeton plus

vite. La vitesse d'écriture tient donc plus à la mémoire de la machine qu'à sa puissance de calcul brute, et celle du serveur ne se prédira pas d'ici : le chapitre 11 la mesurera. La vérification du mode direct, ensuite : Gemma réfléchit par défaut, et c'est le paramètre d'appel « `think:false` »⁶⁸ qui le coupe ; sur la question type, la réflexion fait passer la réponse d'environ 3 secondes à environ 50. Le régime direct retenu à la section 4.1 s'en trouve confirmé.

L'environnement est prêt et chiffré, et les scripts de mesure sont versionnés au dépôt du projet (Garcia 2026). Restent les deux dernières pièces de la pile à choisir : l'interface, d'abord.

9.5 L'interface

L'interface n'est pas un maillon du pipeline RAG : c'est la page qui reçoit la question et affiche la réponse. Le besoin est précis : une page de conversation qui tourne en local au-dessus du pipeline Haystack et d'Ollama, sur le budget mémoire du chapitre 4, et qu'une seule personne peut faire vivre. La décision se traite donc en version courte, sur des faits relevés au 6 août 2026, date de référence des deux dernières sections ; la pièce reste remplaçable sans toucher à la chaîne.

Pourquoi ces deux candidats ? Écrire l'interface à la main, avec un cadre web généraliste, ajouterait du développement sans rien apporter à la question de recherche. Restent deux familles d'outils prêts à l'emploi. D'un côté, les applications de conversation clés en main pour modèles locaux : Open WebUI en est le représentant le plus suivi, avec plus de 148 000 étoiles au relevé (Open WebUI 2026). De l'autre, les bibliothèques Python qui engendrent la page web depuis le script : Streamlit en est la référence, utilisée par plus d'un million de dépôts au relevé (Streamlit 2026). Chaque famille compte d'autres représentants ; l'instruction courte confronte les deux têtes de file.

Open WebUI est une application complète : une interface auto-hébergée pour Ollama et les API compatibles, conçue pour fonctionner hors ligne, riche en fonctions. Trois réserves la disqualifient ici. Elle embarque son propre moteur RAG : la brancher reviendrait à superposer un second pipeline à celui de ce travail, ou à contourner les pièces que l'évaluation doit précisément tracer. C'est une application de plus à administrer, avec son service, ses comptes et ses mises à jour, pour un exploitant seul (exigence NF4 du Tableau 9.1). Et sa licence propre impose de conserver la marque

⁶⁸ Paramètre de l'interface de programmation d'Ollama qui désactive la production du brouillon de raisonnement avant la réponse.

Open WebUI dans tout déploiement, hors exceptions limitées : moins simple qu'une licence permissive classique pour un service public.

Streamlit est l'inverse : une bibliothèque Python sous licence Apache 2.0, qui transforme un script en application web et sert couramment à créer des pages de conversation. Elle s'embarque dans l'application : la page appelle directement la fonction de réponse du pipeline, rien ne s'intercale entre l'utilisateur et la chaîne, aucune pièce d'exploitation ne s'ajoute.

Ce travail retient donc Streamlit : l'intégration directe, qui garantit que tout ce que l'évaluation mesurera passe par le pipeline du projet ; la simplicité d'exploitation, dans la ligne du choix de la base embarquée au chapitre 6 ; la licence permissive. Les renoncements sont nommés : pas de comptes ni d'historique natifs, ce qui convient à un prototype qui ne stocke aucune donnée d'utilisateur ; une montée en charge modeste, suffisante ici, à revoir si l'usage croissait. Open WebUI reste le repli documenté si le besoin évoluait vers une façade de conversation générique. Reste la dernière pièce, devant l'interface : le serveur frontal.

9.6 Le serveur frontal

Le serveur frontal est la porte du serveur : il écoute les ports du web, porte le HTTPS et transmet les requêtes à l'interface. Le besoin est là aussi précis : exposer ge-trouve.ch en HTTPS sur le VPS, avec des certificats qui s'obtiennent et se renouvellent sans intervention, une configuration tenable par un exploitant seul, une empreinte négligeable à côté des modèles. Exposer l'interface directement, sans frontal, se pratique en développement mais pas en production : la terminaison du chiffrement, la redirection du port 80 et les en-têtes de sécurité sont le métier d'un serveur frontal, pas d'une bibliothèque d'interface. Parmi les frontaux libres, deux têtes de file s'imposent, et chaque famille compte d'autres représentants : Nginx, le serveur en place, et Caddy, le représentant du HTTPS automatique.

Nginx est le classique du domaine : un logiciel libre sous licence BSD simplifiée à deux clauses, distribué en paquets pour tous les grands systèmes, avec des distributions d'entreprise et un support portés par F5 (Nginx 2026). Rien à lui reprocher sur la solidité : il est éprouvé et documenté partout. La réserve tient au HTTPS : la gestion automatique des certificats ne fait pas partie de son cœur : en pratique, on les obtient et on les renouvelle par un outil externe, puis on entretient la configuration correspondante. Pour un exploitant seul et un site unique, ce sont des pièces en plus, et autant d'occasions d'erreur au renouvellement.

Caddy prend le problème à l'envers : un serveur web multiplateforme dont le HTTPS est automatique par défaut, certificats gérés automatiquement auprès de ZeroSSL ou Let's Encrypt⁶⁹ ; le premier serveur à avoir fait du HTTPS le comportement par défaut, éprouvé en production, configuré par un fichier court, le Caddyfile, et livré en binaire unique sans dépendance, écrit en Go, sous licence Apache 2.0 (Caddy 2026).

Ce travail retient donc Caddy : le HTTPS automatique supprime l'entretien des certificats, la tâche la plus risquée du poste pour un exploitant seul (exigence NF4 du Tableau 9.1) ; la configuration d'un site unique tient en quelques lignes ; le binaire unique s'installe et se met à jour simplement sur le VPS. Les renoncements sont nommés : Nginx reste plus répandu et mieux connu des administrateurs, et sans égal à très forte charge, un registre sans objet ici ; l'écosystème de Caddy est plus jeune. Nginx est le repli documenté, avec renouvellement externe des certificats.

La pile est désormais complète : du corpus au frontal, tous les choix sont arrêtés. Le chapitre 10 passe à la pratique : construire le pipeline, le faire fonctionner, et le mesurer.

10. Construction du pipeline RAG

La construction commence par un essai court, mené de bout en bout. Avant d'ingérer tout le corpus, ce chapitre construit le pipeline sur un périmètre pilote, les pages du permis de séjour, et le fait tourner sur les deux postes de développement : si l'assemblage fonctionne en local et que les mesures des deux machines concordent, le choix du serveur à louer pourra se prendre sur des chiffres. La section 10.1 couvre le corpus pilote, la 10.2 son indexation, la 10.3 l'assemblage et sa première réponse, la 10.4 les mesures du système et le palier de serveur retenu. La section 10.5 construit la consigne de génération, et la 10.6 étend l'ingestion au corpus complet.

10.1 Le corpus pilote

Le pilote applique le périmètre de la section 9.2 à sa plus petite échelle utile : le permis de séjour, le terrain de la question type. Le manifeste des sources est un fichier versionné du dépôt (*src/scraping/manifeste_sources_pilote.csv*) qui liste 23 pages de ge.ch : douze retenues d'emblée, dont six pages « chapeau », auxquelles s'ajoutent onze sous-pages relevées sur ces chapeaux. Le pipeline d'ingestion ne connaît que ce fichier : ajouter une page au corpus, c'est ajouter une ligne.

⁶⁹ ZeroSSL et Let's Encrypt sont des autorités de certification qui délivrent gratuitement, par un protocole automatisable, les certificats attestant l'identité d'un site et permettant le chiffrement HTTPS.

Avant d'aspirer quoi que ce soit, un script (*scripts/scraping/verifier_robots.py*) interroge le robots.txt du site⁷⁰ : les 23 pages sont autorisées, aucun délai n'est imposé, et le projet applique le sien, deux secondes entre chaque requête, avec un agent identifiable. Le scraper (*src/scraping/scrape_pilote.py*) a ensuite téléchargé les 23 pages sans échec, en moins d'une minute presque entièrement occupée par ce délai de politesse : à l'échelle du corpus complet, c'est lui qui fixera la durée, pas la machine. Chaque page devient un fichier JSON portant son texte et les métadonnées que la recherche exploitera ensuite : adresse, titre, section, date de capture.

L'extraction du texte principal est confiée à Trafilatura, une bibliothèque Python spécialisée qui isole le contenu d'une page et écarte menus, entêtes et pieds de page (Barbaresi 2026). Le texte ressort propre, mais un renoncement apparaît : les hyperliens disparaissent avec le balisage. Or c'est souvent le lien qui dit où cliquer pour déposer sa demande. Le constat est consigné au journal des mesures du dépôt (*docs/mesures/journal-des-mesures.md*) (Garcia 2026) et son traitement renvoyé à la section 10.6, avec le corpus complet. Reste à transformer ces 23 textes en mémoire interrogeable : l'indexation.

10.2 L'indexation pilote

L'indexation transforme les 23 textes en mémoire interrogeable. Le script du dépôt (*src/indexing/indexer_pilote.py*) enchaîne les trois gestes préparés aux chapitres 5 et 6 : découper, encoder, ranger.

Le découpage suit un réglage initial, posé comme révisable tant que l'évaluation n'a pas parlé (Partie 3) : des fragments de 200 mots, avec un recouvrement de 40 pour ne pas couper une information en deux. Les 23 pages donnent 72 fragments, les pages de fond pesant naturellement plus que les brèves pages chapeau. Chaque fragment emporte ses métadonnées, adresse, titre, section, date de capture et position dans la page ; c'est la source qui garde lisible la provenance de chaque extrait.

L'encodage passe par Qwen3-Embedding-0.6B, sans consigne côté documents, la consigne de tâche restant réservée aux requêtes comme le prévoit la section 5.4 : la dimension des vecteurs, 1 024, est vérifiée à l'écriture. Chroma range le tout dans sa base persistante, hors dépôt comme toutes les données : 2,7 Mo pour ce pilote. L'écriture est idempotente : l'identifiant de chaque fragment est un condensé de son

⁷⁰ Robots.txt : fichier déposé à la racine d'un site web, par lequel celui-ci indique aux programmes automatisés les pages qu'ils peuvent ou non visiter.

contenu, et ré-exécuter l'indexation remplace au lieu de dupliquer. C'est déjà, en petit, le geste de la mise à jour par réindexation (exigence NF5 du Tableau 9.1).

Une mesure complète le tableau : 0,8 fragment par seconde sur le poste principal, sous les 1,2 documents du banc d'essai de la section 9.4. L'écart s'explique : les fragments sont plus longs que les paragraphes du banc, et la durée inclut le chargement à froid du modèle (Garcia 2026). À cette cadence, l'indexation n'est pas le poste critique du corpus complet ; le délai de politesse du scraping pèsera toujours davantage. Reste à faire répondre cette mémoire : l'assemblage du pipeline.

10.3 Le pipeline de réponse

L'assemblage tient dans un fichier du dépôt (*src/repondre.py*), avec une configuration commune lue depuis l'environnement (*src/config.py*). Le pipeline Haystack y est explicite, composant par composant, connexion par connexion, comme le chapitre 3 l'avait retenu. Quatre maillons : l'encodeur de requête, qui applique à la question la consigne de tâche au format prévu par la section 5.4 ; la recherche Chroma, qui remonte les cinq fragments les plus proches ; le constructeur de prompt, qui assemble un message système et un message utilisateur, les extraits chacun précédés de leur adresse, puis la question ; et le générateur, Gemma 4 12B en mode direct. Le paramètre *think:false* est transmis nativement par l'intégration, et vérifié : au premier appel journalisé, le champ de raisonnement revient vide. En bout de chaîne, une commande : une question en entrée, la réponse et la liste des fragments utilisés en sortie.

Le message système porte une consigne provisoire, quatre instructions simples : l'assistant s'appelle Ge-Trouve, répond en français simple, s'appuie uniquement sur les extraits fournis en conservant les noms exacts, et le dit clairement quand les extraits ne suffisent pas. La section 10.5 la remplacera par une consigne construite et testée.

La première réponse à la question type est tombée juste : la demande se dépose auprès de l'OCPM, en ligne ou par courrier, pages sources à l'appui. Juste, sourcée, et incomplète : ni lien vers le guichet en ligne, ni adresse. Rien n'est inventé pour combler, la fidélité tient (section 7.2) ; ce qui manque à la réponse manque au corpus, les hyperliens retirés à l'extraction (section 10.1), les e-démarches et les formulaires hors du périmètre pilote (section 9.2). Il en sort deux chantiers : la consigne (section 10.5) et le corpus (section 10.6).

Le Tableau 10.1 récapitule les réglages du pilote et leur statut ; aucun n'est définitif. Restent les mesures du système assemblé, et la décision qu'elles portent.

Tableau 10.1 : Les réglages du pipeline pilote et leur statut

Réglage	Valeur du pilote	Statut
Découpage	200 mots, recouvrement 40	Révisable à l'évaluation
Fragments remontés (top_k)	5	Révisable à l'évaluation
Consigne de tâche, côté requête	Format de la carte Qwen, libellé d'origine	Révisable
Consigne de génération	Provisoire : rôle, extraits seuls, noms exacts, aveu d'ignorance	Remplacée à la section 10.5
Génération	Gemma 4 12B en mode direct (think:false)	Régime de production (section 9.4)

(élaboration personnelle)

10.4 Les mesures du système et le palier de serveur

Le pipeline assemblé a été mesuré sur les deux postes, aux mêmes conditions que les bancs d'essai : une exécution à froid, trois à chaud, médiane (*scripts/mesures/bench_pipeline.py*). À chaud, en rejouant la question type, la réponse tombe en huit secondes environ sur les deux machines ; l'exigence d'un temps mesuré et publié est tenue (NF3 du Tableau 9.1) (détail à l'Annexe 2). À froid, le premier appel coûte 73 à 88 secondes selon le poste, le temps de charger les modèles : un coût unique, payé au démarrage du service, qui frappe surtout la machine à 16 Go, où charger 11,3 Go force la pagination.

La mémoire, justement, porte la décision. Les empreintes ne bougent pas d'un poste à l'autre : Gemma et Qwen chargés font 11,3 Go, le processus du pipeline et de Chroma moins de 100 Mo. Sur 16 Go, l'ensemble tient tout juste, 0,3 Go libre en fonctionnement, environnement de développement compris. Le palier plancher de 12 Go est donc exclu pour le régime retenu, et 16 Go serait un plancher extrême ; un serveur épuré rendrait environ 1,6 Go pris ici par les outils de développement, mais la marge resterait mince. Le palier de confort de 18 Go est retenu ; la mesure du système complet confirme l'arbitrage conjoint du chapitre 4 (section 4.4), et la location du serveur se fera au chapitre 11 sur cette base. Le détail des deux sessions vit dans l'Annexe 2, refondue pour accueillir chaque étape.

Le système répond, et il est chiffré. Reste à lui apprendre à mieux répondre : la consigne.

10.5 La consigne de génération

La consigne provisoire a fait son temps. Pour la remplacer, l'étape s'est donné un petit banc d'essai : huit questions de mise au point, six tirées du corpus pilote et deux hors corpus,

portées	par	un	fichier	versionné
---------	-----	----	---------	-----------

(*src/generation/questions_mise_au_point_pilote.py*). Ce jeu ne sert qu'à régler la consigne ; il est distinct du jeu d'évaluation du chapitre 7, qui doit arriver vierge à la Partie 3. Un script rejoue chaque variante sur chaque question, à conditions égales (*scripts/mesures/comparer_consignes.py*). Réponses et latences sont archivées au dépôt, dans *resultats/consignes/* : les réponses pour trancher, les latences en simple relevé. Les variantes vivent dans un module unique (*src/generation/consignes.py*), d'où le pipeline lit la consigne active.

Les premiers essais ont tranché un point, à rebours de l'intuition : la brièveté. Les variantes qui l'imposaient tronquaient ou appauvrissaient les réponses, alors qu'un usager qui demande où déposer sa demande veut aussi les conditions, les pièces et les délais. Le critère a donc changé : non plus une réponse brève, mais une réponse complète et fondée, qui ne dit rien d'autre que ce que disent les extraits.

Le deuxième enseignement porte sur l'échantillonnage. Au régime recommandé par l'éditeur, température⁷¹ à 1.0, Gemma déroule sur processeur jusqu'au plafond de jetons : 400 à 500 jetons par réponse, 100 à 160 secondes chacune, parfois plus de 300. Trop bavard et trop lent pour ce cas d'usage. La température a été abaissée à 0.3 : les réponses deviennent quasi reproductibles, ce qui permet de comparer les variantes à conditions égales, un *seed*⁷² étant fixé le temps des essais. Le plafond de jetons a suivi le chemin inverse : 256 coupait des réponses en pleine phrase, il passe à 512 pour les essais puis à 1 024 en production ; c'est un plafond, pas une cible, le modèle s'arrêtant de lui-même à basse température. Le régime de production est acté ainsi : température 0.3 ; top_p 0.95 et top_k 64⁷³, les valeurs recommandées par l'éditeur, conservées faute de raison mesurée d'y toucher (Ollama 2026b) ; un plafond de 1 024 jetons ; un contexte de 4 096 jetons, inchangé depuis les premières mesures (section 9.4) ; le tout sans seed.

La consigne retenue, quatrième variante essayée et notée V3, les versions successives étant numérotées de V0, la consigne provisoire, à V3, tient en six demandes: s'appuyer uniquement sur les extraits ; donner une réponse complète et concrète, conditions, délais, pièces et démarche ; écrire en français simple en conservant les noms exacts, OCPM, formulaire M, permis B ou C ; citer les pages officielles utilisées, à partir des

⁷¹ La température règle la part de hasard dans la génération : à 0, le modèle choisit toujours le jeton le plus probable et la réponse est déterministe ; en montant vers 1, il tire davantage au sort, ce qui varie les formulations et, sur ce matériel, allonge nettement les réponses.

⁷² Seed : valeur d'amorçage du générateur de nombres aléatoires ; à seed identique, les tirages se répètent et la réponse redevient reproductible.

⁷³ Top_k et top_p bornent le tirage aléatoire aux jetons les plus probables : top_k limite le nombre de candidats (ici 64), top_p la masse de probabilité cumulée admise (ici 95 %).

adresses fournies avec les extraits ; ne reprendre un formulaire, un numéro ou une adresse que s'il figure dans les extraits ; et, hors corpus, avouer son ignorance en une phrase puis orienter vers le guichet ou le site officiel, sans rien inventer. Face au témoin, elle l'emporte sur la complétude, la citation des sources et l'orientation hors corpus. Un contrôle a suivi, central pour la fidélité (section 7.2) : les détails les plus pointus de ses réponses, le SEFRI, un délai de six à huit semaines, le permis Ci, la carte de légitimation du DFAE, ont tous été retrouvés dans les pages sources du corpus pilote. La richesse vient des extraits, pas de l'imagination du modèle.

Cette complétude a un prix, mesuré à part sur le poste principal au repos : les latences des séries d'essais ne valaient pas référence, l'une perturbée par une mise en veille nocturne du poste principal, l'autre par la surchauffe du second poste (détail à l'Annexe 2). Sur la question type, rejouée à chaud, la réponse passe de quelques secondes avec le témoin (section 10.4) à une quarantaine avec la consigne retenue ; les réponses détaillées des essais dépassent parfois les deux minutes (détail à l'Annexe 2). La latence suit la longueur de la réponse, de quelques dizaines de secondes à près de trois minutes sur ce processeur : l'arbitrage entre complétude et vitesse est assumé, et le chapitre 12 le jugera sur pièces. Une limite, enfin : la consigne promet de reprendre formulaires et contacts quand les extraits les portent, mais le corpus pilote en contient peu. Elle prendra toute sa mesure avec le corpus complet, l'affaire de la section suivante.

10.6 Le corpus complet et l'ingestion reproductible

Le pilote avait prouvé la chaîne sur 23 pages ; reste à la faire tourner sur le corpus arrêté à la section 9.2. Encore faut-il savoir quelles pages existent. La réponse passe par le *sitemap*⁷⁴ de chaque domaine : le fichier par lequel un site publie lui-même la liste de ses pages, la voie officielle pour l'énumérer. Celui de ge.ch est annoncé par son robots.txt ; celui de geneve.ch est exposé à l'emplacement standard. Un script du dépôt (*scripts/scraping/decouvrir_sitemap.py*) récupère cette liste, classe les adresses par catégorie, le premier segment du chemin, puis filtre ; le raisonnement complet est documenté à côté du script (*scripts/scraping/decouverte_corpus.md*).

Le paysage révélé justifie le filtrage. Sur les 27 484 pages de ge.ch, l'essentiel est du bruit pour un assistant de démarches : 17 362 fiches de documents en vrac, des milliers d'actualités et d'événements datés, des variantes de langue. Restent 4 072 pages, les démarches et les dossiers thématiques. Le site de la Ville est rangé autrement : sur 31 218 pages, très majoritairement institutionnelles, les démarches vivent toutes dans un

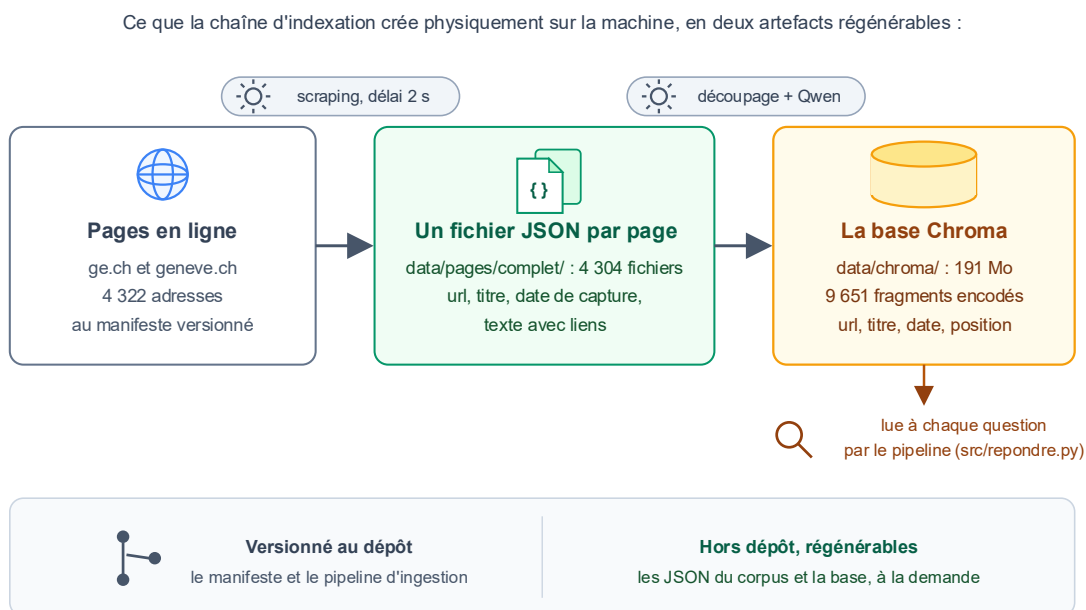
⁷⁴ Sitemap : fichier, généralement XML, qu'un site publie pour lister ses propres pages à l'intention des programmes qui l'explorent ; norme sitemaps.org.

dossier propre, et le filtre inverse, ne garder que ce dossier, en retient 250. La raison de fond n'est ni la place ni le temps, mais la pertinence : un corpus rempli d'actualités ferait remonter du bruit dans la recherche (section 2.5). Le manifeste final compte 4 322 adresses, une par ligne ; le titre de chaque page sera extrait à l'aspiration.

Le scraping reprend les règles du pilote, à l'échelle : robots.txt vérifié par domaine et à chaque page, agent identifiable, délai de deux secondes, une passe, reprise possible (*src/scraping/scrape_complet.py*). Résultat : 4 304 pages sur 4 322, en deux heures et demie environ, dominées par le délai de politesse comme la section 10.1 l'annonçait. Les 18 pages manquantes sont des exclusions justifiées, une page supprimée et dix-sept pages d'accès restreint, non contournées. Et le renoncement constaté au pilote est levé : l'extraction conserve désormais les hyperliens, résolus en adresses complètes. L'assistant peut ainsi citer le guichet en ligne ou un formulaire sans que la page du document soit indexée ; le contenu des PDF reste hors corpus, à revoir seulement si l'évaluation révélait un manque.

L'indexation garde les réglages du Tableau 10.1 : fragments de 200 mots, recouvrement de 40, embeddings Qwen, dimension 1 024 vérifiée (*src/indexing/indexer_complet.py*). Chaque fragment porte son adresse, son titre, sa date de capture et sa position dans la page ; le texte encodé est nettoyé des adresses, la version avec liens étant conservée à part pour la génération. La base atteint 9 651 fragments, 2,24 par page, pour 191 Mo, au même débit que le pilote, 0,8 fragment par seconde. L'ingestion est reproductible au sens strict : les identifiants sont des empreintes du contenu, et une exécution interrompue reprend où elle s'est arrêtée. L'incident réel l'a démontré : une mise en veille a coupé l'indexation aux deux tiers ; veille désactivée, la reprise a traité le solde en 70 minutes, et la base finale est identique à ce qu'aurait produit une passe unique, au fragment près. Au total, environ trois heures et demie de calcul, le prix d'un corpus entier encodé sur processeur, payé une fois. La Figure 10.1 situe ces artefacts, de la page en ligne à la base sur disque

Figure 10.1 : Les artefacts du pipeline, de la page en ligne à la base sur disque



(élaboration personnelle)

La validation est venue de la question type. Sur le corpus complet, le pipeline, dont le point d'entrée devient *src/repondre.py*, rend une réponse fondée et sourcée, plus complète que sur le pilote : elle couvre l'arrivée d'un autre canton, la recherche d'emploi, le regroupement familial et l'asile, et cite ses liens officiels, dont une fiche de document jamais aspirée. La limite pointée à la section 10.5 est levée par le corpus, exactement comme prévu. Le tout sans un octet hors de la machine : découverte, aspiration, encodage et base restent locaux, et le dépôt versionne le manifeste et le pipeline, pas le corpus brut, régénérable à la demande. Le détail chiffré vit à l'Annexe 2.

Le chapitre s'était ouvert sur un pilote de 23 pages ; il se referme sur le corpus entier. Le pipeline est construit, réglé, nourri et chiffré : le système sait répondre. Restent l'interface et le serveur : l'affaire du chapitre suivant.

11. Interface et déploiement

Le système sait répondre ; il lui faut une porte d'entrée. Ce chapitre couvre les deux dernières pièces du prototype : l'interface, qui met le pipeline entre les mains du citoyen, et le déploiement, qui l'installera sur son serveur. C'est ici que l'accessibilité, fil rouge de ce travail, devient concrète : tout ce qui précède ne vaut que si un usager non expert peut poser sa question et se fier à la réponse. La section 11.1 présente la page, la 11.2 le traitement de l'attente, la 11.3 portera le déploiement.

11.1 Une page pour le citoyen

L'interface tient en une page (*src/app/app.py*), construite avec Streamlit comme arrêté à la section 9.5 et lancée d'une seule commande. La promesse d'alors est tenue : la page appelle directement le pipeline de production (*src/repondre.py*) et n'y ajoute rien, ni compte, ni réglage, ni second moteur ; ce que l'évaluation mesurera au chapitre 12 est exactement ce que l'utilisateur utilise. Et elle tourne sur la même machine que le reste, sans un appel vers l'extérieur : l'exigence d'exécution locale (NF1 du Tableau 9.1) tient jusqu'à l'écran.

L'écran s'en tient à l'essentiel, et chaque élément sert l'utilisateur décrit à la section 9.1. Le champ de saisie invite à poser sa question sur une démarche genevoise, en langue courante : c'est l'exigence F1. La réponse s'écrit en français simple dans la conversation, avec les liens officiels cliquables directement dans le texte, guichet en ligne ou formulaire, rendus possibles par la préservation des hyperliens (section 10.6) : l'utilisateur peut agir sans chercher plus loin. La barre latérale, elle, liste les pages sources de la dernière réponse : les pages d'où viennent les cinq fragments retenus par la recherche (*top_k 5*, Tableau 10.1). Une mention le rappelle sous la liste : les réponses s'appuient sur les pages officielles citées, à vérifier à la source. L'utilisateur peut ainsi ouvrir chaque page et juger par lui-même plutôt que croire sur parole (section 2.2), l'exigence F3 devenue un geste. Un indicateur d'état complète l'écran, en attente, génération en cours, prêt. Le pied de page assume le cadre : un prototype de travail de Bachelor, dont le code est public (github.com/DanielGarcia85/ge-trouve-ch).

Un choix mérite explication : l'assistant n'a pas de mémoire conversationnelle. L'historique reste affiché à l'écran le temps de la session, mais chaque question est traitée pour elle-même, le modèle ne recevant que la question courante et ses extraits. Trois raisons. C'est le régime du RAG retenu (section 2.5) ; c'est celui du protocole d'évaluation, qui jugera question par question (chapitre 7) ; et c'est le plus simple pour un prototype qui ne stocke aucune donnée d'utilisateur, dans la ligne des renoncements assumés à la section 9.5. L'Annexe 3 montre la page au repos.

Reste l'attente, le vrai défi d'un assistant qui calcule sur processeur.

11.2 Le streaming et la consigne V5

Sur processeur, la génération prend son temps : selon le poste et la longueur de la réponse, la réponse complète à la question type s'écrit en une à deux minutes à chaud (détail à l'Annexe 2). Ce temps de calcul est le prix assumé de la souveraineté depuis le chapitre 4, et l'interface ne peut pas le réduire. Ce qu'elle peut changer, c'est l'attente

ressentie. D'où la distinction qui structure cette section : la latence de calcul, le temps que met la réponse à exister en entier, et la latence perçue, le temps avant que l'utilisateur ait quelque chose à lire.

La réponse est donc diffusée en *streaming*⁷⁵ : chaque mot s'affiche dès que le modèle le produit, au lieu d'un écran muet suivi d'un bloc. L'utilisateur lit ainsi pendant que la suite s'écrit, l'indicateur d'état dit le reste, et l'Annexe 3 montre une réponse ainsi affichée ; le délai avant le premier mot, lui, dépend du traitement de la question, et la section 11.3 le mesure en production. Le banc de mesure a tourné dans les deux modes, avec et sans diffusion, sur les deux postes : les temps totaux sont les mêmes, le streaming n'ajoute aucun surcoût de calcul, il ne change que la perception (Annexe 2). Les deux machines, aux profils différents, donnent les mêmes ordres de grandeur, et la génération domine largement le reste : c'est elle que le matériel du serveur déterminera. Un cas reste à part : le tout premier appel après démarrage, qui paie le chargement des modèles, plus d'une minute avant le premier mot. Le maintien des modèles en mémoire relève du déploiement, et la section 11.3 y reviendra.

L'interface a aussi fait évoluer la consigne. La V3 de la section 10.5 demandait de citer les pages officielles en fin de réponse ; la barre latérale rend cette liste redondante, l'application l'affichant déjà d'elle-même. La consigne V5, deux révisions plus loin, la retire donc, en conservant la reprise des liens utiles dans le corps du texte, guichet en ligne ou formulaire. C'est une révision assumée d'une décision de l'étape 2 : la bonne réponse dépend aussi de l'écran qui la porte. Effet mesuré au passage : des réponses plus courtes, donc un peu plus vite écrites (Annexe 2).

Il reste à sortir l'assistant du poste de développement : le déploiement.

11.3 Le déploiement

Le prototype quitte le poste de développement pour un serveur loué en Suisse : un VPS Infomaniak au palier de confort arrêté à la section 10.4, 18 Go de mémoire et six processeurs virtuels, sous Ubuntu, dans un centre de données suisse. Pas de carte graphique : l'inférence reste sur processeur. La souveraineté s'y lit de bout en bout, les questions des usagers sont traitées sur cette machine et n'en sortent pas, aucun service d'intelligence artificielle externe n'est appelé. Une nuance mérite d'être posée : le modèle exécuté, Gemma, reste un modèle ouvert d'origine américaine ; la souveraineté

⁷⁵ Streaming : diffusion de la réponse au fil de sa génération, mot à mot, au lieu d'un envoi en un seul bloc à la fin.

revendiquée porte sur l'exécution et sur les données, pas sur la nationalité des poids (section 8.3).

La sécurisation précède tout service, dans les usages établis : un compte d'administration dédié, l'accès par clé SSH seulement et la connexion du superutilisateur (root) désactivée, un pare-feu sur la machine doublé de celui de l'hébergeur, qui n'ouvre que l'administration et les ports du web, et un outil qui bannit les adresses aux essais répétés. Rien d'exotique : le nécessaire, correctement fermé.

L'assemblage est conteneurisé⁷⁶ avec Docker Compose : quatre services décrits dans un fichier versionné (*deploy/docker-compose.yml*), qu'une seule commande construit et relance. Le service ollama sert les modèles, gardés résidents en mémoire (*keep_alive*) pour que le chargement ne frappe qu'au démarrage du serveur ; ollama-pull, éphémère, télécharge les modèles à la première mise en route, leur liste vivant à un seul endroit de la configuration (*src/config.py*) ; app porte l'application, interface et pipeline, exécutée sans privilèges ; caddy, enfin, est le seul exposé au web. La base Chroma du chapitre 10 est copiée telle quelle sur le serveur puis montée dans l'application : c'est la base mesurée qui est servie, et rien ne se re-scrape depuis le VPS. Rien n'empêcherait pourtant de le faire : le dépôt entier y est déployé, scripts de découverte, de scraping et d'indexation compris, et étendre le corpus reviendrait à rejouer la chaîne du chapitre 10 puis à remonter la base. La résilience a été constatée en conditions réelles : après un redémarrage, les services repartent seuls.

La mise en ligne tient la promesse de la section 9.6 : ge-trouve.ch pointe vers le serveur, Caddy obtient et renouvelle le certificat auprès de Let's Encrypt et redirige www vers l'adresse nue, le tout en quelques lignes versionnées (*deploy/Caddyfile*). Un détail dit bien le projet : il n'existe aucun fichier de secrets. La configuration se lit dans l'environnement ou retombe sur ses valeurs par défaut ; sans service externe, il n'y a ni clé ni jeton à cacher. La souveraineté simplifie jusqu'à l'exploitation.

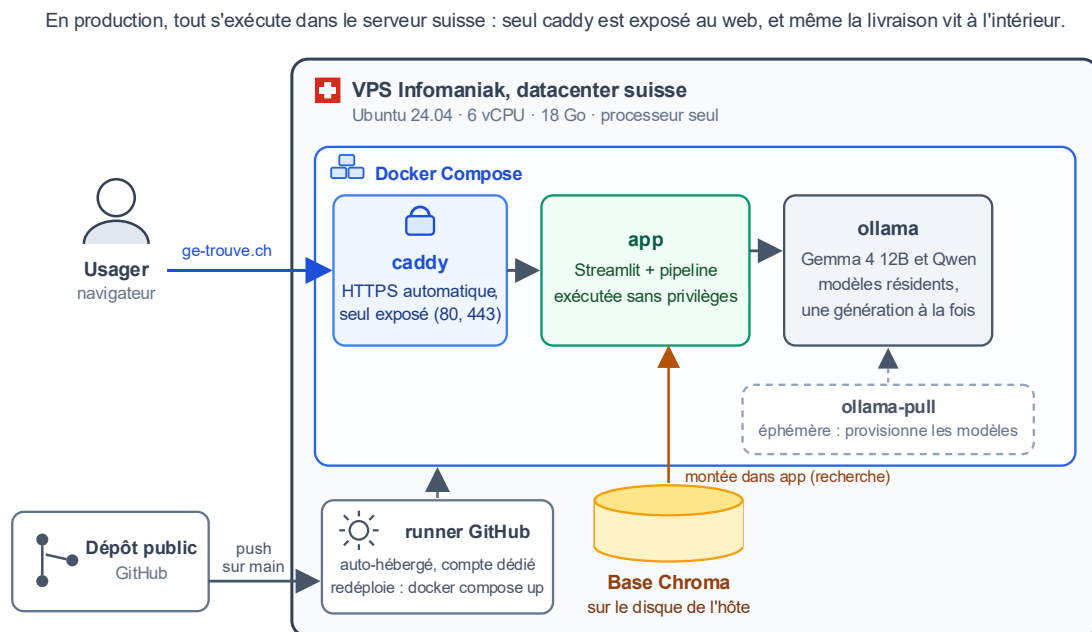
La livraison est continue⁷⁷ : à chaque poussée de code sur la branche principale, le dépôt public déclenche un déploiement (*.github/workflows/deploy.yml*), exécuté par un *runner* installé sur le VPS lui-même, sous un compte dédié sans privilèges. La chaîne de livraison reste ainsi sur le serveur, aucune infrastructure tierce n'y exécute quoi que ce

⁷⁶ Conteneur : environnement d'exécution isolé et reproductible, qui embarque une application et ses dépendances ; Docker Compose décrit et pilote un ensemble de conteneurs depuis un fichier unique.

⁷⁷ Livraison continue (CI/CD) : chaîne qui reconstruit et redéploie automatiquement l'application à chaque modification du code ; le runner est le programme qui exécute cette chaîne, ici hébergé sur le serveur lui-même.

soit, et les garde-fous du dépôt public sont posés : seule la branche principale déploie, jamais une contribution externe, et une modification purement documentaire ne déclenche rien. Pour un exploitant seul, publier une correction se résume à pousser le code. La Figure 11.1 rassemble cette architecture, de l'utilisateur au runner.

Figure 11.1 : L'architecture de production : tout s'exécute dans le serveur suisse



Aucun appel vers un service d'intelligence artificielle externe : questions et réponses ne quittent pas le serveur.

(élaboration personnelle)

Le déploiement a livré son enseignement le plus utile sur la mémoire. Les deux modèles résidents occupent environ 11 Go ; il a suffi d'une génération lancée pendant qu'une autre tournait pour saturer les 18 Go, le système coupant alors le serveur de modèles. La parade est un garde-fou assumé : une seule génération à la fois (OLLAMA_NUM_PARALLEL=1). Le palier de confort du chapitre 4 se révèle donc juste, au sens propre : suffisant pour le service visé, sans marge pour la concurrence ; un usage à plusieurs demanderait le palier supérieur, et la limite est assumée comme telle.

Reste la latence, la limite la plus visible, à nommer sans détour. En production, pour une question nouvelle, le premier mot apparaît après une minute et demie à deux minutes : le temps que le processeur digère le long prompt, consigne, extraits et question, avant d'écrire quoi que ce soit ; la réponse se déroule ensuite au fil de l'eau (section 11.2). Les mesures de développement donnaient une image plus flatteuse : rejouant la même question, elles profitaient d'un cache et mesuraient tout autre chose ; la mesure en conditions réelles corrige ce constat, dont le détail vit au journal des mesures (Annexe

2). L'origine est structurelle, un serveur modeste et sans carte graphique, le prix assumé de la souveraineté. Des pistes existent, un modèle plus léger, moins de fragments remontés, une fenêtre de contexte réduite ; elles restent des perspectives, car avant d'accélérer, encore faut-il savoir ce que valent les réponses : c'est l'affaire de la Partie 3.

Le service est ouvert : l'assistant répond sur ge-trouve.ch, entier sur sa machine suisse, du corpus au certificat. Le système est en place ; reste le verdict des mesures, l'évaluation.

12. Résultats de l'évaluation

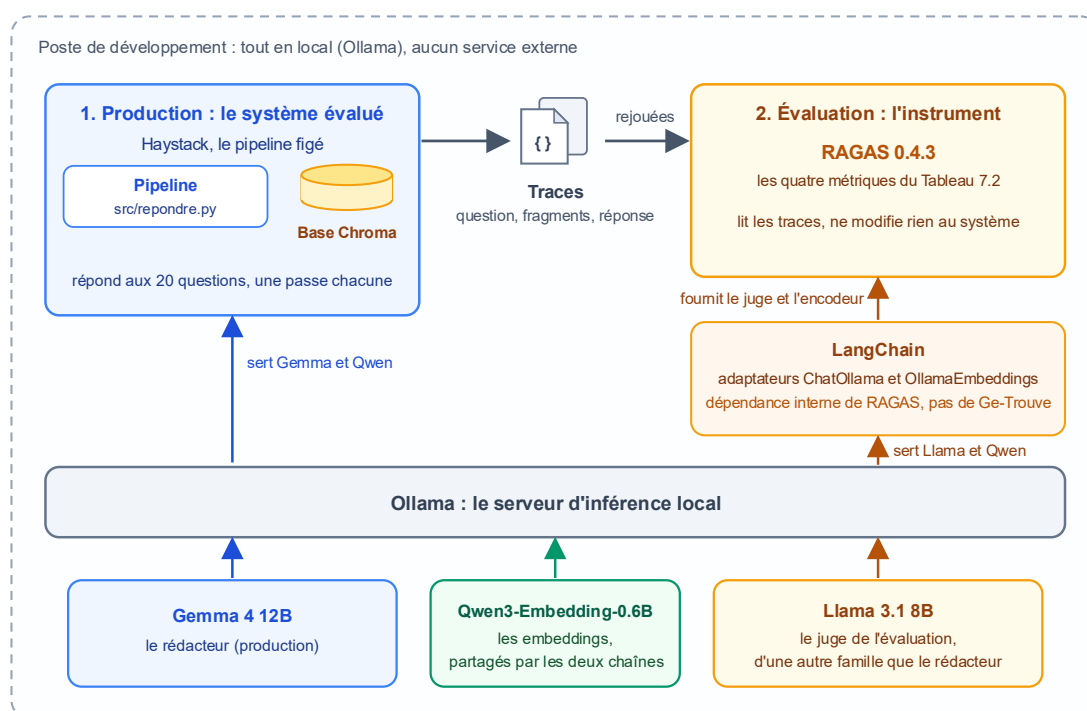
Le système a été mis à l'épreuve ; voici ce que les mesures en disent. Ce chapitre livre les résultats de l'évaluation : le protocole tel qu'il a été exécuté (12.1), ce que disent les scores (12.2), les cas limites et le garde-fou humain (12.3), puis le diagnostic et les leviers (12.4). La réponse à la question de recherche, elle, se construira en conclusion, sur ces résultats.

12.1 Le protocole exécuté

Le protocole du chapitre 7 a été exécuté sur le système tel que le chapitre 11 l'a laissé : pipeline de production figé, consigne V5, réglages actés, sans aucune retouche pendant toute l'évaluation. Le jeu compte vingt questions écrites à la main, en plusieurs registres, du courant au familier ; seize attendent une réponse du corpus, quatre sont volontairement hors périmètre, du ressort fédéral, d'un autre canton ou d'un service non couvert, pour éprouver l'aveu d'ignorance (exigence F5, Tableau 9.1). Chaque question porte sa réponse de référence, bâtie à la main depuis les pages brutes du corpus, jamais en interrogeant le système : l'étanchéité est totale, aucune de ces questions n'avait été posée à l'assistant avant le run officiel. Le jeu complet vit au dépôt (*evaluation/jeu_evaluation.py*) ; l'Annexe 4 en reprend les questions et les scores.

La notation suit le Tableau 7.2 : les quatre métriques de RAGAS, en version 0.4.3, calculées en local. Le juge est d'une autre famille que le rédacteur, Llama 3.1 8B, pour écarter l'auto-jugement ; l'encodeur d'évaluation reste Qwen. Le chapitre 3 notait que Haystack intègre RAGAS comme composant de pipeline. Ce branchement direct n'a pas pu être établi avec la version retenue de la bibliothèque. RAGAS a donc été enveloppé autrement, par ses composants LangChain pour Ollama. Le harnais reste ainsi une pièce à part, qui rejoue les traces du pipeline ; la Figure 12.1 en schématise le câblage. Ce critère d'intégration n'aura donc pas servi. La compatibilité de fond, elle, a tenu : RAGAS reçoit la question, les fragments et la réponse, d'où qu'ils viennent. Aucun service externe, rien ne quitte la machine.

Figure 12.1 : Le câblage de l'évaluation



(élaboration personnelle)

Chaque question a été posée une seule fois. C'est la production qui est mesurée, pas un réglage de laboratoire. En parallèle, un contrôle humain a relu cinq réponses, un quart du jeu, à l'aveugle, avant toute lecture des scores, selon les cinq critères du protocole ; les défauts relevés sont classés sur les sept points de défaillance de Barnett et al. (Tableau 7.1). Trois instruments, trois angles : la note automatique (avec RAGAS), le garde-fou humain et la grille de diagnostic (Barnett).

Le banc a tourné en une heure environ sur le poste de développement, autour de trois minutes par question selon la longueur des réponses, en simple relevé (Annexe 2). Les sections suivantes lisent les résultats : les scores d'abord, les cas limites ensuite, le diagnostic enfin.

12.2 Ce que disent les scores

Le Tableau 12.1 donne les moyennes des quatre métriques⁷⁸ sur les seize questions avec réponse. Les quatre hors périmètre ne s'y mêlangent pas : leurs scores se lisent à l'envers, un zéro y est un bon refus, et la section 12.3 les traite cas par cas. Le détail question par question vit à l'Annexe 4 , et l'Annexe 5 explique le calcul de chacune des quatre métriques.

⁷⁸ Chaque métrique s'exprime sur une échelle de 0 à 1, où 1 est la note parfaite.

Tableau 12.1 : Les moyennes des quatre métriques sur les questions avec réponse

Métrique	Ce qu'elle mesure	Moyenne
Précision du contexte	Les fragments remontés sont-ils utiles, et bien classés ?	0,92
Rappel du contexte	Les fragments couvrent-ils la réponse de référence ?	0,93
Fidélité	La réponse s'appuie-t-elle sur les fragments, sans inventer ?	0,85
Pertinence de la réponse	La réponse reste-t-elle sur la question posée ?	0,71

(élaboration personnelle)

La récupération est le point fort. Avec 0,92 de précision et 0,93 de rappel, le bon fragment est presque toujours trouvé, et presque toujours bien classé. La chaîne de recherche construite aux chapitres 5 et 6 tient donc son rôle sur un corpus administratif réel. Une réserve d'honnêteté : le rappel est agrégé sur quinze questions, la seizième ayant rencontré un échec technique du juge, assumé et consigné (Annexe 4).

La fidélité suit, à 0,85. Les écarts à la note pleine partagent une même signature : le modèle fusionne des passages voisins du corpus, il n'invente rien de toutes pièces. La section 12.3 le montrera sur pièces. Pour un assistant administratif, la nuance compte : une erreur par fusion se corrige en ouvrant la page citée, une invention pure tromperait sans laisser de prise.

La pertinence ferme la marche, à 0,71. Le mécanisme de la métrique explique l'écart : elle reconstruit des questions à partir de la réponse et les compare à la question posée. Or l'assistant répond juste, mais long : il déborde la question, couvre les cas voisins, et cette verbosité éloigne la réponse de ce qui était demandé. Le défaut est réel pour un usager pressé. Il est aussi le plus accessible des leviers, la section 12.4 y revient.

12.3 Les hors-périmètre et le garde-fou humain

Les quatre questions hors périmètre éprouvaient l'exigence F5 (Tableau 9.1) : savoir dire « je ne sais pas », et orienter. Trois refus sont nets. Pour le permis vaudois, les horaires

des TPG et le visa de travail à l'étranger, précision, fidélité et pertinence tombent à zéro ou presque : l'assistant n'a rien fabriqué, il a reconnu la limite de son corpus. Le quatrième cas déraile. Interrogé sur la future rente AVS, l'assistant répond comme si la question était couverte : il puise dans des pages cantonales voisines et les fusionne, jusqu'à une règle des trois mois qui n'a rien à y faire, au lieu d'avouer que la matière est fédérale. L'aveu d'ignorance est donc inconstant. Il tient quand le corpus est muet sur le thème ; il cède quand des pages proches offrent de quoi répondre à côté.

Le contrôle humain a relu cinq réponses à l'aveugle, tirées au hasard : les questions 1, 5, 8, 14 et 17. La grille remplie vit au dépôt (*resultats/evaluation/controle_humain.md*). Quatre relectures confirment la note automatique. La réponse sur le subsidie est sans défaut. Celle sur la naturalisation est correcte et sourcée, mais incomplète, avec une fusion, un permis Ci glissé où il n'a pas sa place. Celle sur les coordonnées de l'OCPM est exacte et n'invente rien, mais tait une absence, les horaires, sans l'avouer. Et la rente AVS confirme le constat du paragraphe précédent.

La cinquième relecture attrape ce que le juge n'a pas vu. La question 8 demandait les démarches après une naissance. Ses scores sont excellents, récupération et fidélité à 1,00 : la réponse s'appuie bien sur de vrais fragments du corpus. Elle est pourtant hors sujet, la section réellement utile de la page n'ayant pas été extraite. RAGAS mesure la cohérence entre la réponse et ses fragments, pas l'adéquation au besoin de l'utilisateur : c'est son angle mort. Une relecture humaine d'un quart du jeu suffit à le couvrir. Le garde-fou prévu au chapitre 7 a donc joué son rôle, ni plus ni moins : il confirme la tendance des scores, et il rattrape le cas que la machine ne pouvait pas voir. La section suivante classe ces défauts et en tire les leviers.

12.4 Le diagnostic et les leviers

Les défauts observés au contrôle humain se classent sur la grille du Tableau 7.1. Quatre des sept points sont touchés. Le contenu absent (point 1) apparaît deux fois : les horaires de l'OCPM manquent au corpus et la question 14 les tait sans l'avouer ; la matière AVS n'y est pas et la question 17 répond quand même. La spécificité inadaptée (point 6) s'ajoute pour cette même question 17, qui vise à côté du besoin. La non-extraction (point 4) porte la question 8 : la réponse utile était dans la page, le modèle ne l'en a pas tirée. L'incomplète (point 7) décrit la question 1, juste mais partielle, avec sa fusion. Trois points ne sont jamais apparus : les bons documents mal classés, les perdus à l'assemblage et le mauvais format. La récupération forte explique les deux premiers, la consigne le troisième. Et le constat transversal demeure : aucune invention pure, les erreurs factuelles sont des fusions de passages voisins.

Les leviers découlent du diagnostic, et leur exploration appartient à la conclusion. Le premier est la consigne : demander des réponses plus courtes attaque la verbosité, donc la pertinence, et allège la latence du même geste. Le deuxième est le réglage : les paramètres d'échantillonnage sont centralisés dans la configuration (*src/config.py*), prêts à l'essai. Le troisième est le modèle lui-même : en changer tient en une ligne, et la chaîne de livraison du chapitre 11 remet le tout en production d'un push ; essayer coûte peu. Le dernier est le matériel, un palier de serveur supérieur pour desserrer la latence. Un point ne se règle pas, il se garde : le contrôle humain, seul à voir l'angle mort du juge.

Le système répond juste, un peu long, et il sait presque toujours avouer quand la question sort de son périmètre : les mesures l'ont établi, chiffres à l'appui. Ce que ce verdict répond à la question de recherche, la conclusion le dira.

13. Conclusion

Ce travail est parti d'un constat simple, celui-là même de sa proposition initiale : l'administration publie tout, mais l'accès reste une épreuve pour qui ne manie pas le web, personnes âgées, nouveaux arrivants, usagers peu familiers du vocabulaire des institutions. Quatre objectifs en découlaient : concevoir, développer et déployer un prototype fonctionnel ; définir des critères de qualité et les mesurer ; en analyser les limites et formuler des recommandations ; et, au-dessus de tout, répondre à la question de recherche : « Dans quelle mesure un système RAG appliqué à des documents administratifs en français produit-il des réponses fiables et pertinentes pour des utilisateurs non-experts ? » Le prototype existe et répond sur ge-trouve.ch. L'évaluation a livré ses chiffres. La réponse peut donc se donner, mesure en main.

Elle est graduée, comme la question le demande. La fiabilité, d'abord, se joue en deux temps. La récupération est le point fort : 0,92 de précision, 0,93 de rappel ; sur un corpus réel de plus de 4 300 pages officielles, le bon fragment est presque toujours trouvé et bien classé, et la promesse du RAG tient en français administratif. La fidélité suit, à 0,85, avec le constat qui compte le plus : aucune invention pure sur les vingt questions. Les erreurs sont des fusions de passages voisins, corrigéables en ouvrant la page citée ; le risque d'hallucination, jugé disqualifiant pour un service public à la section 2.2, est contenu. La pertinence, à 0,71, est le point faible : des réponses justes mais longues, qui débordent la question posée. Le défaut est réel pour un usager pressé ; il est expliqué, et actionnable. L'aveu d'ignorance, enfin, tient trois fois sur quatre, et cède quand des pages voisines offrent de quoi répondre à côté. Le verdict s'écrit alors ainsi : dans le périmètre couvert, le système produit des réponses fiables, sourcées et majoritairement pertinentes pour un non-expert. La mesure est substantielle, pas totale ; ses défauts sont nommés, classés et corrigéables.

Les chiffres ne voient pas tout, et c'est un résultat en soi. La question sur la naissance affichait des scores excellents pour une réponse hors sujet ; la fusion du permis Ci a passé la fidélité à note pleine. L'évaluation automatique mesure la cohérence entre la réponse et ses fragments, pas l'adéquation au besoin réel : elle fonctionne en local, à condition d'un garde-fou humain. Les limites du dispositif restent celles d'un prototype, vingt questions, un juge de huit milliards de paramètres dont les notes valent comme instrument de comparaison, une seule passe ; le cadre est posé pour faire mieux.

Les recommandations découlent du diagnostic du chapitre 12. La première concerne la consigne : demander des réponses plus courtes devrait améliorer la pertinence et

réduire la latence en même temps. La deuxième concerne l'aveu d'ignorance : les métadonnées déjà stockées en base, la source et la section des pages, peuvent servir à restreindre la recherche, pour que les questions frontalières comme la rente AVS reçoivent un refus au lieu d'une réponse à côté. Les suivantes reprennent la question laissée ouverte à la section 2.5 : le pipeline simple a fait ses preuves, et il est maintenant justifié de tester des techniques Advanced ciblées, un re-classement des fragments contre la non-extraction, une reformulation de requête pour les questions vagues ; l'architecture Modular reste une étape ultérieure. Enfin, agrandir le jeu de questions et faire relire les réponses par d'autres personnes que l'auteur rendrait la mesure plus solide.

Sur le plan technique, plusieurs pistes existent pour réduire la latence. Le système a été construit de manière modulaire grâce au framework : changer de modèle tient en une ligne de configuration, et le déploiement continu met la modification en production après un simple push du code sur le dépôt. Tester un modèle plus léger, comme Llama 3.1 8B, coûte donc peu, et le protocole d'évaluation peut être rejoué à l'identique pour vérifier ce que la qualité y perdrait. Les réglages du pipeline peuvent aussi être ajustés : la taille des fragments, le nombre de fragments remontés, les plafonds de jetons et de contexte, tous notés comme révisables au Tableau 10.1. Reste le matériel : un palier de serveur supérieur accélérerait la génération, mais la marge est limitée, un serveur avec carte graphique dépassant le budget d'un projet de ce type. En attendant, le streaming rend l'attente supportable.

Plusieurs extensions sont ensuite possibles. Le corpus d'abord : ajouter les documents PDF et les formulaires, les textes du recueil législatif gardés en réserve, puis les démarches d'autres communes. Le cas fribourgeois montre qu'un canton a déjà investi dans un service de ce type, et qu'aucun ne couvre Genève. L'accueil d'autres langues est envisageable, le modèle retenu étant multilingue. Une administration pourrait enfin reprendre le projet : le code, les journaux et le protocole d'évaluation sont publics. L'apport annoncé au chapitre 8 est ainsi réalisé, à l'échelle d'un prototype : une évaluation documentée et reproductible d'un assistant RAG sur les démarches administratives genevoises, en français.

Les quatre objectifs de la proposition de sujet sont donc atteints. Ce travail s'achève, mais le projet continue : ge-trouve.ch reste en ligne, et j'ai l'intention de le maintenir et de l'améliorer en suivant les pistes listées ci-dessus, le dépôt public documentant la suite. L'objectif de départ reste inchangé : rendre l'information administrative accessible à celles et ceux que la recherche en ligne rebute.

Bibliographie

- AGENCE FRANCE-PRESSE, 2026. Albert, l'outil d'IA générative, ne sera pas généralisé. *weka.fr* [en ligne]. 12 janvier 2026. Disponible à l'adresse : <https://www.weka.fr/actualite/administration/article/albert-l-outil-d-ia-generative-experimente-a-france-services-ne-sera-pas-generalise-209194/> [consulté le 28 août 2026].
- BAAI, 2024. *bge-m3* [logiciel] [en ligne]. 2024. [consulté le 20 juillet 2026]. Disponible à l'adresse : <https://huggingface.co/BAAI/bge-m3> [consulté le 20 juillet 2026].
- BARBARESI, Adrien, 2026. *adbar/trafilatura* [logiciel]. Version 2.2.0. 10 août 2026. [consulté le 10 août 2026]. Disponible à l'adresse : <https://github.com/adbar/trafilatura> [consulté le 10 août 2026].
- BARNETT, Scott et al., 2024. *Seven Failure Points When Engineering a Retrieval Augmented Generation System*. arXiv:2401.05856. arXiv. arXiv:2401.05856. DOI 10.48550/arXiv.2401.05856. Publié dans Proceedings of CAIN 2024, ACM. DOI 10.1145/3644815.3644945
- BORGEAUD, Sebastian et al., 2021. *Improving language models by retrieving from trillions of tokens*. arXiv.2112.04426. arXiv. arXiv.2112.04426. DOI 10.48550/arXiv.2112.04426.
- BUSSLINGER, Boris, 2024. Des chatbots «sauvages» créés pour les villes suisses. *Le Temps* [en ligne]. 5 janvier 2024. Disponible à l'adresse : <https://www.letemps.ch/cyber/des-chatbots-sauvages-crees-pour-les-villes-suissees> [consulté le 28 août 2026].
- CADDY, 2026. *caddyserver/caddy* [logiciel]. Version 2.11.4. 6 août 2026. ZeroSSL. [consulté le 6 août 2026]. Disponible à l'adresse : <https://github.com/caddyserver/caddy> [consulté le 6 août 2026].
- CHANCELLERIE DE L'ÉTAT DE FRIBOURG, 2026. Nouveau chatbot IA de fr.ch : faciliter l'accès à l'information publique. *fr.ch* [en ligne]. 13 mars 2026. Disponible à l'adresse : <https://www.fr.ch/cha/actualites/nouveau-chatbot-ia-de-frch-faciliter-lacces-a-linformation-publique> [consulté le 28 août 2026].
- CHANCELLERIE FÉDÉRALE, s.d. Intelligence artificielle (utilisation dans l'administration fédérale). *bk.admin.ch* [en ligne]. s.d. Disponible à l'adresse : <https://www.bk.admin.ch/fr/intelligence-artificielle> [consulté le 11 juillet 2026].
- CHASE, Harrison, 2022. *LangChain* [logiciel]. Version 1.3.13. octobre 2022. LangChain, Inc. [consulté le 13 juillet 2026]. Disponible à l'adresse : <https://github.com/langchain-ai/langchain> [consulté le 13 juillet 2026].
- CHAVANNE, Yannick, 2024. Cinq chatbots de l'administration fédérale. *ICTJournal* [en ligne]. 17 octobre 2024. Disponible à l'adresse : <https://www.ictjournal.ch/articles/2024-10-17/cinq-chatbots-de-ladministration-federale> [consulté le 19 mars 2026].
- CHEN, Jianlv et al., 2024. *M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. arXiv:2402.03216. arXiv. arXiv:2402.03216. DOI 10.48550/arXiv.2402.03216. arXiv:2402.03216 [cs.CL]
- CHIRKOVA, Nadezhda et al., 2024. *Retrieval-augmented generation in multilingual settings*. arXiv:2407.01463. arXiv. arXiv:2407.01463. DOI 10.48550/arXiv.2407.01463. arXiv:2407.01463 [cs.CL]
- CHROMA, 2026a. *chroma-core/chroma* [logiciel] [en ligne]. 1 août 2026. Chroma. [consulté le 1 août 2026]. Disponible à l'adresse : <https://github.com/chroma-core/chroma> [consulté le 1 août 2026].
- CHROMA, 2026b. Introduction. *Chroma Docs* [en ligne]. 2026. Disponible à l'adresse : <https://docs.trychroma.com/docs/overview/introduction> [consulté le 1 août 2026].
- CIANCONE, Mathieu et al., 2024. *MTEB-French: Resources for French Sentence Embedding Evaluation and Analysis*. arXiv:2405.20468. arXiv. arXiv:2405.20468. DOI 10.48550/arXiv.2405.20468. arXiv:2405.20468 [cs.CL]

COMMISSION DE L'INTELLIGENCE ARTIFICIELLE, 2024. *IA : notre ambition pour la France* [en ligne]. Paris : Gouvernement français. Disponible à l'adresse : <https://www.info.gouv.fr/upload/media/content/0001/09/4d3cc456dd2f5b9d79ee75feea63b47f10d75158.pdf> [consulté le 4 août 2026]. Rapport remis au président de la République le 13 mars 2024

COMPAR:IA, 2026. Classement - compar:IA, le comparateur d'IA conversationnelles. *compar:IA* [en ligne]. 2026. Disponible à l'adresse : <https://comparia.beta.gouv.fr/ranking> [consulté le 17 juillet 2026].

DEEPSET, 2026a. FAISS. *Haystack* [en ligne]. 2026. Disponible à l'adresse : <https://haystack.deepset.ai/integrations/faiss/> [consulté le 1 août 2026].

DEEPSET, 2026b. Qdrant. *Haystack* [en ligne]. 2026. Disponible à l'adresse : <https://haystack.deepset.ai/integrations/qdrant-document-store/> [consulté le 1 août 2026].

DEEPSET, 2026c. Chroma. *Haystack* [en ligne]. 2026. Disponible à l'adresse : <https://haystack.deepset.ai/integrations/chroma-documentstore/> [consulté le 1 août 2026].

DÉNERVAUD, Thomas, 2026. Un chatbot IA souverain pour l'Etat de Fribourg. *Liip* [en ligne]. 15 juin 2026. Disponible à l'adresse : <https://www.liip.ch/fr/blog/un-chatbot-ia-souverain-pour-l-etat-de-fribourg> [consulté le 28 août 2026].

DEVLIN, Jacob et al., 2018. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv:1810.04805. arXiv. arXiv:1810.04805. DOI 10.48550/arXiv.1810.04805. arXiv:1810.04805 [cs.CL]

DOUZE, Matthijs et al., 2024. *The Faiss library*. arXiv:2401.08281. arXiv. arXiv:2401.08281. DOI 10.48550/arXiv.2401.08281. arXiv:2401.08281 [cs.LG]

ENEVOLDSEN, Kenneth et al., 2025. *MMTEB: Massive Multilingual Text Embedding Benchmark*. arXiv:2502.13595. arXiv. arXiv:2502.13595. DOI 10.48550/arXiv.2502.13595. arXiv:2502.13595 [cs.CL]

ES, Shahul et al., 2023. *Ragas: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217. arXiv. arXiv:2309.15217. DOI 10.48550/arXiv.2309.15217. arXiv:2309.15217 [cs]

ÉTAT DE GENÈVE, 2024. Intelligence artificielle générative dans la fonction publique: l'Etat de Genève pose un cadre. *ge.ch* [en ligne]. 21 novembre 2024. Disponible à l'adresse : <https://www.ge.ch/document/intelligence-artificielle-generative-dans-fonction-publique-etat-geneve-pose-cadre> [consulté le 11 juillet 2026].

ÉTAT DE GENÈVE, 2026a. Genève développe sa stratégie en matière de souveraineté numérique. *ge.ch* [en ligne]. 7 juillet 2026. Disponible à l'adresse : <https://www.ge.ch/document/geneve-developpe-sa-strategie-matiere-souverainete-numerique> [consulté le 10 juillet 2026].

ÉTAT DE GENÈVE, 2026b. Intelligence artificielle : Genève mise sur la collaboration. *ge.ch* [en ligne]. 25 août 2026. Disponible à l'adresse : <https://www.ge.ch/node/44214> [consulté le 28 août 2026].

ÉTAT DE GENÈVE (SITG), 2026. Explorer les données du territoire à l'aide d'un assistant conversationnel. *sitg.ge.ch* [en ligne]. 27 avril 2026. Disponible à l'adresse : <https://sitg.ge.ch/actualites/experimentation-dun-assistant-conversationnel-pour-explorer-les-donnees-du-territoire> [consulté le 28 août 2026].

FRANCE 24, 2026. Les États-Unis lèvent leurs restrictions sur les modèles d'IA les plus puissants d'Anthropic. *France 24* [en ligne]. 1 juillet 2026. Disponible à l'adresse : <https://www.france24.com/fr/am%C3%A9riques/20260701-etats-unis-levent-restrictions-modeles-intelligence-artificielle-puissants-anthropic-commerce-claude-mythos-s%C3%A9curite-nationale> [consulté le 17 juillet 2026].

GAO, Yunfan et al., 2023. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997. arXiv. arXiv:2312.10997. DOI 10.48550/arXiv.2312.10997. arXiv:2312.10997 [cs]

GARCIA, Daniel, 2026. *DanielGarcia85/ge-trouve-ch* [logiciel] [en ligne]. 6 août 2026. [consulté le 6 août 2026]. Disponible à l'adresse : <https://github.com/DanielGarcia85/ge-trouve-ch> [consulté le 6 août 2026].

GEMMA TEAM et al., 2026. *Gemma 4 Technical Report*. arXiv:2607.02770. arXiv. arXiv:2607.02770. DOI 10.48550/arXiv.2607.02770. arXiv:2607.02770 [cs.CL]

GLM-5-TEAM et al., 2026. *GLM-5: from Vibe Coding to Agentic Engineering*. arXiv:2602.15763. arXiv. arXiv:2602.15763. DOI 10.48550/arXiv.2602.15763. arXiv:2602.15763 [cs.LG]

GOOGLE, 2025. *EmbeddingGemma 300m* [logiciel] [en ligne]. 2025. [consulté le 20 juillet 2026]. Disponible à l'adresse : <https://huggingface.co/google/embeddinggemma-300m> [consulté le 20 juillet 2026].

GOUVERNEMENT FRANÇAIS, 2024. IA : connaissez-vous Albert ? *info.gouv.fr* [en ligne]. 22 mai 2024. Disponible à l'adresse : <https://www.info.gouv.fr/actualite/ia-connaissiez-vous-albert> [consulté le 28 août 2026].

GOUVERT, Olivier et al., 2025. *The Lucie-7B LLM and the Lucie Training Dataset: Open resources for multilingual language generation*. arXiv:2503.12294. arXiv. arXiv:2503.12294. DOI 10.48550/arXiv.2503.12294. arXiv:2503.12294 [cs.CL]

GRATTAFIORI, Aaron et al., 2024. *The Llama 3 Herd of Models*. arXiv:2407.21783. arXiv. arXiv:2407.21783. DOI 10.48550/arXiv.2407.21783. arXiv:2407.21783 [cs.AI]

GUU, Kelvin et al., 2020. *REALM: Retrieval-Augmented Language Model Pre-Training*. arXiv.2002.08909. arXiv. arXiv.2002.08909. DOI 10.48550/arXiv.2002.08909.

HASAN, Md Toufique et al., 2025. *Engineering RAG Systems for Real-World Applications: Design, Development, and Evaluation*. arXiv.2506.20869. arXiv. arXiv.2506.20869. DOI 10.48550/arXiv.2506.20869. arXiv:2506.20869 [cs.SE]

Ji, Ziwei et al., 2022. *Survey of Hallucination in Natural Language Generation*. arXiv. DOI 10.48550/arXiv.2202.03629. DOI: 10.1145/3571730

KARPUKHIN, Vladimir et al., 2020. *Dense Passage Retrieval for Open-Domain Question Answering*. arXiv:2004.04906. arXiv. arXiv:2004.04906. DOI 10.48550/arXiv.2004.04906. arXiv:2004.04906 [cs.CL]

LEWIS, Mike et al., 2019. *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. arXiv.1910.13461. arXiv. arXiv.1910.13461. DOI 10.48550/arXiv.1910.13461.

LEWIS, Patrick et al., 2020. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401. arXiv. arXiv:2005.11401. DOI 10.48550/arXiv.2005.11401. arXiv:2005.11401 [cs]

LINAGORA, 2025. *Openrag* [logiciel]. Version 2.0.0. juillet 2025. Linagora. [consulté le 14 juillet 2026]. Disponible à l'adresse : <https://github.com/linagora/openrag> [consulté le 14 juillet 2026].

LIU, Alexander H. et al., 2026. *Minstral 3*. arXiv:2601.08584. arXiv. arXiv:2601.08584. DOI 10.48550/arXiv.2601.08584. arXiv:2601.08584 [cs.CL]

LIU, Jerry, 2022. *LlamaIndex* [logiciel]. Version 0.14.23. novembre 2022. LlamaIndex, Inc. DOI 10.5281/zenodo.1234.

LOKMANE-KHELIL, Samia, 2025. La Belgique encadre le déploiement de l'IA dans les services publics. *acteurspublics.fr* [en ligne]. 8 septembre 2025. Disponible à l'adresse : <https://acteurspublics.fr/articles/la-belgique-encadre-le-deploiement-de-lia-dans-les-services-publics/> [consulté le 28 août 2026].

MALKOV, Yu A. et YASHUNIN, D. A., 2016. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. arXiv:1603.09320. arXiv. arXiv:1603.09320. DOI 10.48550/arXiv.1603.09320. arXiv:1603.09320 [cs.DS]

META, 2024. *Llama 3.1 8B Instruct* [logiciel]. Version 3.1. juillet 2024. Meta. [consulté le 16 juillet 2026]. Disponible à l'adresse : <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct> [consulté le 16 juillet 2026].

META AI, 2026. *facebookresearch/faiss* [logiciel] [en ligne]. 1 août 2026. Meta Research. [consulté le 1 août 2026]. Disponible à l'adresse : <https://github.com/facebookresearch/faiss> [consulté le 1 août 2026].

MISTRAL AI, 2025. *Minstral 3 8B Instruct* [logiciel]. Version 2512. décembre 2025. Mistral AI. [consulté le 16 juillet 2026]. Disponible à l'adresse : <https://huggingface.co/mistralai/Minstral-3-8B-Instruct-2512> [consulté le 16 juillet 2026].

MTEB, 2026a. Multilingual · MTEB Leaderboard. *MTEB Leaderboard* [en ligne]. 20 juillet 2026. Disponible à l'adresse : [https://mteb-leaderboard.hf.space/benchmark/MTEB\(Multilingual%2C%20v2\)](https://mteb-leaderboard.hf.space/benchmark/MTEB(Multilingual%2C%20v2)) [consulté le 20 juillet 2026].

MTEB, 2026b. French · MTEB Leaderboard. *MTEB Leaderboard* [en ligne]. 20 juillet 2026. Disponible à l'adresse : [https://mteb-leaderboard.hf.space/benchmark/MTEB\(fra%2C%20v1\)](https://mteb-leaderboard.hf.space/benchmark/MTEB(fra%2C%20v1)) [consulté le 20 juillet 2026].

MUENNIGHOFF, Niklas et al., 2022. *MTEB: Massive Text Embedding Benchmark*. arXiv:2210.07316. arXiv. arXiv:2210.07316. DOI 10.48550/arXiv.2210.07316. arXiv:2210.07316 [cs.CL]

NGINX, 2026. *nginx/nginx* [logiciel]. Version 1.31.3. 6 août 2026. F5. [consulté le 6 août 2026]. Disponible à l'adresse : <https://github.com/nginx/nginx> [consulté le 6 août 2026].

OLLAMA, 2024a. llama3.1. *Ollama* [en ligne]. 2024. Disponible à l'adresse : <https://ollama.com/library/llama3.1> [consulté le 17 juillet 2026].

OLLAMA, 2024b. bge-m3. *Ollama* [en ligne]. 2024. Disponible à l'adresse : <https://ollama.com/library/bge-m3> [consulté le 20 juillet 2026].

OLLAMA, 2025. minstral-3. *Ollama* [en ligne]. 2025. Disponible à l'adresse : <https://ollama.com/library/minstral-3> [consulté le 18 juillet 2026].

OLLAMA, 2026a. qwen3.5. *Ollama* [en ligne]. 2026. Disponible à l'adresse : <https://ollama.com/library/qwen3.5> [consulté le 18 juillet 2026].

OLLAMA, 2026b. gemma4. *Ollama* [en ligne]. 2026. Disponible à l'adresse : <https://ollama.com/library/gemma4> [consulté le 18 juillet 2026].

OPEN WEBUI, 2026. *open-webui/open-webui* [logiciel]. Version 0.11.0. 6 août 2026. Open WebUI. [consulté le 6 août 2026]. Disponible à l'adresse : <https://github.com/open-webui/open-webui> [consulté le 6 août 2026].

OPENAI et al., 2025. *gpt-oss-120b & gpt-oss-20b Model Card*. arXiv:2508.10925. arXiv. arXiv:2508.10925. DOI 10.48550/arXiv.2508.10925. arXiv:2508.10925 [cs.CL]

OPENLLM-FRANCE, 2025. OpenLLM-France/Lucie-7B-Instruct. *Ollama* [en ligne]. 2025. Disponible à l'adresse : <https://ollama.com/OpenLLM-France/Lucie-7B-Instruct> [consulté le 17 juillet 2026].

PAPADOPOULOS, Theodoros, ALEXOPOULOS, Charalampos Harris et CHARALABIDIS, Yannis, 2025. Evaluating chatbot architectures for public service delivery: balancing functionality, safety, ethics, and adaptability. *Frontiers in Political Science*. 1 octobre 2025. Vol. 7. DOI 10.3389/fpos.2025.1601440.

PIETSCH, Malte et al., 2019. *Haystack: the end-to-end NLP framework for pragmatic builders* [logiciel]. Version 2.31. novembre 2019. deepset GmbH. [consulté le 13 juillet 2026]. Disponible à l'adresse : <https://github.com/deepset-ai/haystack> [consulté le 13 juillet 2026].

PROJECT APERTUS et al., 2025. *Apertus: Democratizing Open and Compliant LLMs for Global Language Environments*. arXiv:2509.14233. arXiv. arXiv:2509.14233. DOI 10.48550/arXiv.2509.14233. arXiv:2509.14233 [cs.CL]

QDRANT, 2026a. *qdrant/qdrant* [logiciel] [en ligne]. 1 août 2026. Qdrant. [consulté le 1 août 2026]. Disponible à l'adresse : <https://github.com/qdrant/qdrant> [consulté le 1 août 2026].

QDRANT, 2026b. Qdrant Documentation. *Qdrant* [en ligne]. 2026. Disponible à l'adresse : <https://qdrant.tech/documentation/> [consulté le 1 août 2026].

QWEN TEAM, 2025. *Qwen3-Embedding-0.6B* [logiciel] [en ligne]. 2025. [consulté le 20 juillet 2026]. Disponible à l'adresse : <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B> [consulté le 20 juillet 2026].

QWEN TEAM, 2026. *Qwen3.5 9B Instruct* [logiciel] [en ligne]. mars 2026. Alibaba Cloud. [consulté le 16 juillet 2026]. Disponible à l'adresse : <https://huggingface.co/Qwen/Qwen3.5-9B> [consulté le 16 juillet 2026].

RAGAS, 2026a. *vibrantlabsai/ragas* [logiciel]. Version v0.4.3. 4 août 2026. Vibrant Labs. [consulté le 4 août 2026]. Disponible à l'adresse : <https://github.com/vibrantlabsai/ragas> [consulté le 4 août 2026].

RAGAS, 2026b. List of available metrics - Ragas. *Ragas* [en ligne]. 4 août 2026. Disponible à l'adresse : https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/ [consulté le 4 août 2026].

RAGAS, 2026c. Customise models - Ragas. *Ragas* [en ligne]. 4 août 2026. Disponible à l'adresse : https://docs.ragas.io/en/stable/howtos/customizations/customize_models/ [consulté le 4 août 2026].

ROSSELLI, Michel, 2026. Apertus. *Ollama* [en ligne]. 2026. Disponible à l'adresse : <https://ollama.com/MichelRosselli/apertus> [consulté le 16 juillet 2026].

STREAMLIT, 2026. *streamlit/streamlit* [logiciel]. Version 1.61.1. 6 août 2026. Streamlit. [consulté le 6 août 2026]. Disponible à l'adresse : <https://github.com/streamlit/streamlit> [consulté le 6 août 2026].

SWISS AI INITIATIVE, 2025. *Apertus 8B Instruct* [logiciel]. Version 2509. septembre 2025. Swiss AI Initiative. [consulté le 16 juillet 2026]. Disponible à l'adresse : <https://huggingface.co/swiss-ai/Apertus-8B-Instruct-2509> [consulté le 16 juillet 2026].

VASWANI, Ashish et al., 2017. *Attention Is All You Need*. arXiv:1706.03762. arXiv. arXiv:1706.03762. DOI 10.48550/arXiv.1706.03762. arXiv:1706.03762 [cs]

VERA, Henrique Schechter et al., 2025. *EmbeddingGemma: Powerful and Lightweight Text Representations*. arXiv:2509.20354. arXiv. arXiv:2509.20354. DOI 10.48550/arXiv.2509.20354. arXiv:2509.20354 [cs.CL]

WIKIPEDIA, 2026. Llama (language model). *Wikipedia* [en ligne]. Disponible à l'adresse : [https://en.wikipedia.org/w/index.php?title=Llama_\(language_model\)&oldid=1363576201](https://en.wikipedia.org/w/index.php?title=Llama_(language_model)&oldid=1363576201) [consulté le 16 juillet 2026]. Page Version ID: 1363576201

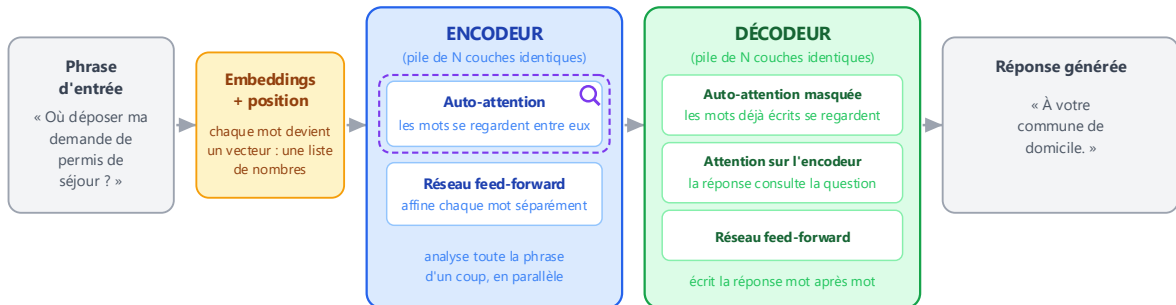
YANG, An et al., 2025. *Qwen3 Technical Report*. arXiv:2505.09388. arXiv. arXiv:2505.09388. DOI 10.48550/arXiv.2505.09388. arXiv:2505.09388 [cs.CL]

ZHANG, Yanzhao et al., 2025. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. arXiv:2506.05176. arXiv. arXiv:2506.05176. DOI 10.48550/arXiv.2506.05176. arXiv:2506.05176 [cs.CL]

ZHAO, Wayne Xin et al., 2023. *A Survey of Large Language Models*. arXiv:2303.18223. arXiv. arXiv:2303.18223. DOI 10.48550/arXiv.2303.18223. arXiv:2303.18223 [cs.CL]

Annexe 1 : Le Transformer – vue d'ensemble et mécanisme d'attention

1. Vue d'ensemble : de la question de l'utilisateur à la réponse générée

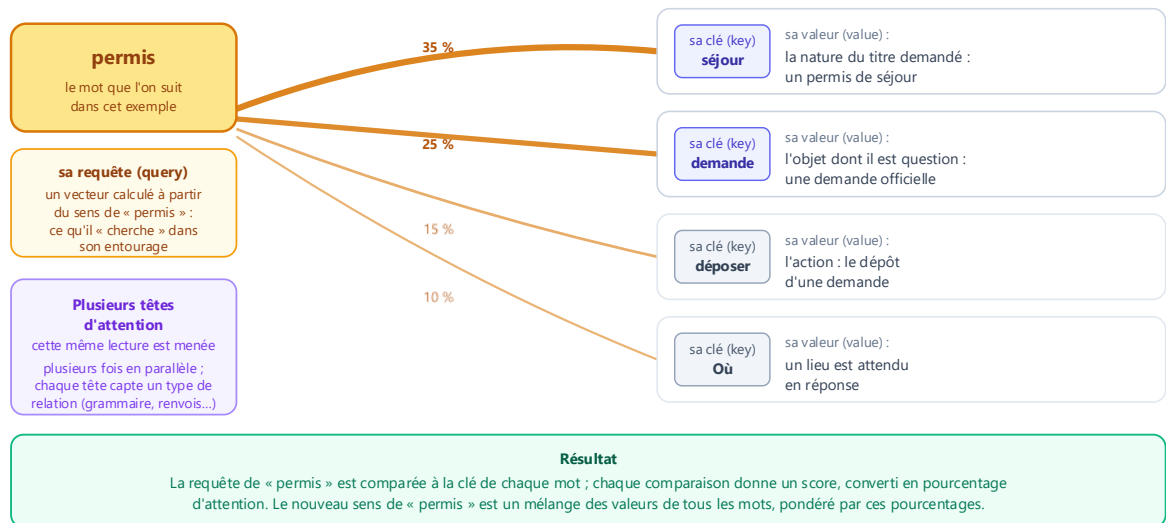


L'encodeur transforme la question en une représentation riche, où chaque mot a été précisé par ses voisins. Le décodeur s'appuie sur cette représentation pour écrire la réponse un mot à la fois : à chaque nouveau mot, il relit ce qu'il a déjà écrit (auto-attention masquée) et va consulter la question (attention sur l'encodeur).

Le cœur commun de ces blocs est le mécanisme d'attention, détaillé dans la zone 2.

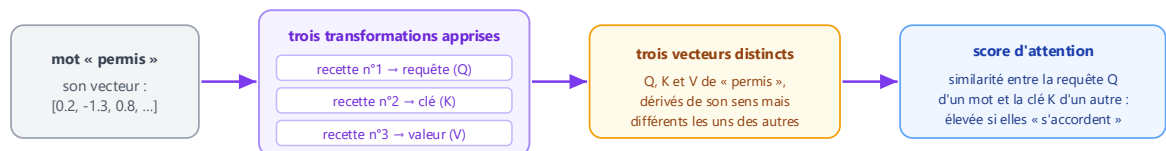
2. Zoom sur l'auto-attention : le mot « permis » interroge la phrase

Ce qui se passe dans le bloc « Auto-attention » entouré en pointillés ci-dessus. On suit ici la requête du mot « permis », mais chaque mot de la phrase fait exactement la même chose en parallèle, avec sa propre requête, sa clé et sa valeur.



3. D'où viennent la requête, la clé et la valeur ?

Elles ne sont pas écrites à la main : le modèle applique au vecteur de chaque mot trois transformations, identiques pour tous les mots, dont le contenu a été appris automatiquement pendant l'entraînement.



Comment ces recettes ont-elles été trouvées ?

Elles démarrent remplies de nombres aléatoires. Pendant l'entraînement, le modèle s'exerce des millions de fois à prédire des mots sur d'immenses corpus de texte ; chaque erreur ajuste légèrement les nombres. À la fin, les recettes ont convergé : les mots liés par la grammaire ou par le sens obtiennent naturellement des scores d'attention élevés entre eux. Personne n'a écrit ces règles : elles ont émergé de l'apprentissage. C'est le sens du mot « apprendre » en apprentissage automatique.

Annexe 2 : Mesures datées de la partie pratique

Cette annexe compile le journal des mesures du dépôt du projet (*docs/mesures/journal-des-mesures.md*) (Garcia 2026). Chaque session est datée et reproductible : scripts versionnés, paramètres consignés, un relevé par poste quand la mesure dépend du matériel. Seules des valeurs mesurées y figurent.

Les deux machines du projet, utilisées pour toutes les mesures :

Poste	Processeur	RAM	Système	Python	Ollama
DANIELGARCIA	AMD Ryzen 7 7840HS	32 Go DDR5 (27,7 disponibles, ~4 Go pour la puce graphique)	Windows 11 Pro	3.13.2	0.32.5
GARCIAD	Intel Core i7-12700	16 Go DDR4 (15,7 disponibles)	Windows 10 Entreprise LTSC	3.13.1	0.32.6

Paquets épinglés sur les deux postes :

- haystack-ai 3.0.0
- ollama-haystack 6.8.0
- chroma-haystack 4.4.0.

Modèles identiques, mêmes identifiants ; la taille indiquée est celle du fichier sur disque, l'occupation en mémoire une fois chargés figure aux tableaux de RAM :

Rôle	Paquet Ollama	Identifiant	Quantisation	Contexte	Taille disque
LLM rédacteur	gemma4:12b	4eb23ef187e2	Q4_K_M	262 144	7,6 Go
Repli et juge	llama3.1:8b	46e0c10c039e	Q4_K_M	131 072	4,9 Go
Embeddings	qwen3-embedding:0.6b	ac6da0dfba84	Q8_0	32 768	639 Mo

Étape 0 : l'environnement mesuré (6 août 2026)

Protocole : génération par l'API en flux (think:false, température 0, seed 42), une exécution à froid puis trois à chaud, médiane ; RAM relevée modèle par modèle, chaque modèle isolé.

Vitesse de génération des deux LLM, mesurée modèle par modèle sur chaque poste :

Modèle	Poste	Chargement à froid	Latence 1 ^{er} jeton	Débit
gemma4:12b	DANIELGARCIA	14,3 s	3,2 s	5,6 jetons/s
gemma4:12b	GARCIAD	22,5 s	1,2 s	3,8 jetons/s
llama3.1:8b	DANIELGARCIA	8,8 s	2,5 s	9,3 jetons/s
llama3.1:8b	GARCIAD	7,8 s	0,5 s	6,0 jetons/s

Les embeddings tiennent en une ligne : encodage d'une requête en ~0,2 s, indexation à 1,1-1,2 documents par seconde, vecteurs de 1 024 dimensions, identiques sur les deux postes. Côté mémoire, les empreintes chargées, rapportées par ollama ps, sont identiques partout : Gemma 8,9 Go, Llama 5,6 Go, Qwen 2,4 Go, soit un budget de production Gemma + Qwen d'environ 11,3 Go ; sur la machine à 16 Go, l'écart entre la hausse de RAM utilisée et l'empreinte tient probablement à la pagination d'une partie du modèle, et Gemma seul n'y laisse que 1,4 Go libre, un plancher extrême ; les deux LLM chargés ensemble saturent même le portable de 32 Go utiles. Observations : la puce graphique intégrée n'est pas prise en charge par Ollama sous Windows (inférence processeur, ~4 Go réservés sur le portable) ; Gemma réfléchit par défaut et le mode direct exige think:false, la réflexion allongeant la réponse d'environ 3 à environ 50 secondes ; le poste de bureau sort son premier jeton plus tôt mais décode à environ 65 % du portable, la vitesse d'accès à la mémoire bornant le débit ; en production, un seul LLM est servi.

Étape 1 : le pipeline pilote (6 et 7 août 2026)

Les artefacts (corpus, index, réponse) sont indépendants de la machine et produits une fois ; les mesures de performance sont relevées sur les deux postes.

Robots.txt de www.ge.ch: les 23 URL du manifeste (12 de départ, plus 11 sous-pages relevées sur les 6 pages chapeau) sont autorisées, aucun délai imposé, le projet applique le sien (2 secondes, agent GeTrouveBot). Scraping : 23 pages sur 23, textes de 556 à 7 379 caractères, ≈ 51 s dominées par le délai de politesse. Index Chroma : 72 fragments (200 mots, recouvrement 40), dimension 1 024, métadonnées complètes, identifiants par condensé du contenu (ré-exécution idempotente), base $\approx 2,7$ Mo ; débit 0,8 fragment/s, chargement à froid de Qwen inclus. Réponse : mode direct confirmé (champ de raisonnement vide) ; correcte et sourcée (dépôt auprès de l'OCPM, en ligne ou par courrier) mais incomplète, ni lien ni adresse, ces éléments étant absents du corpus pilote ; rien d'inventé.

Mesures du pipeline assemblé (*bench_pipeline.py*, question type, 1 à froid puis 3 à chaud, médiane) :

Grandeur	DANIELGARCIA	GARCIAD
Latence à chaud (médiane)	8,1 s (8,3 / 8,1 / 8,0)	7,8 s (8,5 / 7,8 / 7,7)
dont génération (25 jetons)	4,8 s	6,8 s
Latence à froid	73 s (Gemma : 13,7 s)	88 s (Gemma : 40 s)
RAM système sous pipeline	22,4 / 32 Go (outils de développement inclus)	15,4 / 16 Go (0,3 Go libre)

Empreinte des modèles identique : 11,3 Go, plus ≈ 92 Mo pour le processus Python et Chroma. Observations : latences à chaud quasi égales ; à froid, la machine à 16 Go paie la pagination ; le pipeline y tient tout juste, le palier plancher de 12 Go est exclu pour le régime retenu, et le palier de confort de 18 Go, sur un serveur épuré rendant environ 1,6 Go d'outils de développement, donne la marge nécessaire.

Étape 2 : la consigne de génération (10 et 11 août 2026)

Essais des variantes successives de la consigne système, notées V0 (la provisoire du pilote) à V3 (la retenue, section 10.5), sur les huit questions de mise au point, à régime modéré (température 0.3, seed 42, plafond de 512 jetons) ; réponses et latences archivées au dépôt (*resultats/consignes/*). Constats de régime : à température 1.0, réponses de 400 à 500 jetons en 100 à 160 s, parfois plus de 300 s ; un plafond de 256

jetons tronquait en pleine phrase, porté à 512 pour les essais puis 1 024 en production. Les réponses de ces essais ont servi le verdict sur les consignes ; leurs latences ne valent pas référence, une série perturbée par une mise en veille nocturne, l'autre par une surchauffe probable, d'où une mesure refaite à part, sur DANIELGARCIA au repos (V3 seul, 11.08) :

Mesure (11.08, DANIELGARCIA)	Valeur
Production (plafond 1 024, sans seed), question type, réponse courte	À froid 110,6 s ; à chaud, médiane 42,8 s (42,8 / 43,3 / 26,8), dont 23,0 s de génération
Essais (V3, plafond 512, seed 42), 8 questions, réponses souvent longues	Médiane 147,4 s (de 88,1 à 208,2 s)
RAM du pipeline chargé	26,2 / 27,7 Go (1,5 Go libre); empreinte = 11,3 Go

Ancrage vérifié à la source : les détails spécifiques des réponses V3 (SEFRI, délai de six à huit semaines, permis Ci, carte de légitimation DFAE, pays à accord d'établissement) ont été retrouvés dans les pages du corpus pilote. La réponse pilote du dépôt est régénérée en V3 ; la description de l'étape 1 reste un constat d'époque.

Étape 3 : le corpus complet (12 et 13 août 2026)

Découverte par sitemap, scraping et indexation sur DANIELGARCIA, veille désactivée.

Mesure	Valeur
Sitemap ge.ch : pages listées, puis retenues (démarches et dossiers)	27 484, puis 4 072
Sitemap geneve.ch : pages listées, puis retenues (dossier /demarches/)	31 218, puis 250
Manifeste final	4 322 adresses

Scraping	4 304 pages sur 4 322, ~2,4 h ; 18 exclusions (1 page supprimée, 17 accès restreints non contournés)
Indexation	9 651 fragments (2,24 par page), débit 0,8 fragment/s, base Chroma 190,9 Mo, dimension 1 024
Durées d'indexation	124,4 min (2 750 pages), puis reprise : 70,5 min (1 554 pages), ~3,3 h au total

Observations : métadonnées par fragment url, titre (extrait de la page), date de capture, position dans la page ; interruption réelle par mise en veille à 2 750 pages, reprise sans doublon ni manque, décompte final conforme au calcul déterministe ; contrôle qualitatif sur la question type : réponse fondée, sourcée, plus complète que sur le pilote, liens officiels cités dont une fiche de document non aspirée.

Étape 4 : l'interface et la latence perçue (23 août 2026)

Latence du pipeline de production (consigne V5, top_k 5) sur la question type, par bench_pipeline.py en deux modes, appel direct et streaming ; une passe à froid puis trois à chaud, médiane. Ces bancs rejouent la même question : le premier mot « en 1 à 3,5 s » relève probablement d'un cache de traitement du prompt et ne vaut pas pour une question nouvelle (voir Étape 5).

Question type (mode streaming)	GARCIAD	DANIELGARCIA
Premier mot (question rejouée, effet de cache)	1,0 s	3,5 s
Temps total à chaud (médiane)	124,7 s	80,2 s
Temps total à froid (premier appel)	237,9 s	175,4 s
RAM utilisée / libre	13,8 / 1,9 Go (sur 16)	~23 / 4-5 Go (sur 27,7)

En appel direct, les totaux à chaud sont équivalents (132,3 et 87,1 s) : le streaming n'ajoute aucun surcoût de calcul, les écarts entre passes venant du nombre de jetons,

non déterministe sans seed. Anomalie assumée sur DANIELGARCIA : premier mot et préparation plus lents malgré un processeur plus rapide, cause probable côté activité de fond, non surinterprétée. Effet de la consigne V5 : réponses plus courtes (~350 à 455 jetons) qu'en V4 (735), la liste « Sources » ayant été retirée. Empreinte des modèles inchangée : 11,3 Go. Captures de l'interface reprises à l'Annexe 3.

Étape 5 : le déploiement (23 au 25 août 2026)

Mesures de production sur le VPS (6 vCPU, 18 Go, Ubuntu 24.04, processeur seul), consignées au journal des mesures du dépôt.

Production (VPS, question type)	Valeur
Premier mot, question nouvelle	~87 s (prompt d'environ 2 000 jetons, prefill ~21-23 jetons/s)
Génération	~7-8 jetons/s
Même question immédiatement répétée (cache de prompt)	~2-3 s
RAM sous charge	~16 / 17 Go (~1 Go libre)

Dans le navigateur, l'attente ressentie avant le premier mot est d'environ deux minutes pour une question nouvelle. Ce cache de traitement explique aussi les valeurs flatteuses des bancs de développement de l'Étape 4, mesurées sur question rejouée. Après un incident de saturation provoqué par deux générations simultanées, une seule génération à la fois est autorisée. Après redémarrage du serveur, les services repartent seuls.

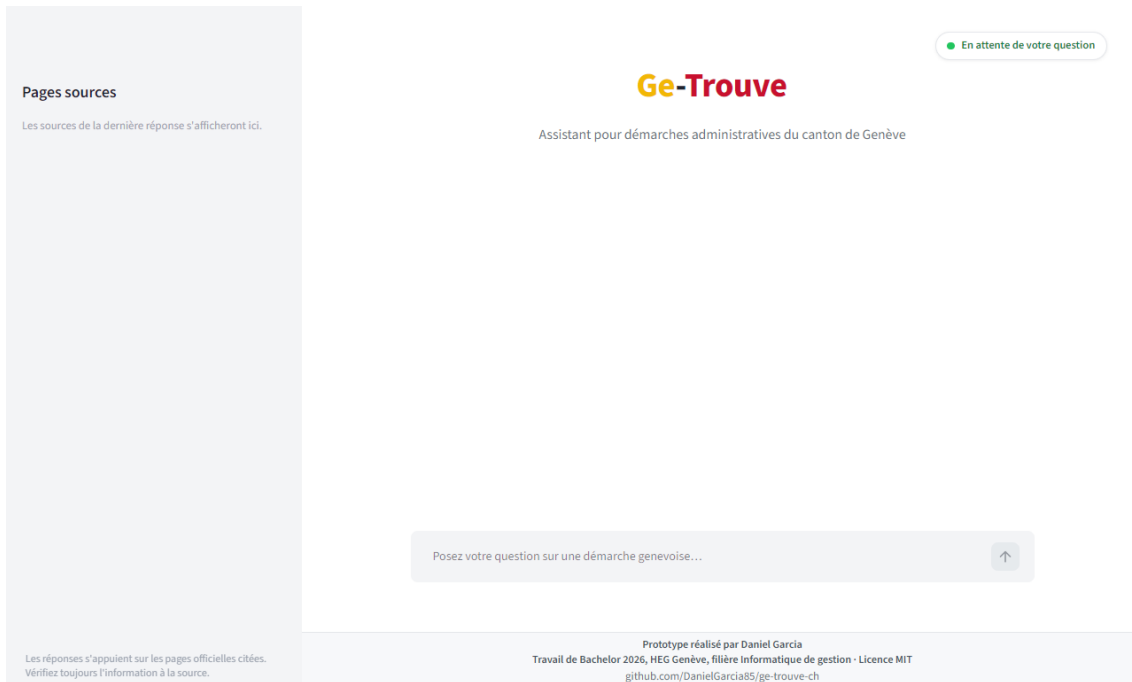
Étape 6 : le run d'évaluation (27 août 2026)

Banc exécuté sur DANIELGARCIA au repos : les 20 questions posées au pipeline de production figé, une passe chacune, sans seed. Durée totale ~66 min, soit ~198 s par question (de 116 à 342 s selon la longueur des réponses) ; premier appel à froid absorbé par un délai client relevé à 1 200 s. Notation RAGAS en série, le juge local ne supportant pas les appels concurrents sur processeur. Scores par question et agrégés archivés au dépôt (*resultats/evaluation/scores/*) et repris à l'Annexe 4 .

Annexe 3 : Captures de l'interface

L'interface Ge-Trouve (*src/app/app.py*), photographiée le 23 août 2026 sur le poste de développement.

La page au repos, avant toute question :



La page après la question type « Où déposer ma demande de permis de séjour ? » : la réponse s'écrit au fil de la génération dans la conversation, et la barre latérale affiche les pages sources de la réponse :

«

Pages sources

- <https://www.ge.ch/annoncer-mon-arrivee-ocpm/changement-canton-arrivee-provenance-autre-canton>
- <https://www.ge.ch/demander-permis-sejour-sans-activite-lucrative/demande-permis-ue/aele-recherche-emploi>
- <https://www.ge.ch/demander-permis-c/annees-sejour-permis-requis-faire-sa-demande>
- <https://www.ge.ch/demander-permis-sejour-ma-famille-regroupement-familial/je-suis-suisse>
- <https://www.ge.ch/annoncer-mon-arrivee-ocpm/annonce-arrivee-geneve-requerants-asile>

Les réponses s'appuient sur les pages officielles citées. Vérifiez toujours l'information à la source.

En attente de la prochaine question

Ge-Trouve

Assistant pour démarches administratives du canton de Genève

🗨️

Où déposer ma demande de permis de séjour ?

📄

La procédure pour déposer votre demande de permis de séjour dépend de votre situation spécifique :

Pour une demande de permis de séjour avec ou sans activité (incluant la recherche d'emploi) :
Vous pouvez déposer votre demande soit **en ligne**, soit **par courrier**.

- Pour une demande en ligne, vous devez disposer d'un [compte e-démarches de type simple](#).
- Pour les demandes par courrier, vous devez joindre le formulaire M dûment complété, daté et signé.

Pour une demande de regroupement familial : Votre demande peut être déposée soit **en ligne**, soit **par courrier**.

Pour une demande d'asile : La demande d'asile se dépose par oral ou par écrit auprès des lieux suivants :

- [L In nosta-frontière suisse](#)

Posez votre question sur une démarche genevoise...

↑

Prototype réalisé par Daniel Garcia

Travail de Bachelor 2026, HEG Genève, filière Informatique de gestion - Licence MIT

github.com/DanielGarcia85/ge-trouve-ch

Ge-Trouve – Assistant conversationnel RAG pour les démarches administratives genevoises
Daniel GARCIA

115

Annexe 4 : Résultats de l'évaluation

Cette annexe compile les résultats du run du 27 août 2026, archivés au dépôt du projet (*resultats/evaluation/*) (Garcia 2026). Lecture : les scores vont de 0 à 1 ; pour les seize questions avec réponse (AR), plus le score est haut, mieux c'est. Pour les quatre hors périmètre (HP), la lecture s'inverse, un score bas signalant un refus, le comportement attendu. Aucune moyenne globale n'est calculée : les zéros des refus corrects fausseraient tout.

N°	Question	Type	Précision	Rappel	Fidélité	Pertinence
1	Comment demander la naturalisation suisse à Genève ?	AR	1,00	1,00	1,00	0,79
2	Quels documents faut-il pour se marier civilement à Genève ?	AR	1,00	0,83	0,38	0,66
3	C'est où pour échanger mon permis de conduire étranger ?	AR	1,00	1,00	0,82	0,66
4	Comment inscrire mon enfant dans une crèche de la Ville de Genève ?	AR	0,33	0,80	0,73	0,66
5	Comment obtenir un subside pour mon assurance-maladie ?	AR	1,00	n/a	1,00	0,62
6	J'ai perdu mon travail : quelles démarches pour toucher le chômage à Genève ?	AR	0,80	0,75	0,61	0,71
7	Comment annoncer mon arrivée à Genève quand je viens d'un autre canton ?	AR	1,00	1,00	1,00	0,91

8	Quelles démarches après une naissance à Genève ?	AR	1,00	1,00	1,00	0,76
9	Comment demander une bourse d'études cantonale ?	AR	1,00	1,00	0,86	0,71
10	Comment fonctionne l'impôt à la source pour un titulaire de permis B ?	AR	1,00	1,00	0,83	0,63
11	Je déménage à l'intérieur du canton : je dois faire quoi ?	AR	0,89	1,00	0,83	0,67
12	Comment obtenir une attestation de résidence ?	AR	1,00	1,00	1,00	0,78
13	Comment contester une amende d'ordre reçue à Genève ?	AR	0,81	1,00	0,90	0,71
14	Quels sont les horaires et l'adresse de l'office cantonal de la population ?	AR	0,87	0,50	0,80	0,72
15	Comment inscrire mon enfant à l'école primaire à Genève ?	AR	1,00	1,00	1,00	0,80
16	Comment obtenir un permis de circulation pour mon véhicule ?	AR	0,95	1,00	0,87	0,60
17	Comment calculer ma future rente AVS ?	HP	0,50	1,00	0,71	0,63
18	Comment obtenir un permis de séjour dans le canton de Vaud ?	HP	0,00	1,00	0,00	0,00

19	Quels sont les horaires de la ligne 12 des TPG ?	HP	0,00	0,50	0,00	0,00
20	Comment obtenir un visa pour travailler à l'étranger ?	HP	0,00	1,00	0,50	0,00
Moyenne AR			0,92	0,93	0,85	0,71

Une réserve technique, consignée au journal. Q05 : échec technique du juge sur une métrique ; la moyenne du rappel est agrégée sur quinze questions. Les traces complètes, réponses et fragments, sont volumineuses : elles restent au dépôt (*resultats/evaluation/traces/*).

Le contrôle humain : cinq réponses relues à l'aveugle, avant toute lecture des scores, selon les cinq critères du protocole :

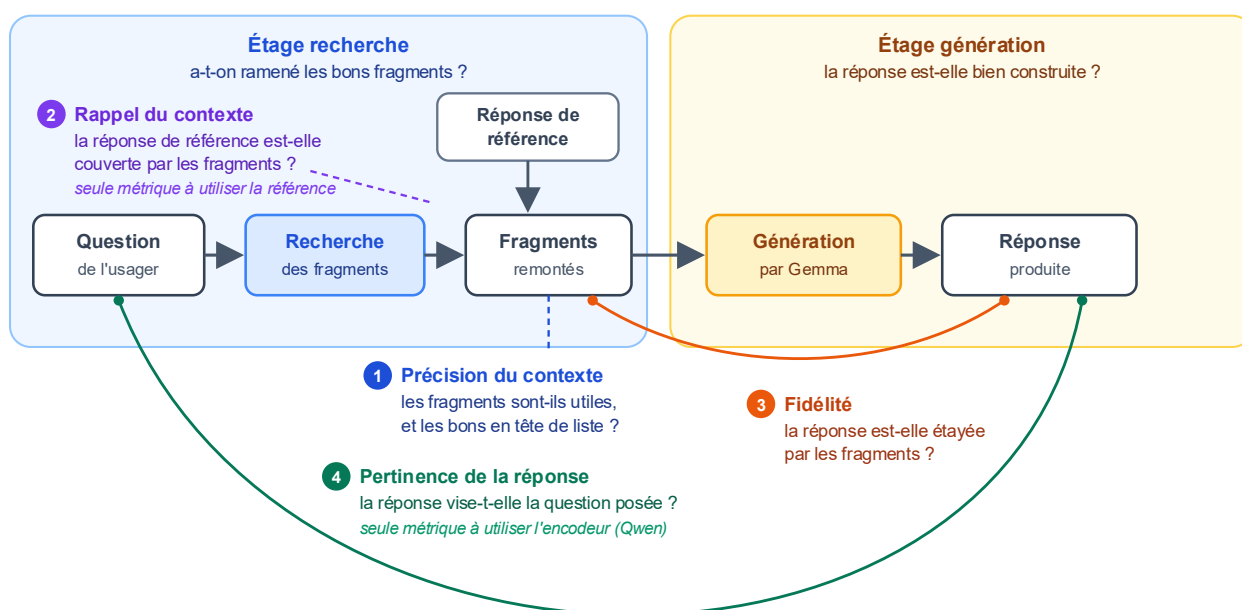
Question	Verdict
1 : naturalisation	Correcte et sourcée, mais incomplète ; une fusion, un permis Ci hors sujet.
5 : subside	Sans défaut.
8 : naissance	Exacte et fidèle, mais hors sujet : la section utile de la page n'a pas été extraite.
14 : coordonnées OCPM	Exacte, rien d'inventé ; l'absence des horaires n'est pas avouée.
17 : rente AVS	Hors périmètre : ni aveu, ni bonne orientation.

Le classement de ces défauts sur les sept points de Barnett et al. est repris à la section 12.4.

Annexe 5 : Le calcul des métriques RAGAS

Les quatre notes ne mesurent pas la même chose. Deux jugent la recherche : a-t-on ramené les bons fragments ? Deux jugent la génération : la réponse est-elle bien construite ? Toutes vont de 0 à 1. Les trois premières reposent sur le juge seul ; la quatrième mobilise aussi l'encodeur. Le schéma ci-dessous situe chaque note sur le flux, du côté où elle se branche ; le câblage des modèles reste celui de la Figure 12.1.

Chaque note se mesure à un endroit différent du flux : deux jugent la recherche, deux jugent la génération.



Précision du contexte (recherche). Elle répond à : **les fragments remontés sont-ils utiles, et bien classés ?** Le juge examine les fragments un par un et dit si chacun sert la réponse. La note pondère ensuite par le rang : un fragment utile en tête vaut plus qu'en fin de liste, un intrus fait d'autant plus mal qu'il est haut placé. C'est sa spécificité : elle note l'ordre, pas seulement le tri. Elle n'utilise pas la réponse de référence. Au run, la question 16 illustre le cas courant : 0,95, un lot presque pur dont l'élément le moins utile reste relégué en queue de liste.

Rappel du contexte (recherche). Elle répond à : **la recherche a-t-elle ramené tout ce qu'il fallait ?** La réponse de référence est découpée en affirmations ; pour chacune, le juge vérifie si les fragments la couvrent. La note est la fraction couverte. C'est la seule métrique qui utilise la référence, et la seule qui cherche ce qui manque plutôt que ce qui est en trop. La question 14 le montre : la référence citait l'adresse et les horaires de l'OCPM, les fragments ne portaient pas les horaires, le rappel tombe à 0,50.

Fidélité (génération). Elle répond à : [la réponse s'en tient-elle aux fragments, sans rien inventer ?](#) La réponse produite est découpée en affirmations ; pour chacune, le juge vérifie qu'elle s'appuie sur les fragments fournis. La note est la fraction étayée : c'est la mesure anti-invention. À ne pas confondre avec le rappel : le rappel découpe la référence et cherche des trous, la fidélité découpe la réponse et cherche des affirmations en l'air. Sa limite est apparue au run : une fusion reste étayée quand le passage voisin figure dans les fragments, le permis Ci de la question 1 a ainsi passé la note pleine. C'est cet angle mort que le contrôle humain couvre (section 12.3).

Pertinence de la réponse (génération). Elle répond à : [la réponse vise-t-elle la question posée ?](#) Le juge reconstruit, depuis la réponse produite, les questions auxquelles elle semble répondre ; l'encodeur compare ces questions reconstruites à la question réelle, par similarité de vecteurs ; la note est la proximité moyenne. Seule métrique à mobiliser l'encodeur. Une réponse verbeuse, qui déborde sur les cas voisins, engendre des questions reconstruites éloignées de celle de l'utilisateur : c'est le mécanisme derrière le 0,71 de la section 12.2, des réponses justes mais trop larges. Les quatre calculs ont tourné en local, juge Llama 3.1 8B et encodeur Qwen.